Automation Node Reference

DigiCRM · generated 2026-09-29 · 144 nodes

Account Created (trigger)
Account Updated (trigger)
Contact Created (trigger)
Contact Updated (trigger)
Lead Score Changed (trigger)
Lead Band Changed (trigger)
Lead Qualified (trigger)
Lead Disqualified (trigger)
Opportunity Created (trigger)
Opportunity Stage Changed (trigger)
Opportunity Won (trigger)
Opportunity Lost (trigger)
Opportunity Updated (trigger)
Opportunity Deleted (trigger)
Opportunity Assigned (trigger)
Task Created (trigger)
Task Completed (trigger)
Task Updated (trigger)
Task Status Changed (trigger)
Task Assigned (trigger)
Task Overdue (trigger)
Person Met at an Event (trigger)
Checked In to an Event (trigger)
Task Reopened (trigger)
Survey Response Received (trigger)
NPS Detractor Detected (trigger)
Survey Published (trigger)
Survey Archived (trigger)
Survey Response Started (trigger)
Ticket Created (trigger)
Ticket Status Changed (trigger)
SLA Breached (trigger)
Goal At Risk (trigger)
Goal Completed (trigger)
Campaign Started (trigger)
Campaign Completed (trigger)
Campaign Updated (trigger)
Campaign Scheduled (trigger)
Campaign Cancelled (trigger)
Campaign Goal Achieved (trigger)
Campaign Goal Missed (trigger)
Campaign Goal At Risk (trigger)
Goal Progress Updated (trigger)
Key Result Updated (trigger)
Goal Target Reached (trigger)
Key Result Target Reached (trigger)
KPI Threshold Breached (trigger)
Work Item Created (trigger)
Work Item Completed (trigger)
Work Item Status Changed (trigger)
Sprint Completed (trigger)
Booking Created (trigger)

Booking Cancelled (trigger)
Booking Rescheduled (trigger)
Booking No-Show (trigger)
Message Delivered (trigger)
Message Failed (trigger)
Unsubscription Submitted (trigger)
Contact Globally Unsubscribed (trigger)
Web Goal Conversion (trigger)
High Intent Visitor (trigger)
New Website / Chat Lead (trigger)
Link Clicked (trigger)
Custom Record Created (trigger)
Custom Record Updated (trigger)
Manual Trigger (trigger)
Scheduled (trigger)
When a number crosses a line (trigger)
Webhook Received (trigger)
Record Created (Any) (trigger)
Record Updated (Any) (trigger)
Custom Record Deleted (trigger)
Field Condition (condition)
Multi-Condition (condition)
Switch / Router (condition)
Formula condition (condition)
Switch on a formula (condition)
Update Field (action)
Create Record (action)
Assign Owner (action)
Change Status (action)
Add Tag (action)
Remove Tag (action)
Create Activity (action)
Create Task (action)
Update Task (action)
Complete Task (action)
Send Notification (action)
Send Email (action)
Send SMS (action)
Send WhatsApp (action)
Send Telegram Message (action)
Add to Campaign (action)
Remove from Campaign (action)
Create Ticket (action)
Escalate Ticket (action)
Send Survey (action)
Update Key Result (action)
Create Work Item (action)
Create Booking (action)
Create Tracked Link (action)
Change Custom Record Status (action)
Submit Custom Record for Approval (action)
Create Custom Record (action)
Update Custom Record (action)
Start Approval (action)
Ask a person (action)

Add a row to a spreadsheet (integration)
Read rows from a spreadsheet (integration)
Upload a file to OneDrive / SharePoint (integration)
Execute Tool (integration)
Delay / Wait (flow_control)
Loop / For Each (flow_control)
Error Handler (flow_control)
End (flow_control)
Not again so soon (flow_control)
Go To (flow_control)
Run Sub-Workflow (flow_control)
Wait Until (flow_control)
Wait for Event (flow_control)
Sticky Note (flow_control)
Run branches at once (flow_control)
Wait for branches (flow_control)
Set Variable (data_utility)
Data Transform (data_utility)
HTTP Request (data_utility)
Call API (data_utility)
Find Records (data_utility)
Aggregate values (data_utility)
Adjust time (data_utility)
Random number (data_utility)
Find or create (data_utility)
Enrich record (data_utility)
Log Message (data_utility)
Calculate (data_utility)
AI Generate Text (ai)
AI Fill fields (ai)
AI Router (ai)
AI Classify (ai)
AI Sentiment Analysis (ai)
AI Extract Data (ai)
AI Summarize (ai)
AI Prompt (ai)
AI Agent (Tool-using) (ai)

Automation Node Reference

Every step you can put on the automation canvas, documented the same way: what it is for, every setting, what it reads and produces, and what happens when it succeeds, fails, waits or runs twice. Pick a node in the menu, or read this overview first — the rules here apply to every node and each node page only adds what is specific to it.

72 Triggers | **5** Conditions | **34** Actions | **12** Flow control | **12** Data & utility | **9** AI

How a run starts and walks the canvas

A run starts in one of two ways. Events from the outbox, schedules, webhooks, the designer's **Test run** and the API run inline as soon as they arrive. Manual runs (Run now, a record's Run automation, a list's bulk bar) and sub-workflows that do not wait are **queued** and picked up by the worker within a second.

The engine then walks the canvas as a single ordered queue of steps. Each step runs at most once per scope (a loop body is its own scope), so there is no way to build an infinite cycle. Top-level branches run breadth-first, one after another — nothing runs concurrently, even after a Fork. The run is **Completed** when the queue is empty, **Waiting** while a Delay, approval or Wait Until holds it, and **Failed** when a step fails and nothing catches it.

A run's statuses: queued, running, waiting, completed, failed, cancelled. If the plan's run quota is used up the run is created as cancelled, not failed.

Where steps run

Steps that change CRM data, evaluate conditions, shape data or control the flow run inside the engine, in the run's own transaction, so their writes and audit rows commit together.

Steps that reach outside — sends (email, SMS, WhatsApp, Telegram, in-app), Integration Hub tools and spreadsheets, and the AI steps that produce a value — can run on **job workers** instead. With the workspace setting `automation/external_jobs` on, the engine writes a job, parks the run as Waiting and commits; a worker for that kind of job (send, tool, AI, each with its own concurrency ceiling) runs the very same step logic with the run's variables, then resumes the run with the result down the normal wires, or the On-error wire after the step's retries are used up. A slow provider no longer holds a run's transaction or a workflow worker.

Test runs, AI Router, AI Agent, HTTP Request and Call API always run inline. With the setting off (the default) everything runs inline as before.

Parallel waits. A job holds only its own branch. After a Fork, a branch that hands its step to a worker steps aside and the engine keeps running the other branches, queuing their jobs too — so two sends or two tool calls really are in flight together. A **Wait for branches** step set to *every branch* holds while any feeding branch is out on a job; each job result resumes just its branch (in its own loop iteration, if it was in one), and the run finishes when the last result is in. A job that fails past its retries takes its On-error wire, or with Stop on error on fails the run and cancels the run's other pending jobs.

Limits and priority

Nothing is capped by default. An admin sets **Automation limits** on the Execution Logs page (Usage & limits; workspace setting `automation/limits`): runs at once, runs started per minute, steps per run, AI steps per day, sends per day, and worker jobs in flight. A run that would pass the concurrency or per-minute cap is not refused: it is **queued**, and the run worker holds it until a slot frees. An AI or send step past its daily budget **fails with the reason** (its On-error wire, or Stop on error, decides what happens next); the step cap ends the run the way the engine's own ceiling does. Test runs are never counted or held.

The same page shows what the workspace used over the last 7 days (runs and steps by kind) and what is in flight right now, read straight from the run history.

Priority lane. Every run carries a priority the worker claims by, before age: a run a person is waiting on (Manual trigger, Run now) goes first, ordinary event runs next, bulk manual runs after them, and schedules / thresholds last. Workers still share each batch fairly between workspaces, so a busy workspace never starves a quiet one.

When a run waits on a person

Two steps park a run on a person: **Start Approval / Ask a person** (an approval request naming the approvers, with questions, choices, reminders and a deadline) and **Execute Tool** under a Hub policy that requires approval (a tool approval anyone with the Integrations permission may decide). Both used to have their own inbox and vocabulary, and tool approvals had no screen at all.

Automation → **Approvals** is now the one inbox: every request the signed-in person may act on, tagged with *why* (assigned to me, open to anyone, or — for workflow operators — assigned to others), with its due time and the record it is about. Approve / Reject decide plain requests in place; a request that asks questions or offers choices opens its own page. One decide call serves both engines, and each engine's own rules still apply (who may act, sequential steps, required answers, expiry).

The people named on a request are notified the moment it is created (before, only reminders and escalations reached anyone); a tool approval notifies the workflow's owner. The sidebar badge on Approvals counts what the person can act on now. Record reviews from the separate approval engine (Workflow inbox) are not runs and keep their own inbox.

Timers and event waits

Every sleeping step is its own **timer row**: a Delay, a Wait Until re-check, or the timeout of a Wait for Event. A run can therefore sleep in several branches at once (after a Fork, two Delays each wake their own branch; before, one clock served the whole run and the second Delay overwrote the first). The worker fires due timers every 15 seconds and continues only the branch that was waiting; a Join set to *every branch* holds until the sleeping branches arrive.

Wait for Event parks a run until something happens: pick a trigger (Contact updated, Ticket status changed, …), whether it must be about *this* record or any record, and optionally how long to wait. The event that arrives is saved under the step's output name (`{{variables.event.data.<field>}}`) and the run continues down the normal wires; a timeout takes the **If it times out** port. Only events the platform dispatches to triggers can be waited on. Test runs never wait: they carry on with an empty event.

Inside a For each. Every item reaches its Delay / Wait for Event / worker job and sleeps at the same time — items no longer wait one after another — and each wakes with its own item variable. The loop's **When done** branch holds until every item has finished, including the sleeping ones.

One formula language

Everything an automation works out, decides on or fills in with an expression uses the **calculated-fields formula engine** — the same functions, references and types as a calculated field: `{field}` and `{link.field}` for the record and its links, `{$trigger.x}` for what started the run, `{$var.name}` for an earlier step's answer; IF / AND / OR, date maths (WORKDAY, DATEDIFF), text and number functions; typed answers (number, currency, date, yes/no, text) and `#ERR` values that fail the step with a message instead of a silent 0.

Where it appears: **Calculate** (a value for later steps), **Formula condition** (yes / no), **Formula switch** (one port per answer), and any action setting written as `={…}`. Formulas are compiled when the workflow is saved or published (a broken one blocks publishing with its message) and every run is capped at a fixed number of formulas.

The two older evaluators are retired from the palette but keep running for workflows that use them: **Expression** (one `left OP right` comparison on resolved text) and the numbers-only **Formula** step. Structured condition rows (Field check, Multi-condition, trigger filters) stay as they are — they are the click-to-build form of the same comparisons.

Versions and runs already in flight

A new **version** is a new copy of the workflow (same code, next number); every run keeps the version it started on, so publishing a new version never changes a run that is queued, waiting or running — it finishes on its own graph. Editing a published workflow **in place** is different: it rewrites the very graph those runs use. A run that then wakes on a step that no longer exists **fails with the reason** ("Cannot resume: step(s) … no longer exist in v3 — the workflow was changed while this run was waiting") instead of ending quietly.

The designer's Version History shows how many runs are still in flight on each version. **Move waiting runs here** moves the queued and waiting runs of older versions onto the current one — only a run whose every remaining step (the one it waits on, its saved queue, its pending jobs, its error branch) exists in the current version; the others stay where they are and the message names the missing steps. Running runs are never moved.

Which wire is followed

- **Triggers, actions and data steps** follow every outgoing wire except On error.
- **Conditions** follow `true` or `false`. A true result with no true wire follows only wires that carry no port (a legacy node); on a node wired by port — say a lone false wire — the branch ends. A false result with no false wire ends the branch quietly.
- **Switch / router steps** take the first matching branch; no match, or a matched branch that is not wired, takes `default`.
- **Choice steps** (Ask a person, AI Router, Switch on a formula, Start Approval with choices) get one port per choice you type; the run follows the one that was picked.
- **On error** is a port every step has. Wire it to an Error Handler (or any step) to keep the run going after a failure.

Failure, retry and time limits

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

- A worker that stops responding for 2 minutes loses the run; another worker resumes it from the last successful step, re-running the one that was in flight.
- A run left **Running** with no sign of life for 30 minutes — no worker heartbeat and no step finished — is cancelled as stalled by the minute reaper. A long loop that keeps completing steps is left alone.

Transactions, duplicates and idempotency

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

- Each event starts at most **one** run per workflow: the run's idempotency key is `wfevt:<event id>:<workflow id>` (schedules use `sched:<workflow>:<minute>`). Manual runs get a random key and are never de-duplicated.
- The dispatcher allows at most **5 runs per workflow per record per minute** and stops automation chains (a run's writes triggering another workflow) at **depth 5**.
- An event the dispatcher fails to hand over is kept as a **dead letter** (Execution Logs → Needs attention) and replayed from there once the cause is fixed; a run quarantined after three failed retries is listed the same way.
- Whether re-running a step *duplicates* its effect is documented per node under **Idempotency**. As a rule: updates and tag changes are safe to repeat; creates and sends are not. Use Not again so soon to guard a send.

What a Test run does

- Steps that change CRM data or send messages are **simulated**: they return `would_have` with their fully resolved settings and write nothing.
- Delays do not wait; approvals are assumed approved; HTTP Request and Call API only really send GET requests; a sub-workflow child is a test run too.
- These run **for real** even in a test run: every AI step, Execute Tool, Add a row to a spreadsheet, Read rows, Upload a file, and record searches (which are read-only). Wait Until takes its met path without checking; Ask a person takes its first choice.
- A test run needs only `workflows.read`, is not metered, is not counted in run statistics and cannot be retried. Its step log is saved so you can inspect the resolved settings.

Data and variables

Any setting can read the run's data with `{{…}}` placeholders (the designer's **{ } Insert** menu lists what is available at that step). A missing value resolves to an empty string, a list or object comes out as text, and there are no filters or conditionals.

Declared variables (Settings → Variables) give a name a type up front — string / number / boolean / json, the same vocabulary as Set Variable — and a default. The run seeds it already in that shape (a Number's default reads as a real number from the first step, not the text you typed), and Validate refuses a workflow with two variables sharing a name, or a default that cannot be read as its declared type ("Variable 'count' is typed number but its default is not a number ('abc')").

A value written into a variable is capped in size: anything over 200 KB is replaced with a note when the run pauses (a full API response, a long AI answer), so one oversized value can't bloat the run's row on every wait. Smaller values, and everything while the run keeps going without pausing, are untouched.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions and identity

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

- `workflows.read` — browse, open runs and step logs, Test run (which makes real AI calls and GET requests).
- `workflows.run` plus visibility of the record — Run now / bulk run of a Manual trigger workflow.
- `workflows.manage` — retry and cancel runs. `workflows.publish` — publish. `workflows.run_as_system` — set Run as to system.
- Audit attribution uses who triggered the run; record scoping uses the Run as user (falling back to the owner, then the creator).

Audit and logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

- Nothing is redacted: resolved settings (including any header values) and response bodies land in the step log and in variables. Response headers drop only `set-cookie` and `authorization`.
- The run's **completed** event carries every run variable, so downstream consumers can read them.
- There is no automatic purge of runs or step logs.

Security

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

- HTTP Request and Call API refuse private, loopback and cloud-metadata addresses and never follow redirects.

- Marketing messages need the contact's opt-in; transactional ones do not. Compliance blocks are reported as skipped steps.
- Formulas and expressions are parsed against an allow-list — no arbitrary code runs.

TRIGGERS · accounts

Account Created

LIVE ENTRY

Fires when a new account is created

NODE TYPE	NODE KEY
Trigger	`trigger.crm.account_created`

MODULE	OUTPUT PORTS
accounts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a new company (account) is saved.

When to use

- Welcome sequence or onboarding task for every new customer account.
- Assign an owner by territory or industry as soon as the account exists.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only companies matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only companies matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Name	text
`industry`	Industry	text
`account_type`	Type	text
`status`	Status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **accounts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Only enterprise accounts

Anything else creates no run at all.

```
{
  "field_filters": [
    {
      "field": "account_type",
      "operator": "eq",
      "value": "enterprise"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · accounts

Account Updated LIVE ENTRY

Fires when an account field changes

NODE TYPE	NODE KEY
Trigger	`trigger.crm.account_updated`
MODULE	**OUTPUT PORTS**
accounts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a company's fields change.

When to use

- React to a change of industry, tier or status without reacting to every save — use **Only when these fields change**.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only when these fields change `watched_fields`	Field picker	No	—	
Only companies matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only when these fields change** `watched_fields`
- **Only companies matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`changes`	What changed (field → new value)	object

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a

value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **accounts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "watched_fields": "<Only when these fields change>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ `{{trigger.changes}}` is a map of field → new value; watched fields keep noise down.
- ✓ An update that names no fields (a restore, an owner hand-off) fires regardless of watched fields — the engine cannot prove it does not match.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

Contact Created LIVE ENTRY

Fires when a new contact is created

NODE TYPE	NODE KEY
Trigger	`trigger.crm.contact_created`
MODULE	**OUTPUT PORTS**
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a new contact is created (any source: form, import, API, manual).

When to use

- Send a welcome email or assign a follow-up task.
- Route new leads (contact type "lead") to a rep with Assign Owner.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only contacts matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only contacts matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`first_name`	First name	text
`last_name`	Last name	text
`email`	Email	text
`account_id`	Company	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Web form leads only

```
{
  "field_filters": [
    {
      "field": "source",
      "operator": "eq",
      "value": "web_form"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

Contact Updated LIVE ENTRY

Fires when a contact field changes

NODE TYPE	NODE KEY
Trigger	`trigger.crm.contact_updated`
MODULE	**OUTPUT PORTS**
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact's fields change.

When to use

- Re-score or re-route when the lifecycle stage, owner or email changes.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only when these fields change `watched_fields`	Field picker	No	—	
Only contacts matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only when these fields change** `watched_fields`
- **Only contacts matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`changes`	Fields that changed	list

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a

value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "watched_fields": "<Only when these fields change>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ `{{trigger.changes}}` lists the fields that changed. Use watched fields; a contact is saved often.
- ✓ An update that names no fields (a restore, an owner hand-off) fires regardless of watched fields.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

Lead Score Changed

LIVE ENTRY

Fires whenever a lead's score changes

NODE TYPE	NODE KEY
Trigger	`trigger.crm.lead_score_changed`
MODULE	**OUTPUT PORTS**
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact's lead score is recalculated to a new value.

When to use

- Notify the owner when a score jumps; log the change as an activity.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`contact_id`	Contact	id
`old_score`	Score before	number
`new_score`	New score	number
`old_band`	Band before	text
`band`	Band (hot / warm / cold)	text
`source`	What re-scored it	text
`lead_qualification`	Qualification	text
`owner_id`	Owner	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Fires on every change — prefer Lead Band Changed or Lead Qualified for actions with side effects.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

Lead Band Changed

LIVE ENTRY

Fires when a lead moves between cold, warm and hot

NODE TYPE	NODE KEY
Trigger	`trigger.crm.lead_band_changed`
MODULE	**OUTPUT PORTS**
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact moves between the cold, warm and hot bands.

When to use

- Hand hot leads to sales, drop cold ones into nurture.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`contact_id`	Contact	id
`old_score`	Score before	number
`new_score`	New score	number
`old_band`	Band before	text
`band`	Band (hot / warm / cold)	text
`source`	What re-scored it	text
`lead_qualification`	Qualification	text
`owner_id`	Owner	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Payload keys to branch on

```
{
  "note": "trigger.band = hot | warm | cold, trigger.old_band, trigger.new_score"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

Lead Qualified

LIVE ENTRY

Fires when a lead's score reaches the qualifying threshold

NODE TYPE	NODE KEY
Trigger	`trigger.crm.lead_qualified`
MODULE	**OUTPUT PORTS**
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact's score first reaches the qualifying threshold.

When to use

- Create the opportunity, notify the rep, add to a sales sequence.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`contact_id`	Contact	id
`old_score`	Score before	number
`new_score`	New score	number
`old_band`	Band before	text
`band`	Band (hot / warm / cold)	text
`source`	What re-scored it	text
`lead_qualification`	Qualification	text
`owner_id`	Owner	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

Lead Disqualified

LIVE ENTRY

Fires when a lead's score falls to the disqualifying threshold

NODE TYPE	NODE KEY
Trigger	`trigger.crm.lead_disqualified`
MODULE	**OUTPUT PORTS**
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact's score drops to the disqualifying threshold.

When to use

- Remove from active campaigns, tag for re-engagement later.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`contact_id`	Contact	id
`old_score`	Score before	number
`new_score`	New score	number
`old_band`	Band before	text
`band`	Band (hot / warm / cold)	text
`source`	What re-scored it	text
`lead_qualification`	Qualification	text
`owner_id`	Owner	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Created

LIVE ENTRY

Fires when a new opportunity is created

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_created`
MODULE	**OUTPUT PORTS**
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a new deal is created.

When to use

- Create the standard first tasks; notify the account owner.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only deals matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only deals matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Deal name	text
`amount`	Amount	text
`currency`	Currency	text
`account_id`	Company	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a

value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Stage Changed

LIVE ENTRY

Fires when an opportunity moves to a new stage

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_stage_changed`
MODULE	**OUTPUT PORTS**
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a deal moves to a different pipeline stage.

When to use

- Kick off proposal tasks when a deal reaches "Proposal"; alert a manager on "Negotiation".

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Moving from `from_stage`	Picker (opportunity_stage)	No	—	Leave blank for any stage
Moving to `to_stage`	Picker (opportunity_stage)	No	—	Leave blank for any stage
Only deals matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Moving from** `from_stage`
- **Moving to** `to_stage`
- **Only deals matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`from_stage`	Stage before	text
`to_stage`	New stage	text
`is_won`	Won	boolean
`is_lost`	Lost	boolean
`amount`	Amount	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Entering Negotiation from anywhere

```
{
  "from_stage": "",
  "to_stage": "negotiation"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "from_stage": "<Moving from>",
  "to_stage": "<Moving to>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Leave **Moving from** blank to catch any origin; set **Moving to** for the status you care about. Both blank fires on every change.

✓ The payload carries the old and new status — branch on `{{trigger.new_status}}` when one workflow serves several targets.

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Won

LIVE ENTRY

Fires when an opportunity is marked won

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_won`
MODULE	**OUTPUT PORTS**
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a deal is marked won.

When to use

- Create the onboarding project, notify finance, send a thank-you.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only deals matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only deals matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`from_stage`	Stage before	text
`to_stage`	New stage	text
`is_won`	Won	boolean
`is_lost`	Lost	boolean
`amount`	Amount	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Payload has `from_stage`, `to_stage`, `amount`, `is_won`.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Lost

LIVE ENTRY

Fires when an opportunity is marked lost

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_lost`

MODULE	OUTPUT PORTS
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a deal is marked lost.

When to use

- Schedule a win-back task in 90 days (Delay / Wait + Create Task).

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only deals matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only deals matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`from_stage`	Stage before	text
`to_stage`	New stage	text
`is_won`	Won	boolean
`is_lost`	Lost	boolean
`amount`	Amount	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Updated LIVE ENTRY

Fires when an opportunity is updated

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_updated`
MODULE	OUTPUT PORTS
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a deal is updated.

When to use

- Watch **amount** or **close_date** only, to avoid firing on every note.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only when these fields change `watched_fields`	Field picker	No	—	
Only deals matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only when these fields change** `watched_fields`
- **Only deals matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`changed_fields`	Fields sent	list
`name`	Deal name	text
`amount`	Amount	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "watched_fields": "<Only when these fields change>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ `{{trigger.changed_fields}}` lists the fields whose value actually changed, so watched fields do not fire on a save that changed nothing.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Deleted

LIVE ENTRY

Fires when an opportunity is deleted

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_deleted`
MODULE	OUTPUT PORTS
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a deal is deleted.

When to use

- Notify the owner; log for audit.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Deal name	text
`amount`	Amount	text
`stage_id`	Stage	id
`owner_id`	Owner	id
`account_id`	Company	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- The record is gone when the run starts — `{{record.*}}` cannot be read; use only the payload.
- No filters, for the same reason.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · opportunities

Opportunity Assigned

LIVE ENTRY

Fires when an opportunity is assigned

NODE TYPE	NODE KEY
Trigger	`trigger.crm.opportunity_assigned`
MODULE	**OUTPUT PORTS**
opportunities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a deal is given an owner.

When to use

- Notify the new owner; create their first-touch task.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only deals matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only deals matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Deal name	text
`owner_id`	New owner	id
`previous_owner_id`	Owner before	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **opportunities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Created

LIVE ENTRY

Fires when a task is created

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_created`
MODULE	**OUTPUT PORTS**
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a task is created.

When to use

- Notify the assignee on high-priority tasks; mirror tasks into a project.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tasks matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Title	text
`status`	Status	text
`priority`	Priority	text
`entity_type`	Linked record type	text
`entity_id`	Linked record	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Payload includes `entity_type` / `entity_id` — the record the task is linked to.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Completed

LIVE ENTRY

Fires when a task is marked complete

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_completed`
MODULE	**OUTPUT PORTS**
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a task is marked done.

When to use

- Create the next task in a sequence; update the linked deal's next step.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tasks matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Title	text
`entity_type`	Linked record type	text
`entity_id`	Linked record	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Updated

LIVE ENTRY

Fires when a task is updated

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_updated`
MODULE	**OUTPUT PORTS**
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a task is updated.

When to use

- Watch `due_date` or `assignee` only.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only when these fields change `watched_fields`	Field picker	No	—	
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only when these fields change** `watched_fields`
- **Only tasks matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Title	text
`status`	Status	text
`priority`	Priority	text
`changed_fields`	Fields changed	list

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "watched_fields": "<Only when these fields change>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ `{{trigger.changed_fields}}` lists the fields whose value actually changed, so a full-record save from the drawer only matches the watched fields it really changed.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Status Changed

LIVE ENTRY

Fires when a task status changes

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_status_changed`

MODULE	OUTPUT PORTS
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a task moves between statuses.

When to use

- Alert when a task becomes **blocked**.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Moving from `from_status`	Picker (task_status)	No	—	Leave blank for any status
Moving to `to_status`	Picker (task_status)	No	—	Leave blank for any status
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Moving from** `from_status`
- **Moving to** `to_status`
- **Only tasks matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Title	text
`old_status`	Status before	text
`new_status`	New status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "from_status": "<Moving from>",
  "to_status": "<Moving to>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Leave **Moving from** blank to catch any origin; set **Moving to** for the status you care about. Both blank fires on every change.

✓ The payload carries the old and new status — branch on `{{trigger.new_status}}` when one workflow serves several targets.

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Assigned

LIVE ENTRY

Fires when a task is assigned to a user

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_assigned`
MODULE	**OUTPUT PORTS**
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a task is assigned to someone.

When to use

- Notify the assignee with a link.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tasks matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`user_id`	Assigned to	id
`is_primary`	Primary assignee	boolean

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Overdue LIVE ENTRY

Fires when a task passes its due date

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_overdue`
MODULE	**OUTPUT PORTS**
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a task passes its due date.

When to use

- Escalate to the manager; bump the priority.

How it works

1. A sweep every 15 minutes finds open tasks whose due date has passed and writes one event per task per due date — a task fires once for its current due date, and again if the date is moved to another past date. `{{trigger.overdue_days}}` says how late it is.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tasks matching** `field_filters`

- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Title	text
`due_date`	Was due	text
`priority`	Priority	text
`assignee_id`	Assignee	id
`entity_type`	Linked record type	text
`entity_id`	Linked record	id
`overdue_days`	Days overdue	number

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Reaction is up to 15 minutes after the due time.
- ✓ Done and cancelled tasks never fire.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · events

Person Met at an Event LIVE ENTRY

Fires when someone is added to an event: the staff app, a quick form, a booking or survey from an event QR code

NODE TYPE	NODE KEY
Trigger	`trigger.events.person_captured`
MODULE	**OUTPUT PORTS**
events (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when someone is added to an event.

When to use

- Send the brochure the same day; create a follow-up task for **hot** leads.

How it works

1. Build it on the **Contact** record type: it runs on the contact the event met. It fires once per person per event, from the staff app, a quick form, or a booking or survey opened from an event QR code. `{{trigger.event_name}}`, `{{trigger.interest}}` and `{{trigger.is_new_contact}}` say which event and how warm.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only this event (name) `event_name`	Text	No	—	e.g. `Leave empty for every event`
Only this interest `interest`	Choice	No	—	Options: `hot` = Hot `warm` = Warm `cold` = Cold

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only this event (name)** `event_name`

- **Only this interest** `interest`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`contact_id`	Contact	id
`event_name`	Event	text
`event_id`	Event id	id
`event_kind`	Hosting or attending	text
`status`	Status	text
`how`	How they came in	text
`interest`	Interest (hot / warm / cold)	text
`interests`	Interested in	list
`is_new_contact`	New contact	boolean
`captured_by`	Captured by	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **events** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "event_name": "Leave empty for every event",
  "interest": "hot"
}
```

Best practices

- ✓ A second touch at the same event (say, a booking after the booth chat) does not fire again.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · events

Checked In to an Event

LIVE ENTRY

Fires when a registered person is checked in

NODE TYPE	NODE KEY
Trigger	`trigger.events.person_checked_in`

MODULE	OUTPUT PORTS
events (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a registered person is checked in to an event.

When to use

- Send the welcome pack; tell the account owner their customer arrived.

How it works

1. Build it on the **Contact** record type.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only this event (name) `event_name`	Text	No	—	e.g. `Leave empty for every event`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only this event (name)** `event_name`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`contact_id`	Contact	id
`event_name`	Event	text
`event_id`	Event id	id
`event_kind`	Hosting or attending	text
`status`	Status	text
`how`	How they came in	text
`interest`	Interest (hot / warm / cold)	text
`interests`	Interested in	list
`is_new_contact`	New contact	boolean
`captured_by`	Captured by	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **events** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "event_name": "Leave empty for every event"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · tasks

Task Reopened

LIVE ENTRY

Fires when a completed task is reopened

NODE TYPE	NODE KEY
Trigger	`trigger.tasks.task_reopened`
MODULE	**OUTPUT PORTS**
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a completed task is reopened.

When to use

- Notify the original assignee.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tasks matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tasks matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Title	text
`old_status`	Status before	text
`new_status`	New status	text
`entity_type`	Linked record type	text
`entity_id`	Linked record	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · surveys

Survey Response Received

LIVE ENTRY

Fires when a survey response is submitted

NODE TYPE	NODE KEY
Trigger	`trigger.surveys.response_received`
MODULE	**OUTPUT PORTS**
surveys (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a survey response is submitted — identified or anonymous.

When to use

- Thank the respondent, create a follow-up task for low scores, tag the contact by answer.

How it works

1. The survey module emits the event against the **survey**, so build the workflow on the Survey entity.
2. The respondent's contact, when identified, is `{{trigger.contact_id}}` (null for anonymous responses); `{{trigger.is_anonymous}}` says which. Anonymous responses fire too — branch on it.
3. Optional **Survey** narrows to one survey.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Survey `survey_id`	Record picker (survey)	No	—	Leave blank for any survey

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Survey** `survey_id`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`survey_id`	Survey	id
`survey_name`	Survey name	text
`submission_id`	Response	id
`respondent_email`	Respondent email	text
`contact_id`	Contact	id
`answer_count`	Answers given	number
`is_anonymous`	Anonymous	boolean

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **surveys** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "survey_id": "<survey id>"
}
```

Best practices

- ✓ Answers are not in the payload — use `{{trigger.submission_id}}` with a Find Records / Call API step if you need them.
- ✓ To act on the contact only, use NPS Detractor Detected for detractors, or add a condition on `{{trigger.is_anonymous}}`.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- The run's record is the survey, not the contact: steps that act on "the record that started this run" act on the survey. Use `{{trigger.contact_id}}` (Send to contact, a Find Records step) to reach the person.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · surveys

NPS Detractor Detected

LIVE ENTRY

Fires when NPS score is 0-6. Build this workflow on the Contact entity — it runs against the respondent, not the survey, and only for identified respondents.

NODE TYPE	NODE KEY
Trigger	`trigger.surveys.nps_detractor`

MODULE	OUTPUT PORTS
surveys (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when an identified respondent gives an NPS score of 0–6.

When to use

- Open a ticket or task so someone calls the detractor within a day.
- Tag the contact and pause marketing.

How it works

1. Emitted against the respondent's contact (record type Contact).
2. Only identified respondents; anonymous detractors never fire.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Survey `survey_id`	Record picker (survey)	No	—	Leave blank for any survey

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Survey** `survey_id`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`survey_id`	Survey	id
`survey_name`	Survey name	text
`nps_score`	NPS score	number
`contact_id`	Contact	id
`respondent_email`	Respondent email	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **surveys** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Payload keys

```
{
  "note": "trigger.nps_score, trigger.survey_name, trigger.contact_id, trigger.respondent_email"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "survey_id": "<survey id>"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Anonymous responses do not fire.
- Workflow must be built on the Contact entity.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · surveys

Survey Published

LIVE ENTRY

Fires when a survey is published

NODE TYPE	NODE KEY
Trigger	`trigger.surveys.survey_published`
MODULE	**OUTPUT PORTS**
surveys (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a survey is published.

When to use

- Announce it to a team; log the launch.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Survey name	text
`old_status`	Status before	text
`new_status`	New status	text
`short_url`	Short link	text
`public_url`	Public link	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **surveys** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · surveys

Survey Archived

LIVE ENTRY

Fires when a survey is archived

NODE TYPE	NODE KEY
Trigger	`trigger.surveys.survey_archived`
MODULE	**OUTPUT PORTS**
surveys (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a survey is archived.

When to use

- Clean-up tasks; notify the owner.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Survey name	text
`type`	Survey type	text
`status_before_archive`	Status before	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **surveys** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · surveys

Survey Response Started

LIVE ENTRY

Fires when a respondent starts a survey

NODE TYPE	NODE KEY
Trigger	`trigger.surveys.response_started`
MODULE	**OUTPUT PORTS**
surveys (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a respondent opens a survey and starts answering.

When to use

- Remind them later if no response arrives (Delay + Find Records).

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Survey `survey_id`	Record picker (survey)	No	—	Leave blank for any survey

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Survey** `survey_id`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`survey_id`	Survey	id
`survey_name`	Survey name	text
`submission_id`	Response	id
`respondent_email`	Respondent email	text
`is_anonymous`	Anonymous	boolean
`last_page_completed`	Last page completed	number

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a

value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **surveys** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "survey_id": "<survey id>"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · service_desk

Ticket Created

LIVE ENTRY

Fires when a support ticket is created

NODE TYPE	NODE KEY
Trigger	`trigger.service_desk.ticket_created`
MODULE	**OUTPUT PORTS**
service_desk (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a support ticket is created (email, chat, form, API or by hand).

When to use

- Auto-acknowledge the requester; route by category or priority; assign by round robin.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tickets matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tickets matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`ticket_number`	Ticket number	text
`subject`	Subject	text
`priority`	Priority	text
`source`	Source	text
`category`	Category	text
`requester_contact_id`	Requester	id
`account_id`	Company	id
`queue_id`	Queue	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **service_desk** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Urgent tickets only

```
{
  "field_filters": [
    {
      "field": "priority",
      "operator": "in",
      "value": "urgent,critical"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · service_desk

Ticket Status Changed

LIVE ENTRY

Fires when a ticket status changes

NODE TYPE	NODE KEY
Trigger	`trigger.service_desk.ticket_status_changed`
MODULE	**OUTPUT PORTS**
service_desk (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a ticket moves between statuses.

When to use

- Send a CSAT survey when a ticket is resolved; alert when one is reopened.

How it works

1. Dispatched inline from the ticket service with the status filter applied at the source, so a non-matching change costs nothing.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Moving from `from_status`	Picker (ticket_status)	No	—	Leave blank for any status
Moving to `to_status`	Picker (ticket_status)	No	—	Leave blank for any status
Only tickets matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Moving from** `from_status`
- **Moving to** `to_status`
- **Only tickets matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`old_status`	Status before	text
`to_status`	New status	text
`ticket_number`	Ticket number	text
`subject`	Subject	text
`requester_contact_id`	Requester	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **service_desk** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Resolved → survey

```
{
  "from_status": "",
  "to_status": "resolved"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "from_status": "<Moving from>",
  "to_status": "<Moving to>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Leave **Moving from** blank to catch any origin; set **Moving to** for the status you care about. Both blank fires on every change.
- ✓ The payload carries the old and new status — branch on `{{trigger.new_status}}` when one workflow serves several targets.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · service_desk

SLA Breached

LIVE ENTRY

Fires when a ticket breaches its SLA

NODE TYPE	NODE KEY
Trigger	`trigger.service_desk.sla_breached`
MODULE	**OUTPUT PORTS**
service_desk (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a ticket misses its first-response or resolution SLA.

When to use

- Escalate to tier 2, notify the queue manager, bump priority.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only tickets matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only tickets matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`ticket_number`	Ticket number	text
`breach_type`	Breached (first_response / resolution)	text
`target_minutes`	Target (minutes)	number
`actual_minutes`	Actual (minutes)	number
`assigned_to`	Assignee	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **service_desk** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ `{{trigger.breach_type}}` is first_response or resolution; target and actual minutes are in the payload.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

Goal At Risk

LIVE ENTRY

Fires when a goal's status moves to at_risk or off_track

NODE TYPE	NODE KEY
Trigger	`trigger.goals.at_risk`
MODULE	**OUTPUT PORTS**
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a goal's status moves to at risk or off track.

When to use

- Notify the owner and their manager; create a review task.

How it works

1. Dispatched inline from the goal service on the status transition; the workflow's record type is Goal.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_id`	Goal	id
`title`	Goal title	text
`old_status`	Status before	text
`new_status`	New status	text
`progress_percentage`	Progress %	number
`owner_reference_id`	Owner	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

Goal Completed

LIVE ENTRY

Fires when a goal's status moves to completed

NODE TYPE	NODE KEY
Trigger	`trigger.goals.completed`

MODULE	OUTPUT PORTS
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a goal's status moves to completed.

When to use

- Celebrate in a channel; close related tasks.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_id`	Goal	id
`title`	Goal title	text
`old_status`	Status before	text
`new_status`	New status	text
`progress_percentage`	Progress %	number
`owner_reference_id`	Owner	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Started

LIVE ENTRY

Fires when a campaign is launched

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.campaign_started`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign is launched.

When to use

- Notify the team; create the reporting task.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only campaigns matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only campaigns matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Campaign	text
`campaign_type`	Type	text
`old_status`	Status before	text
`new_status`	New status	text
`segment_id`	Segment	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Completed LIVE ENTRY

Fires when a campaign finishes

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.campaign_completed`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign finishes.

When to use

- Create the post-mortem task; pull results into a sheet.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only campaigns matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only campaigns matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Campaign	text
`campaign_type`	Type	text
`old_status`	Status before	text
`new_status`	New status	text
`segment_id`	Segment	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Updated LIVE ENTRY

Fires when a campaign is updated

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.campaign_updated`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign is updated.

When to use

- Watch **budget** or **schedule** only.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only when these fields change `watched_fields`	Field picker	No	—	
Only campaigns matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only when these fields change** `watched_fields`
- **Only campaigns matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Campaign	text
`campaign_type`	Type	text
`changed_fields`	Fields sent	list
`old_status`	Status before	text
`new_status`	Status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "watched_fields": "<Only when these fields change>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ `{{trigger.changed_fields}}` lists the fields whose value actually changed (status moves have their own triggers), so the wizard's whole-record saves only match what they really changed.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Scheduled

LIVE ENTRY

Fires when a campaign is scheduled for sending

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.campaign_scheduled`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign is scheduled for sending.

When to use

- Request approval before the send date.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only campaigns matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only campaigns matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Campaign	text
`campaign_type`	Type	text
`old_status`	Status before	text
`new_status`	New status	text
`segment_id`	Segment	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Cancelled

LIVE ENTRY

Fires when a campaign is cancelled

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.campaign_cancelled`

MODULE	OUTPUT PORTS
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign is cancelled.

When to use

- Notify stakeholders.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only campaigns matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only campaigns matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Campaign	text
`campaign_type`	Type	text
`old_status`	Status before	text
`new_status`	New status	text
`segment_id`	Segment	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field_filters": [],
  "logic": "and"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Goal Achieved

LIVE ENTRY

Fires when a campaign goal's measured value crosses its target

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.goal_achieved`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign goal's measured value crosses its target.

When to use

- Announce it; stop spend on a campaign that hit its goal.

How it works

1. Measured by the 15-minute goal tick and dispatched inline from it. The workflow's record is the **campaign**; the goal rides in the payload (`goal_name` , `current_value` , `target_value` , `progress_pct` , `is_primary`).

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_name`	Goal	text
`current_value`	Current value	text
`target_value`	Target	text
`progress_pct`	Progress %	text
`is_primary`	Primary goal	boolean
`campaign_id`	Campaign	id
`campaign_name`	Campaign	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Goal Missed

LIVE ENTRY

Fires when a campaign's window closes without the goal being met

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.goal_missed`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign's window closes without the goal being met.

When to use

- Create a review task with the numbers.

How it works

1. Measured by the 15-minute goal tick; the record is the campaign, the goal is in the payload.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_name`	Goal	text
`current_value`	Current value	text
`target_value`	Target	text
`progress_pct`	Progress %	text
`is_primary`	Primary goal	boolean
`campaign_id`	Campaign	id
`campaign_name`	Campaign	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions

VARIABLE	WHAT IT IS	AVAILABLE
		`record.` .
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record` .
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read` ; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · campaigns

Campaign Goal At Risk

LIVE ENTRY

Fires once per episode when a goal falls well behind its pacing mid-window

NODE TYPE	NODE KEY
Trigger	`trigger.campaigns.goal_at_risk`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a campaign goal falls well behind its pacing mid-window (once per episode).

When to use

- Alert the campaign owner.

How it works

1. Fires once when the goal falls behind and not again until it recovers and falls behind again.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_name`	Goal	text
`current_value`	Current value	text
`target_value`	Target	text
`progress_pct`	Progress %	text
`is_primary`	Primary goal	boolean
`campaign_id`	Campaign	id
`campaign_name`	Campaign	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions

VARIABLE	WHAT IT IS	AVAILABLE
		`record.` .
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record` .
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read` ; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

Goal Progress Updated

LIVE ENTRY

Fires when goal progress changes

NODE TYPE	NODE KEY
Trigger	`trigger.goals.goal_progress_updated`
MODULE	OUTPUT PORTS
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a goal's progress percentage actually changes.

When to use

- Mirror progress into a spreadsheet; notify the owner on a big jump.

How it works

1. Fires from every path that moves a goal's progress — a key-result edit or value update, a check-in, a child goal rolling up, the 30-minute metric recompute — only when the percentage really changed. Build the workflow on the **Goal** entity.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Goal title	text
`old_progress`	Progress before (%)	number
`new_progress`	New progress (%)	number
`status`	Status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Metric-driven goals recompute every 30 minutes, so this can fire often — pair it with Not again so soon or a condition on `{{trigger.new_progress}}`.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

Key Result Updated

LIVE ENTRY

Fires when a key result value changes

NODE TYPE	NODE KEY
Trigger	`trigger.goals.kr_value_updated`
MODULE	OUTPUT PORTS
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a key result's value changes.

When to use

- Recompute a dependent KPI; log the movement.

How it works

1. Fires on every real value change — an edit, a value update, a contribution — and runs on the key result's **goal** (build the workflow on the Goal entity); `{{trigger.key_result_id}}` names the key result and `{{trigger.source}}` says what changed it.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_id`	Goal	id
`key_result_id`	Key result	id
`title`	Key result	text
`old_value`	Value before	number
`new_value`	New value	number
`progress_percentage`	Progress (%)	number
`source`	What changed it	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

Goal Target Reached

LIVE ENTRY

Fires when a goal reaches its target

NODE TYPE	NODE KEY
Trigger	`trigger.goals.goal_target_reached`
MODULE	**OUTPUT PORTS**
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a goal crosses 100 % of its target.

When to use

- Notify stakeholders; close the goal's open tasks.

How it works

1. Fires once, on the crossing from below 100 % to 100 % or more; staying above does not fire again. Build the workflow on the **Goal** entity.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	Goal title	text
`progress`	Progress (%)	number
`target_value`	Target	text
`current_value`	Value now	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

Key Result Target Reached

LIVE ENTRY

Fires when a key result reaches its target value

NODE TYPE	NODE KEY
Trigger	`trigger.goals.kr_target_reached`
MODULE	OUTPUT PORTS
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a key result crosses 100 % of its target.

When to use

- Notify the owner; celebrate in a channel.

How it works

1. Fires once on the crossing and runs on the key result's **goal** (build the workflow on the Goal entity); the key result is `{{trigger.key_result_id}}`.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`goal_id`	Goal	id
`key_result_id`	Key result	id
`title`	Key result	text
`new_value`	Value now	number
`target_value`	Target	number

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · goals

KPI Threshold Breached

LIVE ENTRY

Fires when a KPI crosses its threshold

NODE TYPE	NODE KEY
Trigger	`trigger.goals.kpi_threshold_breached`
MODULE	**OUTPUT PORTS**
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a KPI crosses its warning or critical threshold.

When to use

- Alert operations with the old and new value.

How it works

1. Dispatched inline from the goal service. Build the workflow on the **Goal** entity; the run's record is the KPI. The payload also carries `kpi_id`, `kpi_code`, `linked_goal_id`, `warning_threshold`, `critical_threshold` and `direction_type`.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`title`	KPI	text
`old_value`	Value before	number
`new_value`	New value	number
`severity`	Severity (warning / critical)	text
`threshold_value`	Threshold	number

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched at the source with config-aware matching.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ `{{trigger.severity}}` is warning or critical.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · projects

Work Item Created

LIVE ENTRY

Fires when a project work item is created

NODE TYPE	NODE KEY
Trigger	`trigger.projects.work_item_created`
MODULE	OUTPUT PORTS
projects (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a project work item is created.

When to use

- Notify the project lead.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`key`	Work item key	text
`title`	Title	text
`item_type`	Type	text
`project_id`	Project	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's	Always.

VARIABLE	WHAT IT IS	AVAILABLE
	Outputs).	
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **projects** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · projects

Work Item Completed

LIVE ENTRY

Fires when a work item is completed

NODE TYPE	NODE KEY
Trigger	`trigger.projects.work_item_completed`
MODULE	OUTPUT PORTS
projects (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a work item is completed.

When to use

- Move a linked deal or ticket forward.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`key`	Work item key	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **projects** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · projects

Work Item Status Changed

LIVE ENTRY

Fires when a work item status changes

NODE TYPE	NODE KEY
Trigger	`trigger.projects.work_item_status_changed`

MODULE	OUTPUT PORTS
projects (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a work item moves between statuses.

When to use

- Alert on **blocked**.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Moving from `from_status`	Picker (work_item_status)	No	—	Leave blank for any status
Moving to `to_status`	Picker (work_item_status)	No	—	Leave blank for any status

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Moving from** `from_status`
- **Moving to** `to_status`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`key`	Work item key	text
`old_status`	Status before	text
`new_status`	New status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **projects** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "from_status": "<Moving from>",
  "to_status": "<Moving to>"
}
```

Best practices

- ✓ Leave **Moving from** blank to catch any origin; set **Moving to** for the status you care about. Both blank fires on every change.
- ✓ The payload carries the old and new status — branch on `{{trigger.new_status}}` when one workflow serves several targets.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · projects

Sprint Completed LIVE ENTRY

Fires when a sprint is completed

NODE TYPE	NODE KEY
Trigger	`trigger.projects.sprint_completed`
MODULE	OUTPUT PORTS
projects (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a sprint is completed.

When to use

- Create the retro task; email a summary with the velocity.

How it works

1. Build the workflow on the **Sprint** entity; `{{trigger.velocity}}` is the points completed and `{{trigger.project_id}}` the project. Unfinished work items are moved off the sprint before it fires.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`name`	Sprint name	text
`velocity`	Points completed	number
`project_id`	Project	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **projects** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · calendar

Booking Created

LINKED ENTRY

Fires when a booking is created

NODE TYPE	NODE KEY
Trigger	`trigger.calendar.booking_created`
MODULE	**OUTPUT PORTS**
calendar (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a booking is made on a booking page.

When to use

- Confirm by email or WhatsApp; create the prep task for the host.

How it works

1. Fires through an explicit **event type** → **workflow** link on the booking event type, not by matching every booking — see the dispatch badge.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`booking.invitee_name`	Invitee name	text
`booking.invitee_email`	Invitee email	text
`booking.start_datetime`	Starts at	text
`booking.end_datetime`	Ends at	text
`booking.timezone`	Time zone	text
`booking.status`	Booking status	text
`booking.title`	Title	text
`booking.host_user_id`	Host	id
`booking.id`	Booking	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **calendar** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires when the record is linked to this workflow (e.g. an event type → workflow link).

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Payload keys are under `booking.` — e.g. `{{trigger.booking.invitee_email}}`, `{{trigger.booking.start_datetime}}`.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · calendar

Booking Cancelled

LINKED ENTRY

Fires when a booking is cancelled

NODE TYPE	NODE KEY
Trigger	`trigger.calendar.booking_cancelled`
MODULE	**OUTPUT PORTS**
calendar (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a booking is cancelled.

When to use

- Offer a rebooking link.

How it works

1. Fires through the event type → workflow link.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`booking.invitee_name`	Invitee name	text
`booking.invitee_email`	Invitee email	text
`booking.start_datetime`	Starts at	text
`booking.end_datetime`	Ends at	text
`booking.timezone`	Time zone	text
`booking.status`	Booking status	text
`booking.title`	Title	text
`booking.host_user_id`	Host	id
`booking.id`	Booking	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's	Always.

VARIABLE	WHAT IT IS	AVAILABLE
	Outputs).	
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **calendar** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires when the record is linked to this workflow (e.g. an event type → workflow link).

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · calendar

Booking Rescheduled

LINKED ENTRY

Fires when a booking is rescheduled

NODE TYPE	NODE KEY
Trigger	`trigger.calendar.booking_rescheduled`
MODULE	**OUTPUT PORTS**
calendar (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a booking is moved.

When to use

- Re-send the confirmation with the new time.

How it works

1. Fires through the event type → workflow link.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`booking.invitee_name`	Invitee name	text
`booking.invitee_email`	Invitee email	text
`booking.start_datetime`	Starts at	text
`booking.end_datetime`	Ends at	text
`booking.timezone`	Time zone	text
`booking.status`	Booking status	text
`booking.title`	Title	text
`booking.host_user_id`	Host	id
`booking.id`	Booking	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's	Always.

VARIABLE	WHAT IT IS	AVAILABLE
	Outputs).	
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **calendar** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires when the record is linked to this workflow (e.g. an event type → workflow link).

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · calendar

Booking No-Show

LINKED ENTRY

Fires when a booking is marked as no-show

NODE TYPE	NODE KEY
Trigger	`trigger.calendar.booking_no_show`
MODULE	**OUTPUT PORTS**
calendar (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a booking is marked as a no-show.

When to use

- Send a "sorry we missed you" message; create a follow-up task.

How it works

1. Fires through the event type → workflow link.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`booking.invitee_name`	Invitee name	text
`booking.invitee_email`	Invitee email	text
`booking.start_datetime`	Starts at	text
`booking.end_datetime`	Ends at	text
`booking.timezone`	Time zone	text
`booking.status`	Booking status	text
`booking.title`	Title	text
`booking.host_user_id`	Host	id
`booking.id`	Booking	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's	Always.

VARIABLE	WHAT IT IS	AVAILABLE
	Outputs).	
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **calendar** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires when the record is linked to this workflow (e.g. an event type → workflow link).

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · omnichannel

Message Delivered

LIVE ENTRY

Fires when a message is delivered

NODE TYPE	NODE KEY
Trigger	`trigger.messaging.message_delivered`
MODULE	**OUTPUT PORTS**
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when an outbound message to a known contact is reported delivered by the provider.

When to use

- Log the delivery as an activity; advance a sequence.

How it works

1. Fires from the delivery-status update (provider webhook or the send worker) when the attempt reaches **delivered**. The workflow runs on the **contact** the message was sent to; a send with no contact (a raw address) starts nothing.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`message_request_id`	Message	id
`channel`	Channel	text
`recipient`	Sent to	text
`status`	Status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Only sends addressed to a contact fire. Messages to a typed-in address with no contact do not.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · omnichannel

Message Failed

LIVE ENTRY

Fires when a message delivery fails

NODE TYPE	NODE KEY
Trigger	`trigger.messaging.message_failed`
MODULE	**OUTPUT PORTS**
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when an outbound message to a known contact fails or bounces.

When to use

- Fall back to another channel; flag the contact's address.

How it works

1. Fires from the delivery-status update when the attempt reaches **failed** or **bounced** (permanent send failures and provider-reported bounces alike). Runs on the contact; `{{trigger.error_code}}` / `{{trigger.error_message}}` carry the reason.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`message_request_id`	Message	id
`channel`	Channel	text
`recipient`	Sent to	text
`status`	Status (failed / bounced)	text
`error_code`	Error code	text
`error_message`	Error	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Only sends addressed to a contact fire.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · omnichannel

Unsubscription Submitted

LIVE ENTRY

Fires when a contact submits an unsubscribe request

NODE TYPE	NODE KEY
Trigger	`trigger.omnichannel.unsubscription_submitted`
MODULE	**OUTPUT PORTS**
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact submits an unsubscribe (or re-subscribe) request on an unsubscribe page.

When to use

- Tag the contact; notify the owner; confirm by a transactional email.

How it works

1. The submission event names the contact in its payload; the workflow runs on that contact. A submission for an address that matches no contact starts nothing.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only this kind `action_type`	Choice	No	—	Options: `global_unsubscribe` = Opted out of everything `category_unsubscribe` = Opted out of a category `channel_unsubscribe` = Opted out of a channel `resubscribe` = Opted back in

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only this kind** `action_type`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`action_type`	Kind (global / category / channel / resubscribe)	text
`recipient_identifier`	Address	text
`selected_categories`	Categories	list
`selected_channels`	Channels	list
`page_slug`	Unsubscribe page	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "action_type": "global_unsubscribe"
}
```

Best practices

- ✓ **Only this kind** narrows to global / category / channel opt-out or re-subscribe.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · omnichannel

Contact Globally Unsubscribed

LIVE ENTRY

Fires when a contact opts out of all communications

NODE TYPE	NODE KEY
Trigger	`trigger.omnichannel.contact_global_unsubscribed`
MODULE	**OUTPUT PORTS**
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a contact's global opt-out actually flips on (unsubscribe page or an admin override).

When to use

- Remove from every campaign; log for compliance.

How it works

1. Fires once, when the global preference changes from opted-in to opted-out. Repeating an opt-out that is already in place does not fire again.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`recipient_identifier`	Address	text
`source`	Where it came from	text
`action_type`	Kind	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · web_analytics

Web Goal Conversion

NOT WIRED YET ENTRY

Fires when a web analytics goal converts

NODE TYPE	NODE KEY
Trigger	`trigger.web_analytics.goal_conversion`

MODULE	OUTPUT PORTS
web_analytics (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a web analytics goal converts.

When to use

- Score the contact; alert sales on a pricing-page conversion.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Goal `goal_id`	Picker (web_goal)	No	—	Leave blank for any goal

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Goal** `goal_id`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

This trigger carries no extra payload beyond the record: use `{{record.<field>}}` (or add a condition node so `{{entity_fields.<column>}}` resolves).

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **web_analytics** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: In the palette but nothing dispatches this event yet — the workflow will not fire on its own.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "goal_id": "<Goal>"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Not wired: web analytics automations fire their own actions, not workflows. Check the dispatch badge.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · web_analytics

High Intent Visitor

NOT WIRED YET ENTRY

Fires when a high-intent visitor is detected

NODE TYPE	NODE KEY
Trigger	`trigger.web_analytics.high_intent_visitor`
MODULE	**OUTPUT PORTS**
web_analytics (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a visitor is detected as high intent.

When to use

- Create a task for sales; send a nudge.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

This trigger carries no extra payload beyond the record: use `{{record.<field>}}` (or add a condition node so `{{entity_fields.<column>}}` resolves).

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **web_analytics** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: In the palette but nothing dispatches this event yet — the workflow will not fire on its own.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Not wired: intent scoring runs only from a manual API call and writes no event. Check the dispatch badge.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · contacts

New Website / Chat Lead LIVE ENTRY

Fires when a chatbot visitor gives an email or phone that matches no contact, or the website tracking identifies a visitor

NODE TYPE	NODE KEY
Trigger	`trigger.leads.visitor_identified`

MODULE	OUTPUT PORTS
contacts (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Fires when a chatbot visitor gives an email or phone that matches no contact, or the website tracking identifies a visitor

When to use

- To start a workflow the moment this happens, rather than on a schedule or by hand.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
From `lead_source`	Choice	No	—	Options: `(blank)` = Chatbot or website `chat` = Chatbot only `web` = Website analytics only

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **From** `lead_source`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`lead_source`	From (chat / web)	text
`name`	Name	text
`email`	Email	text
`phone`	Phone	text
`session_id`	Chat session	id
`channel`	Chat channel	text
`user_id`	Website user id	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **contacts** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "lead_source": ""
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

—

TRIGGERS · link_management

Link Clicked

NOT WIRED YET ENTRY

Fires when a tracked link is clicked

NODE TYPE	NODE KEY
Trigger	`trigger.links.link_clicked`
MODULE	**OUTPUT PORTS**
link_management (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a tracked link is clicked.

When to use

- Score the contact; start a follow-up.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Link `link_id`	Picker (link)	No	—	Leave blank for any tracked link

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Link** `link_id`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

This trigger carries no extra payload beyond the record: use `{{record.<field>}}` (or add a condition node so `{{entity_fields.<column>}}` resolves).

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **link_management** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: In the palette but nothing dispatches this event yet — the workflow will not fire on its own.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "link_id": "<Link>"
}
```

Best practices

- ✓ **Link** narrows to one link; blank fires for any tracked link.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Not wired: clicks are recorded anonymously today (no contact on the click), so there is no contact for a workflow to run on. Needs per-contact link tracking first.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · custom_entities

Custom Record Created

LIVE ENTRY

Fires when a record of one of your own entities is created

NODE TYPE	NODE KEY
Trigger	`trigger.custom.record_created`
MODULE	OUTPUT PORTS
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a record of one of your own (custom) entities is created.

When to use

- Notify, assign, or create the next record in a chain.

How it works

1. The record service writes `customentity.record.created` for every custom entity; **Custom entity** picks which one starts this workflow.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	

Required fields

- **Custom entity** `entity_api_name`

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).

- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`entity_api_name`	Custom entity	text
`entity_label`	Entity label	text
`display_name`	Name	text
`record_number`	Record number	text
`status`	Status	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- **Custom entity** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · custom_entities

Custom Record Updated

LIVE ENTRY

Fires when a record of one of your own entities is changed

NODE TYPE	NODE KEY
Trigger	`trigger.custom.record_updated`

MODULE	OUTPUT PORTS
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a record of one of your own entities changes.

When to use

- Recalculate or re-route on a change.

How it works

1. The owning module records the event when it happens (most go through the outbox; a few are dispatched inline at the source).
2. The dispatcher finds every **published** workflow whose trigger matches the event type, then applies the trigger's own settings — status from / to, watched fields, record filters.
3. Guards: the same event never starts the same workflow twice; at most 5 runs per workflow per record per minute; automation chains stop at depth 5.
4. A run starts with `{{trigger.*}}` = the event payload and `{{record.*}}` = the record; execution continues down every wire out of the trigger.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	

Required fields

- **Custom entity** `entity_api_name`

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`entity_api_name`	Custom entity	text
`display_name`	Name	text
`version_no`	Version	number

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- **Custom entity** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>"
}
```

Best practices

- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · core

Manual Trigger

MANUAL ENTRY

Workflow is triggered manually by a user

NODE TYPE	NODE KEY
Trigger	`trigger.core.manual`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Lets a person start the workflow on demand — from the designer's Run now, a record page's Run automation, a list's bulk bar, or the API.

When to use

- A one-off or on-demand process: "send the renewal pack for this account", "re-score these 40 contacts".
- Anything you want a person to decide to run rather than an event.

How it works

1. **Runs on** says whether the person picks a record (of the workflow's record type) or the workflow just runs.
2. **Ask the person for** adds a small form; every answer is required and arrives as `{{trigger.<field name>}}`.
3. **Confirmation message** is shown before the run starts — use it for anything with side effects.
4. The run is queued and picked up by the worker (usually within a second); a bulk run queues one run per selected record.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Runs on `runs_on`	Choice	No	`record`	Options: `record` = A record the person picks (the workflow's record type) `nothing` = Nothing — it just runs
Ask the person for `inputs`	Question rows	No	—	Every answer is required. It arrives as {{trigger.<field name>}}
Confirmation message `confirm_message`	Text	No	—	e.g. `e.g. This emails the customer — continue?`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Runs on** `runs_on` — default `record`
- **Ask the person for** `inputs`
- **Confirmation message** `confirm_message`

Inputs

- The record the person picked (if Runs on = record) and their answers to the questions.

Outputs

The payload is whatever the sender or the person running it supplied — read keys as `{{trigger.<key>}}`; the designer learns the shape from the last test run.

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Running needs `workflows.run` **and** visibility of the chosen record. Designing needs the usual edit rights.

Dependencies

- Dispatch: Runs only when a person or the API starts it — no automatic firing.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

Manual runs get a random idempotency key — clicking Run twice starts two runs.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Ask for a discount before running

Downstream: `{{trigger.discount_pct}}`.

```
{
  "runs_on": "record",
  "inputs": [
    {
      "key": "discount_pct",
      "label": "Discount to offer (%)"
    }
  ],
  "confirm_message": "This emails the customer a revised quote — continue?"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "runs_on": "record",
  "inputs": [],
  "confirm_message": "e.g. This emails the customer — continue?"
}
```

Best practices

- ✓ Use the confirmation message for anything that sends or changes data.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · core

Scheduled

SCHEDULED ENTRY

Fires on a cron schedule

NODE TYPE	NODE KEY
Trigger	`trigger.core.scheduled`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow on a cron schedule in the company's (or a chosen) time zone — daily digests, weekly clean-ups, monthly reports.

When to use

- Recurring housekeeping: close stale deals, remind on overdue tasks, refresh a sheet.
- Anything that is "every Monday at 9" rather than "when X happens".

How it works

1. The scheduler evaluates every published scheduled workflow each minute and starts a run when the cron expression matches in the given time zone (blank = the company's default).
2. The run has **no record**; it starts with `{{trigger.scheduled_at}}` and `{{trigger.cron}}`. Use Find Records to pick the records to work on.
3. The idempotency key `sched:<workflow>:<minute>` means one run per scheduled minute even if the scheduler ticks twice.
4. **Only on working days** skips a matching minute that falls on a weekend or holiday of the workspace's default business calendar (checked on the schedule's local date). No calendar set up = every day counts.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
How often `cron`	Schedule (cron)	Yes	—	
Time zone `timezone`	Time zone	No	—	e.g. `— Company default time zone —`
Only on working days `working_days_only`	Yes / no	No	—	Skips weekends and holidays on the workspace business calendar. Ignored where no calendar is set up.

Required fields

- **How often** `cron`

Optional fields

- **Time zone** `timezone`
- **Only on working days** `working_days_only`

Inputs

- The clock. No record.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`scheduled_at`	When it ran (UTC)	text
`cron`	Schedule	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- Dispatch: Fires on a cron schedule set in the workflow, not on an event.

Validation rules

- **How often** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

One run per scheduled minute per workflow.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Weekdays at 09:00 IST

```
{
  "cron": "0 9 * * 1-5",
  "timezone": "Asia/Kolkata"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "cron": "0 9 * * 1-5",
  "timezone": "Asia/Kolkata",
  "working_days_only": false
}
```

Best practices

- ✓ Pair with Find Records in **loop** mode to act on each matching record.
- ✓ A schedule that fires while a previous run is still going starts another run — keep runs short or use Not again so soon.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- A run with no record: `{{record.*}}` is empty and record-targeted actions skip unless you point them at a found record.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · core

When a number crosses a line

SCHEDULED ENTRY

Fires when a count or total goes over (or under) a value — open tickets above 50, pipeline below target

NODE TYPE	NODE KEY
Trigger	`trigger.core.threshold`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Watches a count or total across many records and fires when it **crosses** a line — open tickets above 50, pipeline below target — rather than reacting to one record.

When to use

- Operational alerts on the shape of the whole book: queue depth, pipeline value, overdue count.
- Capacity warnings that should fire once, not on every check.

How it works

1. Every **Check every (minutes)** (default 15) the scheduler computes the operation (count / sum / avg / min / max of a field) over the records matching the filter.
2. It compares the number with **This number** using the chosen comparison.
3. It fires on the check where the number first crosses the line and stays quiet until it recovers — level-triggering would re-fire every check while it is high.
4. The run has no record; the number arrives as `{{trigger.value}}` with `threshold`, `comparison` and `checked_at`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Count `entity_type`	Choice	Yes	—	Options: `contact` = Contacts `account` = Accounts `opportunity` = Opportunities `campaign` = Campaigns `task` = Tasks `ticket` = Support tickets
Only those matching `filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
operation `operation`	Choice	No	`count`	Options: `count` = How many `sum` = Total of a field `avg` = Average of a field `min` = Lowest value `max` = Highest value
Field `field`	Record field	No	—	Shown when operation is sum / avg / min / max
Fires when it is `comparison`	Choice	No	`gt`	Options: `gt` = more than `gte` = at least `lt` = less than `lte` = at most
This number `value`	Number	Yes	—	
Check every (minutes) `check_every_minutes`	Number	No	`15`	

Required fields

- **Count** `entity_type`
- **This number** `value`

Optional fields

- **Only those matching** `filters`
- **Match** `logic` — default `and`
- **operation** `operation` — default `count`
- **Field** `field`
- **Fires when it is** `comparison` — default `gt`
- **Check every (minutes)** `check_every_minutes` — default `15`

Inputs

- The record type, filter, operation and field.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`value`	The number now	number
`threshold`	The line	number
`comparison`	Comparison	text
`checked_at`	Checked at	text

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.

VARIABLE	WHAT IT IS	AVAILABLE
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- Dispatch: Fires on a cron schedule set in the workflow, not on an event.

Validation rules

- **Count** must be filled in.
- **This number** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

Edge-triggered: one run per crossing.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

More than 50 open tickets

```
{
  "entity_type": "ticket",
  "filters": [
    {
      "field": "status",
      "operator": "in",
      "value": "new,open,pending"
    }
  ],
  "operation": "count",
  "comparison": "gt",
  "value": 50,
  "check_every_minutes": 15
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "contact",
  "filters": [],
  "logic": "and",
  "operation": "count",
  "field": "<Field>",
  "comparison": "gt",
  "value": 0,
  "check_every_minutes": 15
}
```

Best practices

- ✓ Use a Scheduled trigger instead if you want the number on every check regardless of crossing.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.
- ✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Checks at most as often as Check every — a spike that rises and falls between checks is not seen.
- No record on the run.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · core

Webhook Received

WEBHOOK ENTRY

Fires when an external webhook is received

NODE TYPE	NODE KEY
Trigger	`trigger.core.webhook`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when an external system posts JSON to this workflow's own webhook URL — payments, forms, other SaaS tools.

When to use

- Reacting to events from systems that can call a URL: invoice paid, form submitted, order shipped.

How it works

1. Publishing the workflow mints its webhook URL (shown on the trigger). The sender POSTs JSON to it.
2. **Only these events** matches the payload's `event` / `type` / `action` key against a comma list; blank accepts any.
3. **Only payloads matching** filters on the posted JSON with dotted paths (`data.amount`).
4. **Record id is in** names the path of a DigiCRM record id so the run has a record; blank tries `entity_id` , then `record_id` , then `id` .
5. The whole posted body is the payload: `{{trigger.<key>}}` with dotted paths. The designer learns the shape from the last test.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only these events `event_type`	Text	No	—	e.g. `invoice.paid, invoice.void` — blank for any
Only payloads matching `payload_filters`	Filter rows	No	—	Fields of the posted JSON — dotted paths work (data.amount)
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record id is in `record_id_field`	Text	No	—	e.g. `data.contact_id — blank tries id / record_id`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Only these events** `event_type`
- **Only payloads matching** `payload_filters`
- **Match** `logic` — default `and`
- **Record id is in** `record_id_field`

Inputs

- The posted JSON body.

Outputs

The payload is whatever the sender or the person running it supplied — read keys as `{{trigger.<key>}}`; the designer learns the shape from the last test run.

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- Dispatch: Fires when something posts to the workflow's own webhook URL.

Validation rules

- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Paid invoices for a known contact

```
{
  "event_type": "invoice.paid",
  "payload_filters": [
    {
      "field": "data.status",
      "operator": "eq",
      "value": "paid"
    }
  ],
  "record_id_field": "data.contact_id"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "event_type": "invoice.paid, invoice.void — blank for any",
  "payload_filters": [],
  "logic": "and",
  "record_id_field": "data.contact_id — blank tries id / record_id"
}
```

Best practices

- ✓ Send a test payload first so the { } Insert menu knows the fields.
- ✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- The dispatcher's per-record rate limit (5 runs per minute) applies when a record id resolves.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

Anyone who knows the URL can start the run — keep it secret, rotate it if it leaks, and validate what you use from the payload (filter on a shared secret field, never trust amounts blindly). The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

TRIGGERS · core

Record Created (Any) LIVE ENTRY

Fires when any entity record is created

NODE TYPE	NODE KEY
Trigger	`trigger.core.record_created`
MODULE	**OUTPUT PORTS**
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

A generic "created" trigger for any record type — pick the type in the step instead of choosing a per-module trigger.

When to use

- Templates that should work across record types; record types without a dedicated trigger.

How it works

1. Every "created" event the dispatcher knows (contact, company, deal, task, ticket, work item, custom records) is also offered to this trigger; **Record type** decides which one starts this workflow and **Only records matching** filters on its fields.
2. A workflow that carries both this trigger and the record type's own created trigger starts once per event, on the specific one.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Record type	Yes	—	
Only records matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

- **Record type** `entity_type`

Optional fields

- **Only records matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The payload is whatever the sender or the person running it supplied — read keys as `{{trigger.<key>}}`; the designer learns the shape from the last test run.

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- **Record type** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "<Record type>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Only record types with a mapped created event fire: campaigns, surveys and bookings have no generic created event today — use their own triggers.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · core

Record Updated (Any) LIVE ENTRY

Fires when any entity record is updated

NODE TYPE	NODE KEY
Trigger	`trigger.core.record_updated`
MODULE	**OUTPUT PORTS**
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

A generic "updated" trigger for any record type, with watched fields and filters.

When to use

- Templates that work across record types.

How it works

1. Every "updated" event the dispatcher knows (contact, company, deal, task, campaign, custom records) is also offered to this trigger; **Record type** picks the entity, **Only when these fields change** and **Only records matching** narrow it.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Record type	Yes	—	
Only when these fields change `watched_fields`	Field picker	No	—	
Only records matching `field_filters`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions

Required fields

- **Record type** `entity_type`

Optional fields

- **Only when these fields change** `watched_fields`
- **Only records matching** `field_filters`
- **Match** `logic` — default `and`

Inputs

- The event payload written by the module (its keys are listed under Outputs).
- The record the event is about, when there is one.

Outputs

The payload is whatever the sender or the person running it supplied — read keys as `{{trigger.<key>}}`; the designer learns the shape from the last test run.

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- **Record type** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "<Record type>",
  "watched_fields": "<Only when these fields change>",
  "field_filters": [],
  "logic": "and"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- Tickets, surveys, work items and bookings write no generic updated event — use their own status / stage triggers.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

TRIGGERS · custom_entities

Custom Record Deleted

LIVE ENTRY

Fires when a record of one of your own entities is deleted

NODE TYPE	NODE KEY
Trigger	`trigger.custom.record_deleted`
MODULE	**OUTPUT PORTS**
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Starts the workflow when a record of one of your own entities is deleted.

When to use

- Audit notification; clean up linked records.

How it works

1. The record service writes `customentity.record.deleted` keyed to the entity (`custom:<api name>`); **Custom entity** picks which one starts this workflow. Payload: `entity_api_name` , `entity_def_id` , `display_name` .

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	

Required fields

- **Custom entity** `entity_api_name`

Optional fields

None.

Inputs

- The event payload written by the module (its keys are listed under Outputs).

- The record the event is about, when there is one.

Outputs

The event payload — read each key as `{{trigger.<key>}}`. The triggering record itself is `{{record.<field>}}`.

KEY	MEANING	TYPE
`entity_api_name`	Custom entity	text
`display_name`	Name	text
`entity_def_id`	Entity definition	id

Execution behaviour

A trigger does no work of its own: it is the entry point that decides whether an event starts this workflow and hands the payload to the first step. A workflow can have several triggers and starts on any of them — one run per event.

Success path

Every wire out of the trigger is followed.

Failure path

A trigger cannot fail. If its filters do not match, no run is created at all (nothing is logged). If dispatching the event throws, that event is skipped and not retried.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing needs `workflows.read`; the workflow must be **published** to fire. The run acts as the workflow's Run as user.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- Dispatch: Fires automatically — dispatched by the outbox poller when the event happens.

Validation rules

- **Custom entity** must be filled in.
- Must be the first step: nothing can connect into a trigger.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- None of its own. A misconfigured filter simply never matches — use **Validate** and a Test run to check the filters.

Retry behaviour

Not applicable — an event either starts a run or it does not. The dispatcher does not retry an event it failed to hand over.

Timeout

Not applicable to the trigger itself; the run it starts has the workflow's Time limit.

Idempotency

The same event starts the same workflow at most once (idempotency key `wfevt:<event id>:<workflow id>`). Two *different* events for the same record do start two runs, up to 5 per minute.

Transaction / rollback

Not applicable — the trigger writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>"
}
```

Best practices

✓ Narrow with the trigger's own filters rather than a condition step: a filtered-out event creates no run at all, a condition still burns a run.

✓ Need a column that is not in the payload? Use `{{record.<field>}}` (loaded automatically when a condition step is on the canvas or a setting mentions `record.`).

Limitations

- The record is gone when the run starts — only the payload can be read; no filters are offered for that reason.

Audit / logging

The run record stores which trigger fired and the full trigger payload. The trigger step itself appears first in the step log.

Security considerations

The payload is whatever the module wrote — treat webhook and manual-input payloads as untrusted text when you put them in messages.

Related nodes

CONDITIONS · core

Field Condition

Branch based on field value comparison

NODE TYPE	NODE KEY
Condition	`condition.core.field_check`

MODULE	OUTPUT PORTS
core (always available)	true, false

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Branches the run on one comparison — a field against a value — the simplest and most common decision step.

When to use

- "Is the deal over 50 000?" "Is the ticket priority urgent?" "Is the owner empty?"

How it works

1. Reads **Field** from the record snapshot, then the trigger payload, then variables (a later source wins when the name appears in several). Dotted paths work.
2. Applies **Is** (the operator) against **Value**, after resolving placeholders in the value.
3. Numbers are compared as numbers ("50" equals 50); `contains`, `starts_with`, `ends_with` ignore case; `in` / `not_in` accept a list or a comma-separated text; `regex` is a full regular expression.
4. Takes the `true` or `false` port.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Field `field`	Record field	Yes	—	
Is `operator`	Choice	Yes	`eq`	Options: `eq` `neq` `gt` `gte` `lt` `lte` `contains` `not_contains` `starts_with` `ends_with` `is_null` `is_not_null` `in` `not_in` `regex`
Value `value`	Value or variable	No	—	

Required fields

- **Field** `field`

- **Is** `operator`

Optional fields

- **Value** `value`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Evaluates its rows against the record snapshot, the trigger payload and the variables (later ones win when a name appears in more than one) and picks a port. Loading the record for the comparison is what makes `{{record.*}}` available to every later step.

Success path

The run follows the port that matched. A `true` result with no true wire follows only wires that carry no port (a legacy node); on a node wired by port the branch ends. A `false` result with no false wire ends the branch quietly.

Failure path

A comparison never fails the step. Ordered operators (gt / gte / lt / lte) compare numbers as numbers; when either side is not a number both are compared as text (so ISO dates order correctly, but "abc" > "5" is true). An empty value, an unknown operator or a bad regex evaluates **false**. Only formula conditions fail on an error.

Branches

PORT	TAKEN WHEN
`true`	Every row matched (or any row, with Match = ANY).
`false`	The rows did not match, or the comparison could not be made.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Field** must be filled in.
- **Is** must be filled in.
- Field and operator are required; a node with no field configured always evaluates **true**.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- Nothing fails. An empty value or a bad regex evaluates **false**; a non-numeric side of gt / gte / lt / lte is compared as text, not rejected.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only — safe to repeat.

Transaction / rollback

Writes nothing.

Examples

Big deals

```
{
  "field": "amount",
  "operator": "gte",
  "value": 50000
}
```

Ticket from a VIP tag

```
{
  "field": "tags",
  "operator": "contains",
  "value": "vip"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field": "<Field>",
  "operator": "eq",
  "value": "<Value>"
}
```

Best practices

- ✓ For more than one row use Multi-Condition; for sums, date maths or linked records use Formula condition.
- ✓ Wire both ports. A true result with only a false wire ends the branch — nothing runs.
- ✓ Numbers compare as numbers ("50" equals 50, "50" is less than "100"); when a side is not numeric the comparison is text order.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Read-only. The record is loaded without a visibility check because it is the triggering record.

Related nodes

CONDITIONS · core

Multi-Condition

Evaluate multiple conditions with AND/OR logic

NODE TYPE	NODE KEY
Condition	`condition.core.multi_condition`

MODULE	OUTPUT PORTS
core (always available)	true, false

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Branches on several rows combined with ALL (and) or ANY (or).

When to use

- "Urgent **and** unassigned", "from the web **or** from a partner".

How it works

1. Each row is evaluated as in Field Condition; **Match** decides whether all or any must hold; an empty list is **true**.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions
conditions `conditions`	Filter rows	No	—	

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Match** `logic` — default `and`
- **conditions** `conditions`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Evaluates its rows against the record snapshot, the trigger payload and the variables (later ones win when a name appears in more than one) and picks a port. Loading the record for the comparison is what makes `{{record.*}}` available to every later step.

Success path

The run follows the port that matched. A `true` result with no true wire follows only wires that carry no port (a legacy node); on a node wired by port the branch ends. A `false` result with no false wire ends the branch quietly.

Failure path

A comparison never fails the step. Ordered operators (gt / gte / lt / lte) compare numbers as numbers; when either side is not a number both are compared as text (so ISO dates order correctly, but "abc" > "5" is true). An empty value, an unknown operator or a bad regex evaluates **false**. Only formula conditions fail on an error.

Branches

PORT	TAKEN WHEN
`true`	Every row matched (or any row, with Match = ANY).
`false`	The rows did not match, or the comparison could not be made.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only — safe to repeat.

Transaction / rollback

Writes nothing.

Examples

Urgent and unassigned

```
{
  "logic": "and",
  "conditions": [
    {
      "field": "priority",
      "operator": "eq",
      "value": "urgent"
    },
    {
      "field": "assigned_to",
      "operator": "is_null"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "logic": "and",
  "conditions": []
}
```

Best practices

✓ Wire both ports. A true result with only a false wire ends the branch — nothing runs.

✓ Numbers compare as numbers ("50" equals 50, "50" is less than "100"); when a side is not numeric the comparison is text order.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Read-only. The record is loaded without a visibility check because it is the triggering record.

Related nodes

CONDITIONS · core

Switch / Router

Send the run down a different path per value — one path per branch, plus 'Other' for anything that matches none of them

NODE TYPE	NODE KEY
Condition	`condition.core.switch`

MODULE	OUTPUT PORTS
core (always available)	branch_1, branch_2, branch_3, default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sends the run down one of several paths depending on a value — one branch per case plus **Other**.

When to use

- Routing by category, priority, region, plan tier: three or more outcomes from one field.

How it works

1. **Look at** names the field; each **Branch** row has its own comparison (it may compare a different field) and gets its own port on the canvas.
2. Branches are checked in order; the first match wins and its port is taken.
3. No match → `default` (Other). A matched branch whose port is not wired also goes to `default`, and the output notes `unwired`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Look at `field`	Record field	Yes	—	A branch can compare a different field instead
Branches `branches`	Branch rows	No	—	Each branch gets its own path on the canvas. Wire the ones you use — an unwired branch takes 'Other'.

Required fields

- **Look at** `field`

Optional fields

- **Branches** `branches`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Evaluates its rows against the record snapshot, the trigger payload and the variables (later ones win when a name appears in more than one) and picks a port. Loading the record for the comparison is what makes `{{record.*}}` available to every later step.

Success path

The run follows the port that matched. A `true` result with no true wire follows only wires that carry no port (a legacy node); on a node wired by port the branch ends. A `false` result with no false wire ends the branch quietly.

Failure path

A comparison never fails the step. Ordered operators (gt / gte / lt / lte) compare numbers as numbers; when either side is not a number both are compared as text (so ISO dates order correctly, but "abc" > "5" is true). An empty value, an unknown operator or a bad regex evaluates **false**. Only formula conditions fail on an error.

Branches

PORT	TAKEN WHEN
`branch_1`	Branch 1 in the Branches setting matched.
`branch_2`	Branch 2 in the Branches setting matched.
`branch_3`	Branch 3 in the Branches setting matched.
`default`	No branch matched, or the matching branch has nothing wired.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Look at** must be filled in.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only — safe to repeat.

Transaction / rollback

Writes nothing.

Examples

Route tickets by category

```
{
  "field": "category",
  "branches": [
    {
      "label": "Billing",
      "operator": "eq",
      "value": "billing"
    },
    {
      "label": "Technical",
      "operator": "eq",
      "value": "technical"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "field": "<Look at>",
  "branches": []
}
```

Best practices

- ✓ Always wire Other — it is where anything unexpected goes.
- ✓ Wire both ports. A true result with only a false wire ends the branch — nothing runs.
- ✓ Numbers compare as numbers ("50" equals 50, "50" is less than "100"); when a side is not numeric the comparison is text order.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Read-only. The record is loaded without a visibility check because it is the triggering record.

Related nodes

CONDITIONS · core

Formula condition

Continue down Yes when a formula is true — compare two fields, do sums or date maths, reach linked records

NODE TYPE	NODE KEY
Condition	`condition.core.formula`

MODULE	OUTPUT PORTS
core (always available)	true, false

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Branches on a yes/no formula from the calculated-fields engine — sums, date maths, linked-record fields (`{account_id.industry}`), AND / OR / IF.

When to use

- Decisions a plain comparison cannot express: "amount over 100 000 and the account is in banking", "close date within 7 working days".

How it works

1. The formula is evaluated against the record, `{$trigger.x}` and `{$var.x}` values by the formulas engine.
2. Exactly **true** takes the Yes (`true`) port; blank or anything else takes No (`false`).
3. The formula must be a yes/no formula at design time: one that returns a number or text is refused with "#FORMULA!: This step needs a yes/no answer, but the formula returns <type>…" and fails the step (On error applies).
4. A formula **error** (an `#ERR` value) fails the step with the message — it takes On error, never No.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Continue down Yes when `formula`	Formula	Yes	—	A yes/no formula, e.g. AND({amount} > 100000, {account_id.industry} = "Banking"). A blank answer counts as No; an error takes On error.

Required fields

- **Continue down Yes when** `formula`

Optional fields

None.

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Evaluates its rows against the record snapshot, the trigger payload and the variables (later ones win when a name appears in more than one) and picks a port. Loading the record for the comparison is what makes `{{record.*}}` available to every later step.

Success path

The run follows the port that matched. A `true` result with no true wire follows only wires that carry no port (a legacy node); on a node wired by port the branch ends. A `false` result with no false wire ends the branch quietly.

Failure path

A formula error fails the step (message "<#ERR>: ..."). If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`true`	Every row matched (or any row, with Match = ANY).
`false`	The rows did not match, or the comparison could not be made.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's	Always.

VARIABLE	WHAT IT IS	AVAILABLE
	Outputs).	
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Continue down Yes when** must be filled in.
- A formula is required ("The step has no formula").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only — safe to repeat.

Transaction / rollback

Writes nothing.

Examples

Big banking deals

```
{
  "formula": "AND({amount} > 100000, {account_id.industry} = \"Banking\")"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "formula": "<Continue down Yes when>"
}
```

Best practices

- ✓ Wrap risky references in IFERROR() if you would rather get No than a failed step.
- ✓ Wire both ports. A true result with only a false wire ends the branch — nothing runs.
- ✓ Numbers compare as numbers ("50" equals 50, "50" is less than "100"); when a side is not numeric the comparison is text order.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Read-only. The record is loaded without a visibility check because it is the triggering record.

Related nodes

CONDITIONS · core

Switch on a formula

Work out a value and take the path named after it — one path per value, 'Other' for anything else

NODE TYPE	NODE KEY
Condition	`condition.core.formula_switch`
MODULE	**OUTPUT PORTS**
core (always available)	<one per choice>, default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Works out a value with a formula and takes the path named after it — a switch driven by IFS() or any text formula.

When to use

- Sizing deals into Large / Mid / Small, bucketing dates, mapping codes to teams.

How it works

1. Evaluates the formula; the answer's display text is slugified and matched against the slugified **Paths** — that path's port is taken.
2. No match or blank → `default` (Other). Matched but unwired → `default` with `unwired` noted.
3. An `#ERR` value fails the step.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Work out `formula`	Formula	Yes	—	e.g. IFS({amount} >= 1000000, "Large", {amount} >= 100000, "Mid", TRUE, "Small")
Paths `choices`	List of names	Yes	—	One answer per line — each gets its own path. Anything else takes Other.

Required fields

- **Work out** `formula`
- **Paths** `choices`

Optional fields

None.

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Evaluates its rows against the record snapshot, the trigger payload and the variables (later ones win when a name appears in more than one) and picks a port. Loading the record for the comparison is what makes `{{record.*}}` available to every later step.

Success path

The run follows the port that matched. A `true` result with no true wire follows only wires that carry no port (a legacy node); on a node wired by port the branch ends. A `false` result with no false wire ends the branch quietly.

Failure path

A comparison never fails the step. Ordered operators (gt / gte / lt / lte) compare numbers as numbers; when either side is not a number both are compared as text (so ISO dates order correctly, but "abc" > "5" is true). An empty value, an unknown operator or a bad regex evaluates **false**. Only formula conditions fail on an error.

Branches

PORT	TAKEN WHEN
`<one per choice>`	One port per choice the author typed; the run takes the one that was picked.
`default`	The normal continuation after the step succeeds.
`true`	Every row matched (or any row, with Match = ANY).
`false`	The rows did not match, or the comparison could not be made.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Work out** must be filled in.
- **Paths** must be filled in.
- Formula and at least one path are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.

- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only — safe to repeat.

Transaction / rollback

Writes nothing.

Examples

Deal size

```
{
  "formula": "IFS({amount} >= 1000000, \"Large\", {amount} >= 100000, \"Mid\", TRUE, \"Small\")",
  "choices": [
    "Large",
    "Mid",
    "Small"
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "formula": "<Work out>",
  "choices": []
}
```

Best practices

✓ Wire both ports. A true result with only a false wire ends the branch — nothing runs.

✓ Numbers compare as numbers ("50" equals 50, "50" is less than "100"); when a side is not numeric the comparison is text order.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Read-only. The record is loaded without a visibility check because it is the triggering record.

Related nodes

ACTIONS · core

Update Field

Update a field on a CRM record

NODE TYPE	NODE KEY
Action	`action.crm.update_field`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sets one field on a record — the triggering record, a record found earlier, or every record matching a filter.

When to use

- Stamp a date, set a stage, clear a flag, copy a value from the trigger into a field.

How it works

1. **Which record(s)** picks the target: the record that started the run (default), a record an earlier step found (`target_record_id`, usually `{{variables.found.id}}`), or every record matching a filter (up to **Change at most**, hard cap 500).
2. A custom-entity record goes through the record service (its rules apply). Any other record has the column set **directly**, after checks: the field must exist on that type and must not be protected (id, tenant / audit columns, owner, email / phone / mobile, password, privacy fields, version). Contact "source" fields follow the contact form's rules.
3. If the value is already set the step reports `noop`; otherwise it writes an audit row `field_updated` with old and new values.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Which record(s) `target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `found_record` = A record found earlier `matching` = Every record that matches a filter
Record type `target_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `campaign` `task` `ticket`

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Shown when target is found_record / matching
Record id `target_record_id`	Text	No	—	Shown when target is found_record
Only records where `target_filters`	Filter rows	No	—	Shown when target is matching
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions Shown when target is matching
Change at most `max_records`	Number	No	`50`	A guard, not a filter — the run says how many matched Shown when target is matching
field `field`	Record field	Yes	—	
value `value`	Value or variable	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **field** `field`

Optional fields

- **Which record(s)** `target` — default `trigger_record`
- **Record type** `target_entity_type`
- **Record id** `target_record_id`
- **Only records where** `target_filters`
- **Match** `logic` — default `and`

- **Change at most** `max_records` — default `50`
- **value** `value`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The target record(s) and the resolved **Value** (text, number or a placeholder).

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`field`	Field changed	text
`noop`	Already that value	boolean

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **field** must be filled in.
- Field is required.
- The field must be a real column of the record type and not a protected one.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no field configured" / "'x' is not a column on <type>" / "field 'x' is protected" → skipped.

- "no record selected — set the record id (e.g. {{variables.found.id}})" / "'x' is not a record id" — the found-record target is blank or malformed → skipped.
- "this workflow's run-as user can't edit that record" — a found record outside the Run as user's visibility → skipped.
- "entity not found" → skipped.
- A database error (wrong type for the column, a constraint) → failed with the first 200 characters of the message.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent — running twice leaves the same value; the second run reports `noop`.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Stamp the first-response time on a ticket

```
{
  "target": "trigger_record",
  "field": "first_response_at",
  "value": "{{env.now}}"
}
```

Move every stale deal to Nurture

```
{
  "target": "matching",
  "target_entity_type": "opportunity",
  "target_filters": [
    {
```

```
      "field": "stage",
      "operator": "eq",
      "value": "proposal"
    },
    {
      "field": "updated_at",
      "operator": "lt",
      "value": "{{variables.cutoff}}"
    }
  ],
  "max_records": 100,
  "field": "stage",
  "value": "nurture"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "target": "trigger_record",
  "target_entity_type": "contact",
  "target_record_id": "<Record id>",
  "target_filters": [],
  "logic": "and",
  "max_records": 50,
  "field": "<field>",
  "value": "<value>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Owner, email / phone / mobile and the identity, audit, privacy, password and version columns cannot be set here — use Assign Owner for the owner. `status` can be set here, but prefer Change Status or the module's own step so its side effects run.
- Writes the column directly, bypassing the module's own validation (except custom entities and contact source fields).
- Matching mode also reports `matched`, `changed`, `record_ids` and `capped_at` in the step output.
- Matching mode changes at most **Change at most** records (default 50, never more than 500) and stops at the first record that fails.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current

step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Because it writes the column directly, the field allow-list is the only guard — it blocks credentials and contact addresses. The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Create Record

Create a new CRM record (contact, account, opportunity, etc.)

NODE TYPE	NODE KEY
Action	`action.crm.create_record`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Creates a new contact, company, lead or opportunity with the field values you map.

When to use

- Create the opportunity when a lead qualifies; create a contact from a webhook payload; create the company for a new contact.

How it works

1. **Record type**: contact, account, lead (a contact with lifecycle stage "lead") or opportunity.
2. **Fields to set** rows map a value (a placeholder or literal) to a target field.
3. The module's required fields are checked first; then the record is created through the module's own service, so its defaults, validation, audit trail and events (e.g. Contact Created) all apply.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Record type	Yes	—	
Fields to set `field_mappings`	Field → value rows	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Record type** `entity_type`

Optional fields

- **Fields to set** `field_mappings`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The mapped values, resolved from the run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`record_id`	Record id	id
`entity_type`	Record type	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Record type** must be filled in.
- Record type is required.
- Every field the module requires must be mapped: "Can't create a <type>: no value for a, b. Map these fields on the step."
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "unsupported entity_type 'x'" → skipped.
- A missing required field, a schema error or a service error → failed with the message.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — every run creates another record. Use Find or create when the record may already exist.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Opportunity from a qualified lead

Later: `{{variables.new_deal.id}}`.

```
{
  "entity_type": "opportunity",
  "field_mappings": [
    {
      "key": "{{record.first_name}} {{record.last_name}} — new deal",
      "value": "name"
    },
    {
      "key": "{{record.account_id}}",
      "value": "account_id"
    },
    {
      "key": 25000,
      "value": "amount"
    }
  ],
  "output_variable": "new_deal"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "<Record type>",
  "field_mappings": [],
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Creating a record can fire another workflow (chains stop at depth 5) — check you are not looping on yourself.

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Only contact, account, lead and opportunity; other types have their own create steps (task, ticket, work item, booking, link, custom record).

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current

step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Assign Owner

Hand the record to someone — its owner, a ticket's assignee, a task's assignee, or a role on a deal

NODE TYPE	NODE KEY
Action	`action.crm.assign_owner`
MODULE	**OUTPUT PORTS**
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Hands a record to someone: its owner, a ticket's or task's assignee, an extra assignee, or a role on a deal team (with share and commission).

When to use

- Round-robin new leads or tickets across a team.
- Give a deal to the account's owner; add a sales engineer to the deal team.

How it works

1. **Assign as** decides what "assign" means for this record type; the designer only offers what fits, and the engine skips (with a reason) anything that does not apply.
2. **Give it to**: a person, a team (its manager, else the first active member), or a team taking turns (**round robin** — an atomic per-step counter, so parallel runs never pick the same person twice; a Test run peeks without advancing it).
3. The change goes through the record's own module service inside a savepoint, so the module's rules, audit rows and events (e.g. Opportunity Assigned) apply. A task also gets the new owner as primary assignee.
4. A found record ("A record found earlier") is written only when the Run as user may edit it — the same gate as Update Field.
5. Same owner already → `noop`.
6. **Share with** (contact / company): keeps the owner and gives a person, or everyone in a team, **View** or **View and edit** access — the same share the record's Sharing tab lists and can revoke. Optionally for N days. It only matters while the record is private; on a public one the step still writes it and says so in `note`. Manage (re-share, reassign, delete) is never granted by a workflow.
7. **Remove a share** (contact / company): takes back the explicit shares of a person, a whole team or related people — shares made by a sharing rule stay (change the rule). **Take someone off the deal team** (deal): marks their team row removed; the deal owner cannot be removed.
8. **More ways to pick**: *a team, whoever has the least open work* (open tickets and deals use their module's own count; tasks, campaigns and work items skip closed ones; contacts and companies count everything owned) and *the territory owner* (the company's territory — a contact's and a deal's come from their company, as territory realignment does). When sharing: *people related to this record* — the owner of a contact's company, or everyone on its open deals (owners and active deal-team members).

9. **Include the company's contacts** (share / remove a share on a company): its contacts get or lose the same share, up to 500.
10. **Tell them**: an in-app notification (with an Open link) to whoever got the record, a place on it or access to it — for a team share, every member. Not sent for "every record that matches", nor to the person the workflow runs as.
11. **Fine-tune what they can do** (deal team): per-switch Allow / Don't allow on top of the level (see the deal, edit, revenue, tasks, commissions, stakeholders, quotes); untouched switches follow the level.
12. **Keep the previous owner's access** (replace mode on a contact / company): the old owner gets a View or Edit share, so a private record does not disappear from their lists.
13. **Deal team → What they can do**: View (read only), Edit (fields, revenue, tasks, stakeholders) or Full (everything incl. stage changes, quotes, commissions) — the same levels as the deal's Team tab. Contacts and companies on a deal are outside people: they get no access to the CRM.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Assign as `assign_as`	Choice	No	`owner`	Deal roles carry a share and a commission Options: `owner` = The owner (a ticket's or task's assignee) `share` = Share it (keeps the current owner) `unshare` = Remove a share `extra_assignee` = An extra assignee (a ticket or task) `deal_team` = Someone on the deal team (sales rep, co-owner…) `remove_from_deal` = Take someone off the deal team `deal_contact` = A contact on the deal `deal_company` = A company on the deal
Which record(s) `target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `found_record` = A record found earlier `matching` = Every record that matches a filter Shown when assign_as is (blank) / owner / extra_assignee
Record type `target_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `campaign` `task` `ticket`

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Shown when target is found_record / matching
Record id `target_record_id`	Value or variable	No	—	Shown when target is found_record e.g. `{{variables.found.id}}`
Only records where `target_filters`	Filter rows	No	—	Shown when target is matching
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions Shown when target is matching
Change at most `max_records`	Number	No	`50`	A guard, not a filter — the run says how many matched Shown when target is matching
Give it to `assignment_type`	Choice	No	—	Options: `user` = A person `team` = A team (its manager) `round_robin` = A team, taking turns `least_loaded` = A team, whoever has the least open work `territory` = The territory owner `whole_team` = Everyone in a team `related` = People related to this record Shown when assign_as is (blank) / owner / extra_assignee / deal_team / share / unshare / remove_from_deal
Related how `related_to`	Choice	No	`deal_teams`	Options: `account_owner` = The owner of its company `deal_teams` = Everyone on its open deals (owners and deal teams) Shown when assignment_type is related

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
User `user_id`	User picker	No	—	Shown when assign_as is (blank) / owner / extra_assignee / deal_team / share / unshare / remove_from_deal and assignment_type is (blank) / user
Team `team_id`	Picker (team)	No	—	Shown when assign_as is (blank) / owner / extra_assignee / deal_team / share / unshare / remove_from_deal and assignment_type is team / round_robin / least_loaded / whole_team
They can `access_level`	Choice	No	`read`	Only takes effect while the record is private Options: `read` = View it (read only) `edit` = View and edit it (not reassign, share or delete) Shown when assign_as is share
For how many days `share_days`	Number	No	—	Leave blank to share until someone removes it Shown when assign_as is share
Include the company's contacts `include_contacts`	Yes / no	No	—	A company only: its contacts get (or lose) the same share, up to 500 Shown when assign_as is share / unshare
Keep the previous owner's access `keep_previous_owner`	Yes / no	No	—	On a private record the old owner would lose it Shown when assign_as is (blank) / owner
Previous owner can `previous_owner_access`	Choice	No	`read`	Options: `read` = View it (read only) `edit` = View and edit it Shown when keep_previous_owner is true

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Tell them `notify`	Yes / no	No	—	An in-app notification to whoever gets the record, a place on it, or access Shown when assign_as is (blank) / owner / extra_assignee / deal_team / share
Message (optional) `notify_message`	Multi-line text	No	—	Shown when notify is true
Contact `contact_id`	Record picker (contact)	Yes	—	Shown when assign_as is deal_contact
Company `account_id`	Record picker (account)	Yes	—	Shown when assign_as is deal_company
Role on the deal `team_role`	Choice	No	`sales_rep`	Options: `sales_rep` = Sales rep `co_owner` = Co-owner `sales_engineer` = Sales engineer `account_manager` = Account manager `solution_architect` = Solution architect `bdr` = Business development `overlay` = Overlay specialist `support` = Support Shown when assign_as is deal_team
What they can do on the deal `permission_level`	Choice	No	`edit`	Fine-tune any of these below, or later on the deal's Team tab Options: `view` = View — read only; no revenue, tasks, stakeholders, quotes or commissions `edit` = Edit — edit fields, revenue, tasks, stakeholders; no stage changes, quotes or commissions `full` = Full — everything, including stage changes, quotes and commissions Shown when assign_as is deal_team
Fine-tune what they can do `customise_permissions`	Yes / no	No	—	Shown when assign_as is deal_team

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
See the deal `perm_read`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true
Edit the deal `perm_edit`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true
Revenue and projections `perm_revenue`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true
Tasks `perm_tasks`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true
Commissions `perm_commissions`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true
Stakeholders `perm_stakeholders`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Quotes `perm_quotes`	Choice	No	—	Options: `(blank)` = As the level says `allow` = Allow `deny` = Don't allow Shown when customise_permissions is true
Role on the deal `contact_role`	Choice	No	—	Options: `decision_maker` `influencer` `champion` `technical_evaluator` `end_user` `sponsor` `gatekeeper` Shown when assign_as is deal_contact
Role on the deal `company_role`	Choice	No	—	Options: `buyer` `partner` `reseller` `vendor` `consultant` `implementer` Shown when assign_as is deal_company
Share of the deal (%) `share_pct`	Number	No	—	Their contribution (a person) or influence (a contact / company) Shown when assign_as is deal_team / deal_contact / deal_company
Commission `commission_type`	Choice	No	—	Options: `(blank)` = None `percentage` = A percentage of the deal `fixed` = A fixed amount Shown when assign_as is deal_team / deal_contact / deal_company
Commission amount `commission_value`	Number	No	—	The percentage, or the amount in the deal's currency Shown when commission_type is percentage / fixed
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Contact** `contact_id` — only when assign_as is deal_contact
- **Company** `account_id` — only when assign_as is deal_company

Optional fields

- **Assign as** `assign_as` — default `owner`
- **Which record(s)** `target` — default `trigger_record`
- **Record type** `target_entity_type`
- **Record id** `target_record_id`
- **Only records where** `target_filters`
- **Match** `logic` — default `and`
- **Change at most** `max_records` — default `50`
- **Give it to** `assignment_type`
- **Related how** `related_to` — default `deal_teams`
- **User** `user_id`
- **Team** `team_id`
- **They can** `access_level` — default `read`
- **For how many days** `share_days`
- **Include the company's contacts** `include_contacts`
- **Keep the previous owner's access** `keep_previous_owner`
- **Previous owner can** `previous_owner_access` — default `read`
- **Tell them** `notify`
- **Message (optional)** `notify_message`
- **Role on the deal** `team_role` — default `sales_rep`
- **What they can do on the deal** `permission_level` — default `edit`

- **Fine-tune what they can do** `customise_permissions`
- **See the deal** `perm_read`
- **Edit the deal** `perm_edit`
- **Revenue and projections** `perm_revenue`
- **Tasks** `perm_tasks`
- **Commissions** `perm_commissions`
- **Stakeholders** `perm_stakeholders`
- **Quotes** `perm_quotes`
- **Role on the deal** `contact_role`
- **Role on the deal** `company_role`
- **Share of the deal (%)** `share_pct`
- **Commission** `commission_type`
- **Commission amount** `commission_value`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`owner_id`	New owner (user id)	id
`previous_owner_id`	Previous owner (user id)	id
`shared_with`	Shared with (user or team id)	id
`removed`	Shares removed	number
`notified`	People notified (user ids)	list
`access_level`	Access given	text
`noop`	Already theirs (nothing changed)	boolean

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.

VARIABLE	WHAT IT IS	AVAILABLE
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Contact** must be filled in when assign_as is deal_contact.
- **Company** must be filled in when assign_as is deal_company.
- Contact / Company required for the deal-contact / deal-company modes.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no user_id configured" / "user not in tenant" (also inactive users) / "no team selected, or the team has no active members" → skipped.
- "a <type> has no owner this step can set" / "a <type> has one owner — an extra assignee only applies to a ticket or a task" / deal-role modes on a non-deal → skipped.
- "a booking's host is moved from the calendar, which checks the new host is free" → skipped.
- "<type> not found", or the module's own refusal message → skipped.
- "a workflow can share with read or edit access" / "sharing applies to a contact or an account" / "team not in tenant" → skipped.
- "nobody is related that way" / "this contact has no company, so no territory" / "the company has no territory, or its territory has no active owner" / "Cannot remove the deal owner" → skipped.
- "no record selected — set the record id (e.g. {{variables.found.id}})" / "'x' is not a record id" — the found-record target is blank or malformed → skipped.
- "this workflow's run-as user can't edit that record" — a found record outside the Run as user's visibility → skipped.
- "entity not found" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be

retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Owner modes converge (second run `noop`). A share never lowers access someone already has (a Read share leaves an Edit share as Edit). Round robin advances on every run. Deal-team / contact / company modes add a participant — repeat runs may add duplicates depending on the deal service's rules.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Round-robin tickets to Support L1

```
{
  "assign_as": "owner",
  "assignment_type": "round_robin",
  "team_id": "<team id>"
}
```

Share a private contact with the account manager (edit, 30 days)

```
{
  "assign_as": "share",
  "assignment_type": "user",
  "user_id": "<user id>",
  "access_level": "edit",
  "share_days": 30
}
```

Deal lost → take the overlay rep off

```
{
  "assign_as": "remove_from_deal",
  "assignment_type": "user",
  "user_id": "<user id>"
}
```

Share a company and its contacts with everyone on its open deals

```
{
  "assign_as": "share",
  "assignment_type": "related",
  "related_to": "deal_teams",
  "include_contacts": true,
  "notify": true
}
```

Add a sales engineer with 10 % share

```
{
  "assign_as": "deal_team",
  "assignment_type": "user",
  "user_id": "<user id>",
  "team_role": "sales_engineer",
  "share_pct": 10
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "assign_as": "owner",
  "target": "trigger_record",
  "target_entity_type": "contact",
  "target_record_id": "{{variables.found.id}}",
  "target_filters": [],
  "logic": "and",
  "max_records": 50,
  "assignment_type": "user",
  "related_to": "deal_teams",
  "user_id": "<user id>",
  "team_id": "<Team>",
  "access_level": "read",
  "share_days": 0,
  "include_contacts": false,
  "keep_previous_owner": false,
  "previous_owner_access": "read",
  "notify": false,
  "notify_message": "<Message (optional)>",
  "contact_id": "<contact id>",
  "account_id": "<account id>",
  "team_role": "sales_rep",
  "permission_level": "edit",
  "customise_permissions": false,
  "perm_read": "",
  "perm_edit": "",
  "perm_revenue": "",
  "perm_tasks": "",
  "perm_commissions": "",
  "perm_stakeholders": "",
  "perm_quotes": "",
  "contact_role": "decision_maker",
  "company_role": "buyer",
  "share_pct": 0,
  "commission_type": "",
  "commission_value": 0,
  "output_variable": "new_contact",
```

```
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- A found record is only written when the Run as user may edit it (the same gate as Update Field).
- A booking's host cannot be changed here.
- Matching mode also reports `matched`, `changed`, `record_ids` and `capped_at` in the step output.
- Matching mode changes at most **Change at most** records (default 50, never more than 500) and stops at the first record that fails.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Change Status

Change the status of a record

NODE TYPE	NODE KEY
Action	`action.crm.change_status`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sets the status of the triggering record (contact, lead, company, deal, ticket, task or custom record).

When to use

- Close a ticket after a survey; mark a contact inactive after a bounce.

How it works

1. Loads the triggering record and sets its `status` column directly; same value → `noop`; writes an audit row `status_changed`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
new status `new_status`	Text	Yes	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **new status** `new_status`

Optional fields

- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`new_status`	New status	text
`noop`	Already that status	boolean

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **new status** must be filled in.
- New status is required.

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "entity not found or has no status" / "no new_status configured" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Close the ticket

```
{
  "new_status": "closed"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "new_status": "<new_status>",
  "output_variable": "new_contact",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Writes the column directly — the value is **not** checked against the module's allowed statuses, and status-change side effects (resolution timestamps, SLA clocks) do not run. Prefer the module-specific step where one exists: Update Task for tasks, Change Custom Record Status for custom records.
- Triggering record only.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Add Tag

Put a tag on the record — the same tags you filter and segment by

NODE TYPE	NODE KEY
Action	`action.crm.add_tag`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Adds a tag to a record — the same tags lists, segments and reports filter by.

When to use

- Mark contacts that hit a milestone; flag records for a later clean-up; feed a segment.

How it works

1. **Which record(s)** picks the target: the record that started the run (default), a record an earlier step found (`target_record_id` , usually `{{variables.found.id}}`), or every record matching a filter (up to **Change at most**, hard cap 500).
2. Adds the name to the record's `tags` list; already present → `noop` ; an audit row per change.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Tag `tag`	Picker (tag)	Yes	—	Pick one of your tags, or type a new name
Which record(s) `target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `found_record` = A record found earlier `matching` = Every record that matches a filter
Record type `target_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `campaign` `task` `ticket` Shown when target is found_record / matching

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record id `target_record_id`	Value or variable	No	—	Shown when target is found_record e.g. `{{variables.found.id}}`
Only records where `target_filters`	Filter rows	No	—	Shown when target is matching
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions Shown when target is matching
Tag at most `max_records`	Number	No	`50`	A guard, not a filter — the run says how many matched Shown when target is matching
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Tag** `tag`

Optional fields

- **Which record(s)** `target` — default `trigger_record`
- **Record type** `target_entity_type`
- **Record id** `target_record_id`
- **Only records where** `target_filters`
- **Match** `logic` — default `and`
- **Tag at most** `max_records` — default `50`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`tag`	Tag	text
`record_id`	Record tagged	id
`entity_type`	Record type	text
`tagged`	How many were tagged (matching records)	number
`record_ids`	Records tagged (matching records)	list

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a

value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Tag** must be filled in.
- Tag is required (pick one of the workspace's tags or type a new name).
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no tag configured" / "entity not found or not taggable" → skipped.
- "no record selected — set the record id (e.g. {{variables.found.id}})" / "'x' is not a record id" — the found-record target is blank or malformed → skipped.
- "this workflow's run-as user can't edit that record" — a found record outside the Run as user's visibility → skipped.

- "entity not found" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Tag every contact of the winning account

```
{
  "tag": "customer",
  "target": "matching",
  "target_entity_type": "contact",
  "target_filters": [
    {
      "field": "account_id",
      "operator": "eq",
      "value": "{{record.account_id}}"
    }
  ],
  "max_records": 200
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "tag": "<Tag>",
  "target": "trigger_record",
  "target_entity_type": "contact",
  "target_record_id": "{{variables.found.id}}",
  "target_filters": [],
  "logic": "and",
  "max_records": 50,
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- A tag typed here is written to the record but does not appear in Settings → Tags until created there.
- Matching mode changes at most **Change at most** records (default 50, never more than 500) and stops at the first record that fails.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Remove Tag

Take a tag off the record

NODE TYPE	NODE KEY
Action	`action.crm.remove_tag`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Takes a tag off a record.

When to use

- Clear a "needs follow-up" tag once the follow-up happened.

How it works

1. **Which record(s)** picks the target: the record that started the run (default), a record an earlier step found (`target_record_id` , usually `{{variables.found.id}}`), or every record matching a filter (up to **Change at most**, hard cap 500).
2. Removes the name from the `tags` list; already absent → `noop` .

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Tag `tag`	Picker (tag)	Yes	—	Pick one of your tags, or type the name
Which record(s) `target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `found_record` = A record found earlier `matching` = Every record that matches a filter
Record type `target_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `campaign` `task` `ticket` Shown when target is found_record / matching

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record id `target_record_id`	Value or variable	No	—	Shown when target is found_record e.g. `{{variables.found.id}}`
Only records where `target_filters`	Filter rows	No	—	Shown when target is matching
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions Shown when target is matching
Tag at most `max_records`	Number	No	`50`	A guard, not a filter — the run says how many matched Shown when target is matching
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Tag** `tag`

Optional fields

- **Which record(s)** `target` — default `trigger_record`
- **Record type** `target_entity_type`
- **Record id** `target_record_id`
- **Only records where** `target_filters`
- **Match** `logic` — default `and`
- **Tag at most** `max_records` — default `50`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`tag`	Tag	text
`record_id`	Record untagged	id
`entity_type`	Record type	text
`tagged`	How many were untagged (matching records)	number
`record_ids`	Records untagged (matching records)	list

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a

value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Tag** must be filled in.
- Tag is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no tag configured" / "entity not found or not taggable" → skipped.
- "no record selected — set the record id (e.g. {{variables.found.id}})" / "'x' is not a record id" — the found-record target is blank or malformed → skipped.
- "this workflow's run-as user can't edit that record" — a found record outside the Run as user's visibility → skipped.

- "entity not found" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "tag": "<Tag>",
  "target": "trigger_record",
  "target_entity_type": "contact",
  "target_record_id": "{{variables.found.id}}",
  "target_filters": [],
  "logic": "and",
  "max_records": 50,
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Matching mode changes at most **Change at most** records (default 50, never more than 500) and stops at the first record that fails.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Create Activity

Log a call, meeting, or note on a record

NODE TYPE	NODE KEY
Action	`action.crm.create_activity`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Logs a call, meeting, email or note on the record's timeline.

When to use

- Record that an automated email went out; note a status change with context; log a system call.

How it works

1. Links the activity to the triggering record; if the trigger is a task, to the task's own linked record instead. With no record it is created unlinked.
2. Owner defaults to whoever started the run. Creates through the activity service (its audit and events apply).

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
activity type `activity_type`	Choice	No	`call`	Options: `call` `meeting` `email` `note`
subject `subject`	Text	Yes	—	
description `description`	Multi-line text	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit)

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				`any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **subject** `subject`

Optional fields

- **activity type** `activity_type` — default `call`
- **description** `description`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`activity_id`	Activity id	id
`subject`	Subject	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **subject** must be filled in.
- Subject is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no subject configured" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — one activity per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Note the automated reminder

```
{
  "activity_type": "note",
  "subject": "Renewal reminder sent",
  "description": "Automation \"{{env.workflow_name}}\" emailed {{record.email}}."
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "activity_type": "call",
  "subject": "<subject>",
  "description": "<description>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · tasks

Create Task

Create a task and assign it

NODE TYPE	NODE KEY
Action	`action.tasks.create_task`

MODULE	OUTPUT PORTS
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Creates a task and assigns it, linked to the record the run is about (or any record you name).

When to use

- "Call within 24 h" on a new lead; prep tasks on a booking; a review task on a lost deal.

How it works

1. Builds the task from title, description, priority, **Due in N days** (from now) and assignee (default: whoever started the run; the owner is also the assignee).
2. **Linked record**: the triggering record (default), a specific record, or none. A bad custom id creates the task unlinked.
3. Creates through the task service — its defaults, audit and events (Task Created, Task Assigned) apply.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
title `title`	Text	Yes	—	
description `description`	Multi-line text	No	—	
assignee id `assignee_id`	User picker	No	—	
Due in N days `due_days`	Number	No	—	
priority `priority`	Choice	No	`medium`	Options: `low` `medium` `high` `urgent`
Linked record `link_target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `custom` = A specific record `none` = Not linked to any record

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `link_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `ticket` `campaign` `project` Shown when link_target is custom
Record id `link_record_id`	Value or variable	No	—	Shown when link_target is custom e.g. `{{variables.found.id}}` — or paste a record id
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **title** `title`

Optional fields

- **description** `description`
- **assignee id** `assignee_id`
- **Due in N days** `due_days`
- **priority** `priority` — default `medium`
- **Linked record** `link_target` — default `trigger_record`
- **Record type** `link_entity_type`
- **Record id** `link_record_id`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).

- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`task_id`	Task id	id
`title`	Title	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **title** must be filled in.
- Title is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no title configured" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts

back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — one task per run. Guard repeat triggers with Not again so soon.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Follow-up call in 2 days

Later: `{{variables.followup.task_id}}`.

```
{
  "title": "Call {{record.first_name}} about the quote",
  "due_days": 2,
  "priority": "high",
  "assignee_id": "{{record.owner_id}}",
  "link_target": "trigger_record",
  "output_variable": "followup"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "title": "<title>",
  "description": "<description>",
  "assignee_id": "<user id>",
  "due_days": 0,
  "priority": "medium",
  "link_target": "trigger_record",
  "link_entity_type": "contact",
  "link_record_id": "{{variables.found.id}} — or paste a record id",
  "output_variable": "new_contact",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · tasks

Update Task

Change a task's status, priority, due date, owner or linked record

NODE TYPE	NODE KEY
Action	`action.tasks.update_task`

MODULE	OUTPUT PORTS
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Changes a task's status, priority, due date, owner or linked record — only the settings you fill in.

When to use

- Bump priority when the linked ticket is escalated; re-link a task to a found record.

How it works

1. **Which task**: a task from an earlier step (`{{nodes.create_task.task_id}}` or a variable), or, if blank, the task that started the run.
2. Only non-blank settings are applied through the task service; **Linked record** = Leave as it is by default.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Which task `task_ref`	Variable reference	No	—	The task from an earlier step — { } Insert, e.g. {{nodes.create_task.task_id}}. Empty = the task that started this run.
Status `status`	Text	No	—	A status code from Tasks → Statuses. Leave empty to keep it.
Priority `priority`	Choice	No	—	Options: `(blank)` = Leave as it is `low` `medium` `high` `urgent`
Due in N days (from now) `due_days`	Number	No	—	
Owner `owner_id`	User picker	No	—	
Linked record `link_target`	Choice	No	`keep`	Options: `keep` = Leave as it is `trigger_record` = The record that started this run

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				`custom` = A specific record `none` = Not linked to any record
Record type `link_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `ticket` `campaign` `project` Shown when link_target is custom
Record id `link_record_id`	Value or variable	No	—	Shown when link_target is custom e.g. `{{variables.found.id}} — or paste a record id`
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Which task** `task_ref`
- **Status** `status`
- **Priority** `priority`
- **Due in N days (from now)** `due_days`
- **Owner** `owner_id`
- **Linked record** `link_target` — default `keep`
- **Record type** `link_entity_type`
- **Record id** `link_record_id`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`task_id`	Task updated	id
`changed`	Fields changed	list

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no task_ref and entity is not a task" / "task not found" / "nothing configured to change" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether

raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent (absolute values); Due in N days is recomputed from now on each run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Mark the created task urgent

```
{
  "task_ref": "{{variables.followup.task_id}}",
  "priority": "urgent"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "task_ref": "{{variables.something}}",
  "status": "<Status>",
  "priority": "",
  "due_days": 0,
  "owner_id": "<user id>",
  "link_target": "keep",
  "link_entity_type": "contact",
  "link_record_id": "{{variables.found.id}} — or paste a record id",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · tasks

Complete Task

Mark a task as complete

NODE TYPE	NODE KEY
Action	`action.tasks.complete_task`

MODULE	OUTPUT PORTS
tasks (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Marks a task done.

When to use

- Close the "await reply" task when the reply arrives.

How it works

1. Resolves the task as in Update Task and sets its status to done through the task service, so completion side effects (activity logging, Task Completed event) run.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Which task `task_ref`	Variable reference	No	—	The task from an earlier step — { } Insert, e.g. {{nodes.create_task.task_id}}
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Which task** `task_ref`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`task_id`	Task completed	id

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **tasks** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- No task / "task not found" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "task_ref": "{{variables.something}}",
  "output_variable": "new_contact",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Send Notification

Send an in-app notification to a user or team

NODE TYPE	NODE KEY
Action	`action.notify.send_notification`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Puts an in-app notification in one or more people's inbox — a person, a team, everyone with a role, or the record's owner.

When to use

- Alert the owner of a change; tell a queue about a breach; nudge a manager.

How it works

1. Resolves recipients: a user (default: whoever started the run), a team (active members plus the manager), a role, or the triggering record's owner.
2. Writes an inbox notification with title, message, priority and an optional **Link** (an Open button).

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Notify `recipient_type`	Choice	No	—	Options: `user` = A person `team` = A team `role` = Everyone with a role `record_owner` = Whoever owns the record
User `recipient_id`	User picker	No	—	Shown when recipient_type is (blank) / user
Team `team_id`	Picker (team)	No	—	Shown when recipient_type is team
Role `role_id`	Picker (role)	No	—	Shown when recipient_type is role
title `title`	Text	Yes	—	
message `message`	Multi-line text	Yes	—	
Link URL (optional) `link`	Text	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				{{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **title** `title`
- **message** `message`

Optional fields

- **Notify** `recipient_type`
- **User** `recipient_id`
- **Team** `team_id`
- **Role** `role_id`
- **Link URL (optional)** `link`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`notification_id`	Notification id	id
`recipients`	People notified (user ids)	list

KEY	MEANING	TYPE
`title`	Title	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level	After the step that sets it has run.

VARIABLE	WHAT IT IS	AVAILABLE
	default.	
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **title** must be filled in.
- **message** must be filled in.
- Title is required — without one the step is skipped with "no title configured". Message is the notification body (the designer's Validate also expects it).
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no recipients for recipient_type 'x'" / "recipient not a user in this tenant" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — one notification per recipient per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Tell the owner

```
{
  "recipient_type": "record_owner",
  "title": "SLA breached on {{trigger.ticket_number}}",
  "message": "{{trigger.breach_type}} target missed by {{trigger.actual_minutes}} minutes.",
  "link": "/service-desk/tickets/{{entity.id}}"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "recipient_type": "user",
  "recipient_id": "<user id>",
  "team_id": "<Team>",
  "role_id": "<Role>",
  "title": "<title>",
  "message": "<message>",
  "link": "<Link URL (optional)>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · omnichannel

Send Email

Send an email via the messaging system

NODE TYPE	NODE KEY
Action	`action.notify.send_email`

MODULE	OUTPUT PORTS
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sends one email message to a contact through the messaging system (Omnichannel), with the same consent, compliance and delivery tracking as a campaign send.

When to use

- An acknowledgement, confirmation or alert to the person the run is about — a new ticket, a booking, a deal stage.
- A one-to-one follow-up from an automation, where a campaign would be too heavy.

How it works

1. Resolves the recipient: **Send to contact (id)** if set (it must be visible to the Run as user); otherwise the triggering record's contact — the contact itself, or a ticket's requester. A literal **Recipient** overrides the address.
2. **Template**: an active email template supplies subject and body; otherwise **Subject** + **Body** are used. A body containing `<` is sent as HTML.
3. `[[survey_url]]` in the body is replaced with a per-contact signed survey link (30-day token) when a survey is referenced.
4. Checks the message type (marketing needs opt-in), then hands the email to the orchestration service.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Recipient (variable or email) `to`	Text	No	—	
Send to contact (id) `contact_id`	Text	No	—	e.g. `{{variables.found.id}}` — empty = the triggering record's contact
Template `template_id`	Record picker (email_template)	No	—	Optional — its subject and body are used instead of

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				the ones below
subject `subject`	Text	No	—	
body `body`	Rich text (HTML)	No	—	
Message type `message_type`	Choice	No	`transactional`	Options: `transactional` = Transactional — about something they did or asked for `marketing` = Marketing — needs their opt-in, like a campaign
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Recipient (variable or email)** `to`
- **Send to contact (id)** `contact_id`
- **Template** `template_id`
- **subject** `subject`
- **body** `body`
- **Message type** `message_type` — default `transactional`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`

- **Retry at most (times)** `retry_attempts`

Inputs

- The contact (from the setting or the triggering record).
- The message text / template, with placeholders resolved.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`message_id`	Message id	id
`contact_id`	Contact messaged	id
`recipient`	Sent to (address / number / chat id)	text
`channel`	Channel	text
`orchestration_status`	Send status (queued / scheduled)	text

Execution behaviour

With `automation/external_jobs` on, the send runs on the send job worker (see the Overview) and the run waits for it. Queues the message; "success" means it was accepted for delivery (status queued or scheduled), not that it was delivered. Delivery outcomes arrive later as Message Delivered / Message Failed events. In a Test run the send is simulated.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A send the compliance rules block is **skipped** ("blocked: …") and the run carries on; any other outcome than queued / scheduled from the orchestrator fails the step. If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- The Omnichannel module with a configured provider for this channel.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "not a template id" / "isn't a email template (deleted?)" / "is not active" → skipped.

- "the contact has no email address" / "no contact to message and no address given" → skipped.
- "marketing needs the contact's email opt-in …" / "blocked: …" → skipped.
- "nothing to send — add a message or pick a template" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — every run sends another message. Put a Not again so soon step before it when the trigger can fire repeatedly for the same person.

Transaction / rollback

A queued message cannot be recalled by the engine. A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Ticket acknowledgement

```
{
  "subject": "We received your request #{{trigger.ticket_number}}",
  "body": "<p>Hi {{record.first_name}},</p><p>Thanks — we will reply within one business day.</p>",
  "message_type": "transactional"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "to": "<Recipient (variable or email)>",
  "contact_id": "{{variables.found.id}} — empty = the triggering record's contact",
  "template_id": "<email_template id>",
  "subject": "<subject>",
```

```
  "body": "<body>",
  "message_type": "transactional",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result and branch on `{{variables.<name>.orchestration_status}}` if the next step depends on the send being accepted.

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Deal, company, task and lead triggers do not resolve a contact automatically — set **Send to contact** (e.g. from a Find Records step) or a literal **to**.
- Success means accepted for delivery, not delivered.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The message itself is logged in the messaging module with its delivery status.

Security considerations

Personal data in the message body is stored with the message. Marketing sends are gated on consent; transactional ones are not — do not label promotional content transactional.

Related nodes

ACTIONS · omnichannel

Send SMS

Send an SMS via the messaging system

NODE TYPE	NODE KEY
Action	`action.notify.send_sms`

MODULE	OUTPUT PORTS
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sends one SMS message to a contact through the messaging system (Omnichannel), with the same consent, compliance and delivery tracking as a campaign send.

When to use

- An acknowledgement, confirmation or alert to the person the run is about — a new ticket, a booking, a deal stage.
- A one-to-one follow-up from an automation, where a campaign would be too heavy.

How it works

1. Resolves the recipient: **Send to contact (id)** if set (it must be visible to the Run as user); otherwise the triggering record's contact — the contact itself, or a ticket's requester. A literal **to** value overrides the address.
2. Reads the address from the contact (mobile or phone number).
3. Checks the message type: **marketing** needs the contact's opt-in for this channel; **transactional** (default) does not.
4. Hands the message to the orchestration service, which applies compliance rules, queues it and tracks delivery.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Send to contact (id) `contact_id`	Text	No	—	e.g. `{{variables.found.id}} – empty = the triggering record's contact`
...or this number `to`	Text	No	—	e.g. `+15551234567`
message `message`	Multi-line text	Yes	—	
Message type `message_type`	Choice	No	`transactional`	Options: `transactional` = Transactional — about something they did or asked for `marketing` = Marketing — needs their opt-in, like a campaign

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **message** `message`

Optional fields

- **Send to contact (id)** `contact_id`
- **...or this number** `to`
- **Message type** `message_type` — default `transactional`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The contact (from the setting or the triggering record).
- The message text / template, with placeholders resolved.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`message_id`	Message id	id
`contact_id`	Contact messaged	id
`recipient`	Sent to (address / number / chat id)	text

KEY	MEANING	TYPE
`channel`	Channel	text
`orchestration_status`	Send status (queued / scheduled)	text

Execution behaviour

With `automation/external_jobs` on, the send runs on the send job worker (see the Overview) and the run waits for it. Queues the message; "success" means it was accepted for delivery (status queued or scheduled), not that it was delivered. Delivery outcomes arrive later as Message Delivered / Message Failed events. In a Test run the send is simulated.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A send the compliance rules block is **skipped** ("blocked: …") and the run carries on; any other outcome than queued / scheduled from the orchestrator fails the step. If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- The Omnichannel module with a configured provider for this channel.

Validation rules

- **message** must be filled in.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no contact to message and no address given" → skipped (typical for deal, company, task or lead triggers — set **Send to contact** or **to**).
- "the contact has no mobile or phone number" → skipped.
- "marketing needs the contact's opt-in …" → skipped. "blocked: …" → skipped by a compliance rule.
- "nothing to send — add a message or pick a template" → skipped.
- Provider or orchestration failure → the step fails.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be

retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — every run sends another message. Put a Not again so soon step before it when the trigger can fire repeatedly for the same person.

Transaction / rollback

A queued message cannot be recalled by the engine. A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Booking reminder

```
{
  "message": "Reminder: your call with us is at {{trigger.booking.start_datetime}}.",
  "message_type": "transactional"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "contact_id": "{{variables.found.id}} — empty = the triggering record's contact",
  "to": "+15551234567",
  "message": "<message>",
  "message_type": "transactional",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result and branch on `{{variables.<name>.orchestration_status}}` if the next step depends on the send being accepted.

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Deal, company, task and lead triggers do not resolve a contact automatically — set **Send to contact** (e.g. from a Find Records step) or a literal **to**.
- Success means accepted for delivery, not delivered.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The message itself is logged in the messaging module with its delivery status.

Security considerations

Personal data in the message body is stored with the message. Marketing sends are gated on consent; transactional ones are not — do not label promotional content transactional.

Related nodes

ACTIONS · omnichannel

Send WhatsApp

Send a WhatsApp message — an approved template, or free text

NODE TYPE	NODE KEY
Action	`action.notify.send_whatsapp`
MODULE	**OUTPUT PORTS**
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sends one WhatsApp message to a contact through the messaging system (Omnichannel), with the same consent, compliance and delivery tracking as a campaign send.

When to use

- An acknowledgement, confirmation or alert to the person the run is about — a new ticket, a booking, a deal stage.
- A one-to-one follow-up from an automation, where a campaign would be too heavy.

How it works

1. Resolves the recipient as the other send steps do.
2. **Approved template** + **Template values** sends a provider template, which WhatsApp delivers at any time.
3. **Message (free text)**, or a template not yet approved, is only accepted within 24 hours of the contact's last inbound WhatsApp message — otherwise the step is skipped.
4. Hands the message to the orchestration service.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Send to contact (id) `contact_id`	Text	No	—	e.g. `{{variables.found.id}}` — empty = the triggering record's contact
...or this number `to`	Text	No	—	e.g. `+15551234567`
Approved template `template_id`	Record picker (whatsapp_template)	No	—	Delivers at any time. Without one, WhatsApp only accepts a message within 24 hours of the

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				customer's last message to you.
Template values `template_variables`	Rows of key / value	No	—	One row per {{placeholder}} in the template
Message (free text) `message`	Multi-line text	No	—	Used when no template is picked — only within the 24-hour window
Message type `message_type`	Choice	No	`transactional`	Options: `transactional` = Transactional — about something they did or asked for `marketing` = Marketing — needs their opt-in, like a campaign
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Send to contact (id)** `contact_id`
- **...or this number** `to`
- **Approved template** `template_id`

- **Template values** `template_variables`
- **Message (free text)** `message`
- **Message type** `message_type` — default `transactional`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The contact (from the setting or the triggering record).
- The message text / template, with placeholders resolved.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`message_id`	Message id	id
`contact_id`	Contact messaged	id
`recipient`	Sent to (address / number / chat id)	text
`channel`	Channel	text
`orchestration_status`	Send status (queued / scheduled)	text
`as_template`	Sent as an approved template	boolean

Execution behaviour

With `automation/external_jobs` on, the send runs on the send job worker (see the Overview) and the run waits for it. Queues the message; "success" means it was accepted for delivery (status queued or scheduled), not that it was delivered. Delivery outcomes arrive later as Message Delivered / Message Failed events. In a Test run the send is simulated.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A send the compliance rules block is **skipped** ("blocked: …") and the run carries on; any other outcome than queued / scheduled from the orchestrator fails the step. If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the

default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.

- The Omnichannel module with a configured provider for this channel.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "... outside WhatsApp's 24-hour window ... — pick an approved template" → skipped.
- "the contact has no mobile or phone number for WhatsApp" / "no contact to message and no address given" → skipped.
- "marketing needs the contact's opt-in ..." / "blocked: ..." → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — every run sends another message. Put a Not again so soon step before it when the trigger can fire repeatedly for the same person.

Transaction / rollback

A queued message cannot be recalled by the engine. A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Approved booking template

```
{
  "template_id": "<whatsapp template id>",
  "template_variables": [
    {
```

```
      "key": "1",
      "value": "{{record.first_name}}"
    },
    {
      "key": "2",
      "value": "{{trigger.booking.start_datetime}}"
    }
  ],
  "message_type": "transactional"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "contact_id": "{{variables.found.id}} — empty = the triggering record's contact",
  "to": "+15551234567",
  "template_id": "<whatsapp_template id>",
  "template_variables": [],
  "message": "<Message (free text)>",
  "message_type": "transactional",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result and branch on `{{variables.<name>.orchestration_status}}` if the next step depends on the send being accepted.

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Free text only inside the 24-hour customer-service window.
- Deal, company, task and lead triggers do not resolve a contact automatically.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The message itself is logged in the messaging module with its delivery status.

Security considerations

Personal data in the message body is stored with the message. Marketing sends are gated on consent; transactional ones are not — do not label promotional content transactional.

Related nodes

ACTIONS · omnichannel

Send Telegram Message

Message a contact on Telegram

NODE TYPE	NODE KEY
Action	`action.notify.send_telegram`

MODULE	OUTPUT PORTS
omnichannel (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sends one Telegram message to a contact through the messaging system (Omnichannel), with the same consent, compliance and delivery tracking as a campaign send.

When to use

- An acknowledgement, confirmation or alert to the person the run is about — a new ticket, a booking, a deal stage.
- A one-to-one follow-up from an automation, where a campaign would be too heavy.

How it works

1. Resolves the recipient: **Send to contact (id)** if set (it must be visible to the Run as user); otherwise the triggering record's contact — the contact itself, or a ticket's requester. A literal **to** value overrides the address.
2. Reads the address from the contact (verified Telegram chat id).
3. Checks the message type: **marketing** needs the contact's opt-in for this channel; **transactional** (default) does not.
4. Hands the message to the orchestration service, which applies compliance rules, queues it and tracks delivery.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Send to contact (id) `contact_id`	Text	No	—	e.g. `{{variables.found.id}} — empty = the triggering record's contact`
...or this chat id `to`	Text	No	—	e.g. `Numeric Telegram chat id — contacts use their verified one`
message `message`	Multi-line text	Yes	—	
Message type `message_type`	Choice	No	`transactional`	Options: `transactional` = Transactional — about something they did or asked for `marketing` = Marketing — needs their opt-in, like a campaign

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **message** `message`

Optional fields

- **Send to contact (id)** `contact_id`
- **...or this chat id** `to`
- **Message type** `message_type` — default `transactional`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The contact (from the setting or the triggering record).
- The message text / template, with placeholders resolved.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`message_id`	Message id	id
`contact_id`	Contact messaged	id
`recipient`	Sent to (address / number / chat id)	text

KEY	MEANING	TYPE
`channel`	Channel	text
`orchestration_status`	Send status (queued / scheduled)	text

Execution behaviour

With `automation/external_jobs` on, the send runs on the send job worker (see the Overview) and the run waits for it. Queues the message; "success" means it was accepted for delivery (status queued or scheduled), not that it was delivered. Delivery outcomes arrive later as Message Delivered / Message Failed events. In a Test run the send is simulated.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A send the compliance rules block is **skipped** ("blocked: …") and the run carries on; any other outcome than queued / scheduled from the orchestrator fails the step. If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **omnichannel** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- The Omnichannel module with a configured provider for this channel.

Validation rules

- **message** must be filled in.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "the contact has no Telegram chat (they haven't messaged your bot)" → skipped.
- "no contact to message and no address given" → skipped.
- "nothing to send — add a message" → skipped (no template option on Telegram).
- "marketing needs the contact's opt-in …" / "blocked: …" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be

retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — every run sends another message. Put a Not again so soon step before it when the trigger can fire repeatedly for the same person.

Transaction / rollback

A queued message cannot be recalled by the engine. A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "contact_id": "{{variables.found.id}} — empty = the triggering record's contact",
  "to": "Numeric Telegram chat id — contacts use their verified one",
  "message": "<message>",
  "message_type": "transactional",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

- ✓ Save the result and branch on `{{variables.<name>.orchestration_status}}` if the next step depends on the send being accepted.
- ✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Only contacts who have messaged your bot (and so have a verified chat id) can be reached; **...or this chat id** takes a literal numeric id.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The message itself is logged in the messaging module with its delivery status.

Security considerations

Personal data in the message body is stored with the message. Marketing sends are gated on consent; transactional ones are not — do not label promotional content transactional.

Related nodes

ACTIONS · campaigns

Add to Campaign

Add a contact to a campaign

NODE TYPE	NODE KEY
Action	`action.campaigns.add_to_campaign`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Adds the contact to a campaign's static audience.

When to use

- Enrol a new lead in a nurture campaign; add survey respondents to a follow-up.

How it works

1. Resolves the contact (the triggering contact or a ticket's requester) and adds its id to the campaign's first audience's static list; already in it → `noop`; an audit row on change.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
campaign id `campaign_id`	Record picker (campaign)	Yes	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **campaign id** `campaign_id`

Optional fields

- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`campaign_id`	Campaign id	id
`contact_id`	Contact id	id
`noop`	Already in it (nothing changed)	boolean

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **campaign id** must be filled in.
- Campaign is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no campaign_id configured" / "no contact resolved" / "campaign audience not found in tenant" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "campaign_id": "<campaign id>",
  "output_variable": "new_contact",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Consent is **not** checked here — the campaign's own send gate checks it at send time.
- Only the campaign's first audience is edited.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · campaigns

Remove from Campaign

Remove a contact from a campaign

NODE TYPE	NODE KEY
Action	`action.campaigns.remove_from_campaign`
MODULE	**OUTPUT PORTS**
campaigns (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Removes the contact from a campaign's static audience.

When to use

- Stop a nurture when the deal is won; honour an opt-out.

How it works

1. Mirror of Add to Campaign; not in it → `noop`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
campaign id `campaign_id`	Record picker (campaign)	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **campaign id** `campaign_id`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`campaign_id`	Campaign id	id
`contact_id`	Contact id	id
`noop`	Wasn't in it (nothing changed)	boolean

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **campaigns** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no campaign_id configured" / "no contact resolved" / "campaign audience not found in tenant" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "campaign_id": "<campaign id>",
  "output_variable": "new_contact",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · service_desk

Create Ticket

Create a new support ticket

NODE TYPE	NODE KEY
Action	`action.service_desk.create_ticket`

MODULE	OUTPUT PORTS
service_desk (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Creates a support ticket, by default for the contact the run is about.

When to use

- Turn a detractor NPS or a failed message into a ticket; open a ticket from a webhook.

How it works

1. Builds the ticket (subject, description, priority, assignee; requester = the triggering contact or ticket requester; source "internal") and creates it through the ticket service — SLA clocks, numbering, audit and the Ticket Created event all apply.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
subject `subject`	Text	Yes	—	
description `description`	Multi-line text	No	—	
priority `priority`	Choice	No	`medium`	Options: `low` `medium` `high` `urgent` `critical`
assignee id `assignee_id`	User picker	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit)

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				`any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **subject** `subject`

Optional fields

- **description** `description`
- **priority** `priority` — default `medium`
- **assignee id** `assignee_id`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`ticket_id`	Ticket id	id
`ticket_number`	Ticket number	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **service_desk** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **subject** must be filled in.
- Subject is required and at least 3 characters.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "subject missing or too short (min 3 chars)" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — one ticket per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Ticket for a detractor

```
{
  "subject": "NPS {{trigger.nps_score}} from {{record.first_name}} {{record.last_name}}",
  "description": "Survey: {{trigger.survey_name}}",
  "priority": "high"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "subject": "<subject>",
  "description": "<description>",
  "priority": "medium",
  "assignee_id": "<user id>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · service_desk

Escalate Ticket

Escalate a ticket to a higher tier

NODE TYPE	NODE KEY
Action	`action.service_desk.escalate_ticket`

MODULE	OUTPUT PORTS
service_desk (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Records an escalation on the triggering ticket — level, who it goes to, why.

When to use

- After an SLA breach; when a VIP ticket is still open after a delay.

How it works

1. Inserts an escalation record on the ticket and an audit row `escalated`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Escalate to level `escalation_level`	Choice	No	—	Options: `tier_2` = Tier 2 `tier_3` = Tier 3 `manager` = A manager
Escalate to `escalated_to`	User picker	No	—	
Reason `reason`	Text	No	—	e.g. `Why it is being escalated`
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Escalate to level** `escalation_level`
- **Escalate to** `escalated_to`
- **Reason** `reason`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`escalation_id`	Escalation id	id
`level`	Escalation level	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **service_desk** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "escalate_ticket only applies to ticket workflows" / "ticket not found" / "escalated_to is not a user in this tenant" → skipped.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — one escalation record per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "escalation_level": "tier_2",
  "escalated_to": "<user id>",
```

```
  "reason": "Why it is being escalated",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Records the escalation only: it does **not** change the ticket's assignee or priority. Add Assign Owner / Update Field for that.
- Ticket workflows only.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · surveys

Send Survey

Send a survey to a contact

NODE TYPE	NODE KEY
Action	`action.surveys.send_survey`
MODULE	**OUTPUT PORTS**
surveys (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Emails a survey link to the contact the run is about.

When to use

- CSAT after a ticket is resolved; NPS after onboarding.

How it works

1. Resolves the contact (triggering contact or ticket requester only), builds a per-contact survey link and sends a transactional email with the default subject and body through the orchestrator.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Which survey `survey_id`	Record picker (survey)	Yes	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Which survey** `survey_id`

Optional fields

- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`contact_id`	Contact messaged	id
`orchestration_status`	Send status	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **surveys** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Which survey** must be filled in.
- Which survey is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no recipient contact resolved" / "no survey_id/survey_slug configured" → skipped.
- "blocked: …" (compliance / consent) → skipped.
- A send failure or an exception → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — one email per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "survey_id": "<survey id>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No **Send to contact** override — the contact must come from the trigger.
- A blocked send is skipped and a failed one fails the step, like the other send steps. It does not check up front that the contact has an email; the orchestrator reports that as a failure.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · goals

Update Key Result

Update a key result progress value

NODE TYPE	NODE KEY
Action	`action.goals.update_kr_value`

MODULE	OUTPUT PORTS
goals (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sets a key result's current value.

When to use

- Push a computed number (an Aggregate result) into a goal.

How it works

1. Updates the key result through the goal service with source "workflow", which recomputes progress and may fire goal triggers.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Key result `kr_id`	Record picker (key_result)	Yes	—	
New value `value`	Number	Yes	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit)

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				`any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Key result** `kr_id`
- **New value** `value`

Optional fields

- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`kr_id`	Key result id	id
`value`	New value	number

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **goals** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Key result** must be filled in.
- **New value** must be filled in.
- Key result and a numeric value are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "needs kr_id and value" / "value is not numeric" / "key result not found" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent (absolute value).

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Pipeline value into a KR

```
{
  "kr_id": "<key result id>",
```

```
  "value": "{{variables.pipeline.result}}"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "kr_id": "<key_result id>",
  "value": 0,
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · projects

Create Work Item

Create a project work item / task

NODE TYPE	NODE KEY
Action	`action.projects.create_work_item`

MODULE	OUTPUT PORTS
projects (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Creates a work item (task, bug, story or epic) in a project.

When to use

- Open an onboarding epic when a deal is won.

How it works

1. Creates through the project service as the run's acting user.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
project id `project_id`	Record picker (project)	No	—	
title `title`	Text	Yes	—	
type `type`	Choice	No	`task`	Options: `task` `bug` `story` `epic`
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **title** `title`

Optional fields

- **project id** `project_id`
- **type** `type` — default `task`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`work_item_id`	Work item id	id
`title`	Title	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.

VARIABLE	WHAT IT IS	AVAILABLE
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **projects** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **title** must be filled in.
- Project and title are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "needs project_id and title" → skipped.
- "no acting user" → skipped — typical for event-triggered runs, which have nobody who "started" them. Use a Manual trigger or a workflow with a Run as user.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "project_id": "<project id>",
  "title": "<title>",
  "type": "task",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- Needs an acting user; pure event-triggered runs may skip.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · calendar

Create Booking

Create a calendar booking

NODE TYPE	NODE KEY
Action	`action.calendar.create_booking`

MODULE	OUTPUT PORTS
calendar (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Books a slot on a booking event type for a named invitee.

When to use

- Auto-book an onboarding call after signup; rebook a no-show.

How it works

1. Creates the booking through the booking service, which checks the host's availability for the slot and sends the event type's own notifications.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
event type id `event_type_id`	Record picker (event_type)	Yes	—	
Starts at `start_datetime`	Text	Yes	—	e.g. `2026-10-01T09:00:00Z` or `{{variables.slot}}`
timezone `timezone`	Time zone	No	—	
invitee name `invitee_name`	Text	Yes	—	e.g. `{{record.first_name}} {{record.last_name}}`
invitee email `invitee_email`	Text	Yes	—	e.g. `{{record.email}}`
invitee phone `invitee_phone`	Text	No	—	e.g. `{{record.phone}}`
notes `notes`	Multi-line text	No	—	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **event type id** `event_type_id`
- **Starts at** `start_datetime`
- **invitee name** `invitee_name`
- **invitee email** `invitee_email`

Optional fields

- **timezone** `timezone`
- **invitee phone** `invitee_phone`
- **notes** `notes`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`booking_id`	Booking id	id

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **calendar** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **event type id** must be filled in.
- **Starts at** must be filled in.
- **invitee name** must be filled in.
- **invitee email** must be filled in.
- Event type, Starts at, invitee name and email are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "needs event_type_id, start_datetime, invitee_name, invitee_email in config" → skipped.
- A slot conflict or an unparsable time → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — a second run books another slot (or fails on the conflict).

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Onboarding call in 3 working days

Compute the slot with Adjust time first.

```
{
  "event_type_id": "<event type id>",
  "start_datetime": "{{variables.slot.adjusted_time}}",
  "timezone": "Asia/Kolkata",
  "invitee_name": "{{record.first_name}} {{record.last_name}}",
  "invitee_email": "{{record.email}}"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "event_type_id": "<event_type id>",
  "start_datetime": "2026-10-01T09:00:00Z or {{variables.slot}}",
  "timezone": "Asia/Kolkata",
  "invitee_name": "{{record.first_name}} {{record.last_name}}",
  "invitee_email": "{{record.email}}",
  "invitee_phone": "{{record.phone}}",
  "notes": "<notes>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · link_management

Create Tracked Link

Create a short tracked link

NODE TYPE	NODE KEY
Action	`action.links.create_link`

MODULE	OUTPUT PORTS
link_management (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Creates a short tracked link so clicks can be attributed.

When to use

- A per-contact link in a message so a click fires Link Clicked.

How it works

1. Creates the link (destination up to 4096 characters, optional name) through the link service; returns the short code.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Link goes to `target_url`	Text	Yes	—	
Link name `title`	Text	No	—	
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Link goes to** `target_url`

Optional fields

- **Link name** `title`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`link_id`	Link id	id
`short_code`	Short code	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **link_management** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Link goes to** must be filled in.
- Link goes to is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no destination url configured" → skipped.
- A service error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — a new link per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "target_url": "<Link goes to>",
  "title": "<Link name>",
  "output_variable": "new_contact",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · custom_entities

Change Custom Record Status

Move a record of one of your own entities to another status

NODE TYPE	NODE KEY
Action	`action.custom.change_custom_status`

MODULE	OUTPUT PORTS
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Moves a record of one of your own entities to another of its statuses, through the record service so the entity's status model applies.

When to use

- Advance a custom "Application" to Reviewed when a task completes.

How it works

1. Resolves the record (default: the triggering record) and updates its status (and optionally owner) through the record service. A status the entity does not define is rejected.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	
Record `record_id`	Value or variable	Yes	—	e.g. `{{entity.id}} — the record that started the run`
New status `status`	Picker (custom_status)	Yes	—	
Save the result as `output_variable`	Text	No	`custom_record`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Custom entity** `entity_api_name`
- **Record** `record_id`
- **New status** `status`

Optional fields

- **Save the result as** `output_variable` — default `custom_record`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.custom_record}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`record_id`	Record id	id
`display_name`	Name	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Custom entity** must be filled in.
- **Record** must be filled in.
- **New status** must be filled in.
- Custom entity, record and new status are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "nothing to change" / "no custom entity selected" / "there is no custom entity called 'x'" / "<id> is not a record id" / "record not found" → skipped.
- An undefined status → failed with the service message.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>",
  "record_id": "{{entity.id}} — the record that started the run",
  "status": "<New status>",
  "output_variable": "custom_record",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · custom_entities

Submit Custom Record for Approval

Send a draft record of one of your own entities into its approval flow

NODE TYPE	NODE KEY
Action	`action.custom.submit_custom_record`

MODULE	OUTPUT PORTS
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Sends a draft custom-entity record into its approval flow.

When to use

- Auto-submit once every required field is filled.

How it works

1. Calls the record service's submit, which starts the entity's approval flow.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	
Record `record_id`	Value or variable	Yes	—	e.g. `{{entity.id}}`
Save the result as `output_variable`	Text	No	`submitted`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Custom entity** `entity_api_name`
- **Record** `record_id`

Optional fields

- **Save the result as** `output_variable` — default `submitted`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.submitted}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`record_id`	Record id	id
`entity_type`	Custom entity	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Custom entity** must be filled in.
- **Record** must be filled in.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no custom entity selected" / "there is no custom entity called 'x'" / "<id> is not a record id" / "record not found" → skipped.
- "already submitted", approval flow off, or another service refusal → failed with that message.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

A second submit fails ("already submitted").

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>",
  "record_id": "{{entity.id}}",
  "output_variable": "submitted",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · custom_entities

Create Custom Record

Create a record of one of your own entities

NODE TYPE	NODE KEY
Action	`action.custom.create_custom_record`
MODULE	**OUTPUT PORTS**
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Creates a record of one of your own entities — required fields, unique rules, lookups, status model and approval flow all apply exactly as on the record screen.

When to use

- Create the custom "Contract" when a deal is won; log a custom "Event" per webhook.

How it works

1. Maps **Field values** (api name → value), optional status (blank = the entity's first status), owner and **Submit for approval**, then creates through the record service.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	
Field values `field_mappings`	Rows of key / value	No	—	The entity's own fields. { } Insert works in a value.
Status `status`	Text	No	—	Leave blank for the entity's first status
Owner `owner_id`	User picker	No	—	
Submit for approval `submit`	Yes / no	No	`false`	Only for entities with an approval flow; otherwise it stays a draft
Save the result as `output_variable`	Text	No	`custom_record`	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Custom entity** `entity_api_name`

Optional fields

- **Field values** `field_mappings`
- **Status** `status`
- **Owner** `owner_id`
- **Submit for approval** `submit` — default `false`
- **Save the result as** `output_variable` — default `custom_record`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.custom_record}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`record_id`	Record id	id
`entity_type`	Custom entity	text
`display_name`	Name	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Custom entity** must be filled in.
- Custom entity is required; the entity's own required fields must be given.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no custom entity selected" / "there is no custom entity called 'x'" → skipped.
- A validation error (required, unique, lookup) → failed with the message.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent unless a unique rule on the entity rejects the duplicate. Use Find or create to check first.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Contract from a won deal

```
{
  "entity_api_name": "contract",
  "field_mappings": [
    {
      "key": "deal_id",
      "value": "{{entity.id}}"
    },
    {
      "key": "value",
      "value": "{{record.amount}}"
    }
  ],
  "submit": true,
  "output_variable": "contract"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>",
  "field_mappings": [],
  "status": "<Status>",
  "owner_id": "<user id>",
  "submit": false,
  "output_variable": "custom_record",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · custom_entities

Update Custom Record

Change fields on a record of one of your own entities

NODE TYPE	NODE KEY
Action	`action.custom.update_custom_record`

MODULE	OUTPUT PORTS
custom_entities (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Changes fields on a custom-entity record — only the fields you list are touched.

When to use

- Stamp a date, copy a value, change the owner on a custom record.

How it works

1. Resolves the record (default: the triggering record), merges the listed fields plus optional status / owner, and updates through the record service.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Custom entity `entity_api_name`	Picker (custom_entity)	Yes	—	
Record `record_id`	Value or variable	Yes	—	e.g. `{{entity.id}} — the record that started the run`
Field values `field_mappings`	Rows of key / value	No	—	Only the fields you list are touched
Status `status`	Text	No	—	Leave blank to keep it
Owner `owner_id`	User picker	No	—	
Save the result as `output_variable`	Text	No	`custom_record`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Custom entity** `entity_api_name`
- **Record** `record_id`

Optional fields

- **Field values** `field_mappings`
- **Status** `status`
- **Owner** `owner_id`
- **Save the result as** `output_variable` — default `custom_record`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.custom_record}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`record_id`	Record id	id
`entity_type`	Custom entity	text
`display_name`	Name	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.

VARIABLE	WHAT IT IS	AVAILABLE
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **custom_entities** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Custom entity** must be filled in.
- **Record** must be filled in.
- Custom entity and record are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- Nothing to change / bad id / not found / entity missing → skipped.
- A validation error → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Idempotent.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_api_name": "<Custom entity>",
  "record_id": "{{entity.id}} — the record that started the run",
  "field_mappings": [],
  "status": "<Status>",
  "owner_id": "<user id>",
  "output_variable": "custom_record",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Start Approval

Request approval from one or more users

NODE TYPE	NODE KEY
Action	`action.core.start_approval`

MODULE	OUTPUT PORTS
core (always available)	approved, rejected

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Pauses the run and asks one or more people to approve or reject — optionally answering questions or picking from several choices — then continues down the matching path.

When to use

- Discounts over a threshold; refunds; publishing a campaign; anything a person must sign off.

How it works

1. Creates an approval request for the **Approvers** (any one / all / one after another) with the message, the questions (**Ask the approver for**) and the choices, and parks the run as **Waiting** — no `resume_at`; it waits for the decision.
2. The approvers are notified at once and the request appears in **Automation** → **Approvals** (one inbox for approvals, questions and tool calls, tagged with why it is theirs), with reminders after **Remind them after (hours)** and a deadline of **Decide within (hours)** (working hours only, if set).
3. On the deadline, **If nobody decides in time** does nothing, hands it to someone else, or treats it as approved / rejected.
4. Approved → the `approved` port (a plain `default` wire also counts as approved). Rejected → the `rejected` port; if none is wired the run **fails** with the reason. A choice → the port named after it.
5. Answers land in `{{variables.<name>.<field>}}`; the choice in `.decision`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
approvers `approvers`	User picker, many	No	—	
approval type `approval_type`	Choice	No	`any`	Options: `any` `all` `sequential`
message `message`	Multi-line text	No	—	
Ask the approver for `fields`	Question rows	No	—	Answers arrive as {{variables.<name>.<field name>}}

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Or offer these choices `choices`	List of names	No	—	Leave blank for plain Approve / Reject. Each choice gets its own path out of this step. e.g. `Accept, Counter-offer, Send to legal`
Save the answers as `output_variable`	Text	No	`approval`	Shown when fields is filled in
Decide within (hours) `due_hours`	Number	No	—	
Count working hours only `business_hours`	Yes / no	No	—	A 4-hour deadline set at 4pm falls due at 11am tomorrow, not at 8pm tonight Shown when due_hours is filled in
Remind them after (hours) `remind_after_hours`	Number	No	—	
If nobody decides in time `on_timeout`	Choice	No	`keep_waiting`	Options: `keep_waiting` = Keep waiting (do nothing) `escalate` = Hand it to someone else `approved` = Treat as approved `rejected` = Treat as rejected Shown when due_hours is filled in
Hand it to `escalate_to`	User picker	No	—	Shown when on_timeout is escalate
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **approvers** `approvers`
- **approval type** `approval_type` — default `any`
- **message** `message`
- **Ask the approver for** `fields`
- **Or offer these choices** `choices`
- **Save the answers as** `output_variable` — default `approval`
- **Decide within (hours)** `due_hours`
- **Count working hours only** `business_hours`
- **Remind them after (hours)** `remind_after_hours`
- **If nobody decides in time** `on_timeout` — default `keep_waiting`
- **Hand it to** `escalate_to`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The approvers, the record, the resolved message.

Outputs

The result is saved as `{{variables.approval}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`approved`	Approved (or the deadline treated it as approved).
`rejected`	Rejected — unwired means the run fails.
`<one per choice>`	When **Or offer these choices** is set, one port per choice, named by its slug.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Approvers act with their own identity; the record stays visible to them through the approval, not through the run.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- At least one approver is expected; the deadline settings only apply when Decide within is set.

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- If creating the request fails, the error is only logged and the run stays **Waiting** with nothing in anyone's inbox — cancel and fix the settings.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

None on the wait itself except your own **Decide within** deadline. Time spent waiting is not charged to the run's Time limit.

Idempotency

One request per run. Re-running creates another request.

Transaction / rollback

Cancelling the run leaves the approval request behind; decide or withdraw it in the inbox.

Examples

Manager approves a discount

Next step: `{{variables.approval.approved_pct}}`.

```
{
  "approvers": [
    "<manager user id>"
  ],
  "approval_type": "any",
  "message": "{{record.name}} asks for {{trigger.discount_pct}}% off — approve?",
  "fields": [
    {
      "key": "approved_pct",
      "label": "Discount you approve (%)"
    }
  ],
  "due_hours": 24,
  "business_hours": true,
  "remind_after_hours": 8,
  "on_timeout": "escalate",
```

```
  "escalate_to": "<director user id>",
  "output_variable": "approval"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "approvers": [
    "<user id>"
  ],
  "approval_type": "any",
  "message": "<message>",
  "fields": [],
  "choices": [],
  "output_variable": "approval",
  "due_hours": 0,
  "business_hours": false,
  "remind_after_hours": 0,
  "on_timeout": "keep_waiting",
  "escalate_to": "<user id>",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Always wire **rejected** — an unwired rejection fails the whole run.

✓ In a Test run the approval is assumed approved and only that path runs.

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The approval request itself keeps who decided, when and the answers.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not

re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · core

Ask a person

Pause and ask someone to decide, or to fill in what only they know — the run carries on down the path they choose

NODE TYPE	NODE KEY
Action	`action.core.ask_a_person`

MODULE	OUTPUT PORTS
core (always available)	<one per choice>, default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Pauses and asks someone to decide which way to go, or to fill in what only they know — the run continues down the path they pick.

When to use

- "Accept, counter-offer or send to legal?"
- Collect a value a person must supply (a PO number, a date) before continuing.

How it works

1. Same machinery as Start Approval: a request in **Automation** → **Approvals** (the person is notified at once), a **Waiting** run, deadline and reminders. A question always opens its own page — the answers are required.
2. Each **choice** becomes a port named by its slug ("Send to legal" → `send_to_legal`); the picked one is followed and stored in `{{variables.<name>.decision}}`.
3. Extra answers land in `{{variables.<name>.<field name>}}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
What to ask `question`	Multi-line text	Yes	—	e.g. `e.g. This deal is 40% off list — how should we proceed?`
Ask `approvers`	User picker, many	Yes	—	
When several people are asked `approval_type`	Choice	No	`any`	Options: `any` = Whoever answers first decides `all` = Everyone must answer `sequential` = One after another, in order

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Their choices `choices`	List of names	Yes	—	Each one gets its own path out of this step, and lands in {{variables.<name>.decision}} e.g. `Accept, Counter-offer, Send to legal`
Also ask them for `fields`	Question rows	No	—	Answers arrive as {{variables.<name>.<field name>}}
Save their answer as `output_variable`	Text	No	`answer`	
Answer within (hours) `due_hours`	Number	No	—	
Count working hours only `business_hours`	Yes / no	No	—	A 4-hour deadline set at 4pm falls due at 11am tomorrow, not at 8pm tonight Shown when due_hours is filled in
Remind them after (hours) `remind_after_hours`	Number	No	—	
If nobody answers in time `on_timeout`	Choice	No	`keep_waiting`	Options: `keep_waiting` = Keep waiting (do nothing) `escalate` = Hand it to someone else Shown when due_hours is filled in
Hand it to `escalate_to`	User picker	No	—	Shown when on_timeout is escalate
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **What to ask** `question`
- **Ask** `approvers`

- **Their choices** `choices`

Optional fields

- **When several people are asked** `approval_type` — default `any`
- **Also ask them for** `fields`
- **Save their answer as** `output_variable` — default `answer`
- **Answer within (hours)** `due_hours`
- **Count working hours only** `business_hours`
- **Remind them after (hours)** `remind_after_hours`
- **If nobody answers in time** `on_timeout` — default `keep_waiting`
- **Hand it to** `escalate_to`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.answer}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

KEY	MEANING	TYPE
`decision`	The choice they picked	text

Execution behaviour

Resolves its settings, performs the change through the owning module's own service where one exists (so that module's rules, audit trail and events apply), and reports what it did. In a **Test run** the step is simulated and returns `would_have` with the resolved settings (under the step's `output`) instead of acting.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`<one per choice>`	One port per choice; named by the choice's slug.
`default`	Fallback when no choices are configured.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **What to ask** must be filled in.
- **Ask** must be filled in.
- **Their choices** must be filled in.
- Question, at least one person and at least one choice are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A rejection from the inbox with no rejected wire fails the run.
- If creating the request fails the run stays Waiting with nothing in the inbox.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

Your own **Answer within** deadline; on timeout it keeps waiting or hands over to someone else. Time spent waiting is not charged to the run's Time limit.

Idempotency

One request per run.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Choose how to proceed

```
{
  "question": "This deal is 40% off list — how should we proceed?",
  "approvers": [
    "<user id>"
  ],
  "choices": [
    "Accept",
    "Counter-offer",
    "Send to legal"
  ],
  "fields": [
    {
      "key": "counter_pct",
      "label": "Counter-offer (%)"
    }
  ],
  "output_variable": "answer"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "question": "e.g. This deal is 40% off list — how should we proceed?",
  "approvers": [
    "<user id>"
  ],
  "approval_type": "any",
  "choices": [],
  "fields": [],
  "output_variable": "answer",
  "due_hours": 0,
  "business_hours": false,
  "remind_after_hours": 0,
  "on_timeout": "keep_waiting",
  "escalate_to": "<user id>",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ In a Test run nobody answers, so the step takes its **first** choice and only that path runs.

✓ Save the result as a variable when a later step needs the created id — `{{variables.<name>.id}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

ACTIONS · Integration · integration_hub

Add a row to a spreadsheet

Append one row to an Excel table or a Google Sheets tab

NODE TYPE	NODE KEY
Integration (action)	`integration.excel_append_row`

MODULE	OUTPUT PORTS
integration_hub (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Appends one row to an Excel table (OneDrive / SharePoint) or a Google Sheets tab, keyed by the header row.

When to use

- Mirror won deals, new tickets or survey scores into a sheet the team already lives in.

How it works

1. Picks the provider and location, resolves **Row values** (column header → value), and calls the hub tool `graph.excel.append_row` or `google.sheets.append_row`. An Excel **named table** grows its formulas and formatting with the row.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Spreadsheet is in `provider`	Choice	No	`excel`	Options: `excel` = Excel (OneDrive / SharePoint) `google_sheets` = Google Sheets
Where `location`	Choice	No	`onedrive`	Options: `onedrive` = OneDrive `sharepoint` = A SharePoint site Shown when provider is (blank) / excel
SharePoint site id `site_id`	Text	No	—	Find it with the Execute Tool step's 'Find a SharePoint site' Shown when location is sharepoint
File `path`	Value or variable	No	—	A path in the drive, or paste the file's share link (then no site id is needed)

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Shown when provider is (blank) / excel e.g. `Reports/Pipeline.xlsx`
Spreadsheet `spreadsheet_id`	Value or variable	No	—	Shown when provider is google_sheets e.g. `Its id, or paste the link`
Tab `tab`	Value or variable	No	—	Shown when provider is google_sheets e.g. `Sheet1`
Table `table`	Text	No	—	An Excel NAMED table (Insert → Table) — it grows its formulas and formatting with the new row Shown when provider is (blank) / excel
Row values `values`	Rows of key / value	No	—	Keyed by the table's header row; { } Insert works in a value
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Spreadsheet is in** `provider` — default `excel`
- **Where** `location` — default `onedrive`
- **SharePoint site id** `site_id`
- **File** `path`
- **Spreadsheet** `spreadsheet_id`

- **Tab** `tab`
- **Table** `table`
- **Row values** `values`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`added`	Rows added	number
`table`	Table	text

Execution behaviour

Goes through the Integration Hub's execution gateway: policy check, stored credentials, the adapter, and an audit entry. A tool that requires approval parks the run as **Waiting** until someone decides. With `automation/external_jobs` on, the call runs on the tool job worker (see the Overview) and the run waits for it; the step log then shows the job id.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A tool that needs approval parks the run as Waiting and files a request in the hub's approvals inbox; approving it re-runs the step, rejecting it counts as a denial. A policy denial (the tool is not allowed for this workspace or connection, or the approval was rejected) fails the step, or is skipped with the reason where the step offers "Fail flow if approval is denied" = off. Any other failure takes On error / Stop on error like every step.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **integration_hub** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- File (Excel) or Spreadsheet + Tab (Google), and at least one row value, are required; a SharePoint site id when Where = SharePoint.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "no file — give the workbook's path" / "give the spreadsheet and the tab" / "no values to put in the row" / "no SharePoint site id — …" → skipped.
- A denied or failed tool call → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent — a duplicate row on every re-run. A Test run simulates this step (`would_have`) and appends nothing.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Won deals log

```
{
  "provider": "excel",
  "location": "onedrive",
  "path": "Reports/Won deals.xlsx",
  "table": "WonDeals",
  "values": [
    {
      "key": "Deal",
      "value": "{{record.name}}"
    },
    {
      "key": "Amount",
      "value": "{{record.amount}}"
    },
    {
      "key": "Closed",
```

```
      "value": "{{trigger.to_stage}}"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "provider": "excel",
  "location": "onedrive",
  "site_id": "<SharePoint site id>",
  "path": "Reports/Pipeline.xlsx",
  "spreadsheet_id": "Its id, or paste the link",
  "tab": "Sheet1",
  "table": "<Table>",
  "values": [],
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Execute Tool and Read rows run **for real in a Test run**; Add a row and Upload a file are simulated. Point real ones at a test sheet or folder while building.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Credentials stay in the Integration Hub; the step never sees them. The tool's input and output are stored in the step log.

Related nodes

ACTIONS · Integration · integration_hub

Read rows from a spreadsheet

Read rows from an Excel table or range, or a Google Sheets tab — the first row is the header

NODE TYPE	NODE KEY
Integration (action)	`integration.excel_read_rows`

MODULE	OUTPUT PORTS
integration_hub (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Reads rows from an Excel table or range, or a Google Sheets tab — the first row is the header — into a list variable.

When to use

- A price list, a territory map or an allow-list kept in a sheet.

How it works

1. Calls `graph.excel.read_rows` / `google.sheets.read_rows` with table / sheet / range and **At most (rows)** (default 100); saves `{rows, headers, count, capped}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Spreadsheet is in `provider`	Choice	No	`excel`	Options: `excel` = Excel (OneDrive / SharePoint) `google_sheets` = Google Sheets
Where `location`	Choice	No	`onedrive`	Options: `onedrive` = OneDrive `sharepoint` = A SharePoint site Shown when provider is (blank) / excel
SharePoint site id `site_id`	Text	No	—	Find it with the Execute Tool step's 'Find a SharePoint site' Shown when location is sharepoint
File `path`	Value or variable	No	—	A path in the drive, or paste the file's share link (then no site id is needed) Shown when provider is (blank) / excel e.g. `Reports/Pipeline.xlsx`

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Spreadsheet `spreadsheet_id`	Value or variable	No	—	Shown when provider is google_sheets e.g. `Its id, or paste the link`
Tab `tab`	Value or variable	No	—	Shown when provider is google_sheets e.g. `Sheet1`
Table `table`	Text	No	—	Leave blank to read a sheet range instead Shown when provider is (blank) / excel
Sheet `sheet`	Text	No	—	When reading a range Shown when provider is (blank) / excel
Range `range`	Text	No	—	Leave blank for everything the sheet uses e.g. `A1:D50`
At most (rows) `limit`	Number	No	`100`	
Save the rows as `output_variable`	Text	No	`rows`	Read one as {{variables.rows.rows.0.Amount}}
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Spreadsheet is in** `provider` — default `excel`
- **Where** `location` — default `onedrive`
- **SharePoint site id** `site_id`
- **File** `path`
- **Spreadsheet** `spreadsheet_id`
- **Tab** `tab`

- **Table** `table`
- **Sheet** `sheet`
- **Range** `range`
- **At most (rows)** `limit` — default `100`
- **Save the rows as** `output_variable` — default `rows`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.rows}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`rows`	The rows	list
`headers`	Column headers	list
`count`	How many	number
`capped`	More were left	boolean

Execution behaviour

Goes through the Integration Hub's execution gateway: policy check, stored credentials, the adapter, and an audit entry. A tool that requires approval parks the run as **Waiting** until someone decides. With `automation/external_jobs` on, the call runs on the tool job worker (see the Overview) and the run waits for it; the step log then shows the job id.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A tool that needs approval parks the run as Waiting and files a request in the hub's approvals inbox; approving it re-runs the step, rejecting it counts as a denial. A policy denial (the tool is not allowed for this workspace or connection, or the approval was rejected) fails the step, or is skipped with the reason where the step offers "Fail flow if approval is denied" = off. Any other failure takes On error / Stop on error like every step.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **integration_hub** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- File (Excel) or Spreadsheet + Tab (Google) are required.

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Territory lookup

Then Loop over `{{variables.territories.rows}}`.

```
{
  "provider": "google_sheets",
  "spreadsheet_id": "<spreadsheet id or link>",
  "tab": "Territories",
  "limit": 500,
```

```
  "output_variable": "territories"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "provider": "excel",
  "location": "onedrive",
  "site_id": "<SharePoint site id>",
  "path": "Reports/Pipeline.xlsx",
  "spreadsheet_id": "Its id, or paste the link",
  "tab": "Sheet1",
  "table": "<Table>",
  "sheet": "<Sheet>",
  "range": "A1:D50",
  "limit": 100,
  "output_variable": "rows",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Execute Tool and Read rows run **for real in a Test run**; Add a row and Upload a file are simulated. Point real ones at a test sheet or folder while building.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Credentials stay in the Integration Hub; the step never sees them. The tool's input and output are stored in the step log.

Related nodes

ACTIONS · Integration · integration_hub

Upload a file to OneDrive / SharePoint

Put a file in a OneDrive folder or a SharePoint document library

NODE TYPE	NODE KEY
Integration (action)	`integration.upload_file`

MODULE	OUTPUT PORTS
integration_hub (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Puts a file (text, or base64 for binary) in a OneDrive folder or a SharePoint document library.

When to use

- Drop a generated summary or export next to the account's documents.

How it works

1. Calls `graph.onedrive.upload_file` with the path and content.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Where `location`	Choice	No	`onedrive`	Options: `onedrive` = OneDrive `sharepoint` = A SharePoint site
SharePoint site id `site_id`	Text	No	—	Find it with the Execute Tool step's 'Find a SharePoint site' Shown when location is sharepoint
File `path`	Value or variable	Yes	—	e.g. `Reports/Pipeline.xlsx`
File content `content`	Multi-line text	Yes	—	Text, or base64 for a binary file. { } Insert works here.
Save the result as `output_variable`	Text	No	—	Optional. Use it later as {{variables.<name>}} — a created record's id is {{variables.<name>.id}} e.g. `new_contact`
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **File** `path`
- **File content** `content`

Optional fields

- **Where** `location` — default `onedrive`
- **SharePoint site id** `site_id`
- **Save the result as** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`name`	File name	text
`web_url`	Link to the file	text

Execution behaviour

Goes through the Integration Hub's execution gateway: policy check, stored credentials, the adapter, and an audit entry. A tool that requires approval parks the run as **Waiting** until someone decides. With `automation/external_jobs` on, the call runs on the tool job worker (see the Overview) and the run waits for it; the step log then shows the job id.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A tool that needs approval parks the run as Waiting and files a request in the hub's approvals inbox; approving it re-runs the step, rejecting it counts as a denial. A policy denial (the tool is not allowed for this workspace or connection, or the approval was rejected) fails the step, or is skipped with the reason where the step offers "Fail flow if approval is denied" = off. Any other failure takes On error / Stop on error like every step.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **integration_hub** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **File** must be filled in.
- **File content** must be filled in.
- File path and content are required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "that step does not work with Google Sheets" (Google provider) → skipped.
- A denied or failed tool call → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Re-running uploads again (overwriting the same path). A Test run simulates this step and uploads nothing.

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are

never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "location": "onedrive",
  "site_id": "<SharePoint site id>",
  "path": "Reports/Pipeline.xlsx",
  "content": "<File content>",
  "output_variable": "new_contact",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Execute Tool and Read rows run **for real in a Test run**; Add a row and Upload a file are simulated. Point real ones at a test sheet or folder while building.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Credentials stay in the Integration Hub; the step never sees them. The tool's input and output are stored in the step log.

Related nodes

ACTIONS · Integration · integration_hub

Execute Tool

Call any registered tool via the Integration Hub — DigiCRM internal actions, MCP-registered external tools, or ACI/DLT adapters. Tools marked 'requires approval' pause the execution and wait for a human decision.

NODE TYPE	NODE KEY
Integration (action)	`integration.execute_tool`

MODULE	OUTPUT PORTS
integration_hub (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Calls any tool registered in the Integration Hub — DigiCRM internal actions, MCP-registered external tools, ACI / DLT adapters — with mapped inputs.

When to use

- Anything the friendlier integration steps do not cover: create a calendar event in Microsoft 365, post to a chat channel, call a partner tool.

How it works

1. Resolves **Tool inputs**, calls the execution gateway (policy, credentials, adapter, audit).
2. A tool marked "requires approval" parks the run as **Waiting** and files a tool approval linked to this run and step — decided from **Automation** → **Approvals** by anyone holding the Integrations permission (the workflow's owner is notified). **Approving** it re-runs the step with the approval granted and the run carries on; **rejecting** it takes the same path as a policy denial below.
3. A **policy** denial (the tool is not allowed here, or the approval was rejected) fails the step with "Tool call denied: …", or is skipped with the reason when **Fail flow if approval is denied** is off.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Tool `tool_key`	Record picker (integration_tool)	Yes	—	
Tool inputs (variable refs OK) `input_mapping`	Rows of key / value	No	—	
Connection (optional) `connection_id`	Record picker (integration_connection)	No	—	
Store result in variable `output_variable`	Text	No	—	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Fail flow if approval is denied `fail_on_denied`	Yes / no	No	`true`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Tool** `tool_key`

Optional fields

- **Tool inputs (variable refs OK)** `input_mapping`
- **Connection (optional)** `connection_id`
- **Store result in variable** `output_variable`
- **Fail flow if approval is denied** `fail_on_denied` — default `true`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

Execution behaviour

Goes through the Integration Hub's execution gateway: policy check, stored credentials, the adapter, and an audit entry. A tool that requires approval parks the run as **Waiting** until someone decides. With `automation/external_jobs` on, the call runs on the tool job worker (see the Overview) and the run waits for it; the step log then shows the job id.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A tool that needs approval parks the run as Waiting and files a request in the hub's approvals inbox; approving it re-runs the step, rejecting it counts as a denial. A policy denial (the tool is not allowed for this workspace or connection, or the approval was rejected) fails the step, or is skipped with the reason where the step offers "Fail flow if approval is denied" = off. Any other failure takes On error / Stop on error like every step.

Branches

PORT	TAKEN WHEN
`default`	The tool call succeeded (immediately, or after a hub approval).

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **integration_hub** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Tool** must be filled in.
- Tool is required ("execute_tool: tool_key is required").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "Tool call denied: …" → failed (or skipped with fail_on_denied off).
- "Tool call failed: <error>" → failed.

Retry behaviour

Not retried automatically — the gateway reports adapter errors as a failed result ("Tool call failed: …"), which the transient-error retry does not cover. Only an exception raised inside the gateway itself (rare) is retried per the workflow's Max retries.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Not idempotent. **Runs for real in a Test run.**

Transaction / rollback

A run is one database transaction, committed when it finishes or pauses. A failed step does **not** undo what earlier steps wrote — their changes are saved together with the failed run. Messages already sent and external calls already made are never undone. If the engine itself crashes mid-run, the whole transaction is rolled back (database writes and step logs alike) and the run is marked Failed with a worker error.

Examples

Post to a Teams channel

```
{
  "tool_key": "graph.teams.post_message",
  "input_mapping": {
    "channel_id": "<channel id>",
    "text": "Deal won: {{record.name}} ({{record.amount}})"
  },
  "output_variable": "post"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "tool_key": "<integration_tool id>",
  "input_mapping": [],
  "connection_id": "<integration_connection id>",
  "output_variable": "<Store result in variable>",
  "fail_on_denied": true,
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Execute Tool and Read rows run **for real in a Test run**; Add a row and Upload a file are simulated. Point real ones at a test sheet or folder while building.

Limitations

- While the run waits for a tool approval it holds its variables and queue in the run record. The approval request expires after 24 hours by default; an hourly sweep then treats it as rejected and the run takes the denial path.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The hub's policy decides which tools this workspace may call and with which connection; credentials never reach the step. Inputs and outputs are audited by the gateway and stored in the step log.

Related nodes

FLOW CONTROL · core

Delay / Wait

Pause execution for a specified duration

NODE TYPE	NODE KEY
Flow control	`flow.delay`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Pauses the run for a duration, until a date/time, or until a date stored on the record — optionally counting working time only.

When to use

- "Wait 2 days, then check whether the ticket is still open."
- Send a reminder 24 hours before the booking (**until_field** on the record's start time, combined with Adjust time).

How it works

1. Computes the resume time: minutes / hours / days from now, a fixed date/time, or a date field of the record. **Count working time only** converts days to working days (or rolls a fixed time to the next working moment) on the workspace's business calendar; ignored where none is set up.
2. **Until we're next open** resumes at the start of the business calendar's next working window — or carries straight on when it is working time now. Put it in front of a customer-facing send so nothing goes out at 3 a.m. or on a holiday.
3. **Which calendar** picks the business calendar for 'Until we're next open' (blank = the workspace default) — e.g. the Retail Banking preset's *Collections contact hours* (RBI 08:00–19:00). A named calendar that is missing or switched off FAILS the step rather than sending at the wrong time.
4. A time already in the past continues at once ("time already passed — no wait"). In a Test run it never waits.
5. Otherwise the run is parked as **Waiting** on a timer row of its own (several branches can sleep at once); its variables and queue are saved. The worker checks due timers every 15 seconds and resumes only this branch, down every non-error wire.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Wait `delay_type`	Choice	No	`minutes`	'Until we're next open' carries straight on during working hours and otherwise waits for the business calendar's next working window — put it before a customer-facing send.

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `minutes` = Minutes `hours` = Hours `days` = Days `until_date` = Until a date/time `until_field` = Until a date on the record `until_open` = Until we're next open
How long `delay_value`	Number	No	—	Shown when delay_type is (blank) / minutes / hours / days
Resume at `until_date`	Date & time	No	—	Shown when delay_type is until_date
Record date field `until_field`	Record field	No	—	Shown when delay_type is until_field
Count working time only `business_hours`	Yes / no	No	—	Two days becomes two WORKING days on the workspace's business calendar, and a fixed date/time rolls forward to the next working moment. Ignored where no calendar is set up. Shown when delay_type is (blank) / minutes / hours / days / until_date / until_field
Which calendar `business_calendar`	Picker (business_calendar)	No	—	e.g. Collections contact hours. If the chosen calendar is missing or switched off the step FAILS instead of sending at the wrong time. Shown when delay_type is until_open e.g. `— Workspace default —`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Wait** `delay_type` — default `minutes`
- **How long** `delay_value`
- **Resume at** `until_date`
- **Record date field** `until_field`
- **Count working time only** `business_hours`
- **Which calendar** `business_calendar`

Inputs

- The clock, and for until_field the record's current value.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

While waiting the run holds no resources; the record can change meanwhile. Resuming re-reads variables from the saved state, not the record — use a condition after the delay to re-check.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

Only configuration errors fail it (below). If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- How long is required for minutes / hours / days; Resume at for until_date; Record date field for until_field.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "Delay needs a number of minutes / hours / days" / "Delay can't be negative" / "Delay 'until field' has no field selected" → failed.
- "The record's 'x' is empty — nothing to wait for" / "'x' is not a date/time" → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No maximum delay, and the time spent waiting is not charged to the run's **Time limit** — only active steps count.

Idempotency

Repeating the step waits again.

Transaction / rollback

Writes nothing.

Examples

Two working days

```
{
  "delay_type": "days",
  "delay_value": 2,
  "business_hours": true
}
```

Hold the reply until we are open

```
{
  "delay_type": "until_open"
}
```

Until the renewal date on the record

```
{
  "delay_type": "until_field",
  "until_field": "renewal_date"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "delay_type": "minutes",
  "delay_value": 0,
  "until_date": "<Resume at>",
  "until_field": "<Record date field>",
  "business_hours": false,
  "business_calendar": "— Workspace default —"
}
```

Best practices

✓ Naive dates are treated as UTC.

✓ After a long delay re-check the record with a condition — the world moved on.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Loop / For Each

Iterate over a list of items

NODE TYPE	NODE KEY
Flow control	`flow.loop`

MODULE	OUTPUT PORTS
core (always available)	loop_body, done

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Runs the steps on its **loop_body** port once for each item of a list, then continues on **done**.

When to use

- Act on each record a Find Records step returned in **list** mode; process each line item from an API response.

How it works

1. **Collection to iterate** names a variable holding a list (anything else counts as an empty list).
2. For each item (up to **Run at most**, default 100; extra items are dropped silently) the body runs in its own scope with `{{variables.<item name>}}` (default `loop_item`) and `{{variables.loop_index}}`; steps inside run depth-first.
3. `done` runs once after every item.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Collection to iterate `collection`	Variable reference	Yes	—	
Call each item `item_variable`	Text	No	`loop_item`	Read it in the body as {{variables.loop_item}}
Run at most `max_iterations`	Number	No	`100`	A guard — extra items are skipped

Required fields

- **Collection to iterate** `collection`

Optional fields

- **Call each item** `item_variable` — default `loop_item`
- **Run at most** `max_iterations` — default `100`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Runs once when reached (once per item inside a loop). Its settings are resolved just before it runs, so it sees everything earlier steps saved.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`loop_body`	Once per item, with the item in the variable.
`done`	Once, after all items.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Collection to iterate** must be filled in.
- Collection is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

The 2000-step run cap and the Time limit apply across all iterations.

Idempotency

Repeating the loop repeats every body step per item.

Transaction / rollback

Writes nothing.

Examples

Email every found contact

Inside the body: Send Email with Send to contact = `{{variables.contact.id}}`.

```
{
  "collection": "{{variables.found}}",
  "item_variable": "contact",
  "max_iterations": 200
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "collection": "{{variables.something}}",
  "item_variable": "loop_item",
  "max_iterations": 100
}
```

Best practices

- ✓ Find Records in **loop** mode does this without a separate Loop step.
- ✓ Keep bodies short — a 200-item loop with 10 steps is 2000 steps, the run cap.

Limitations

- Sequential, not parallel.
- Max iterations default 100; extra items are skipped silently.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Error Handler

Marks where a failed step carries on — wire a step's "On error" port to it

NODE TYPE	NODE KEY
Flow control	`flow.error_handler`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

A named landing point for a failed step — wire a step's **On error** port to it, then continue with what should happen (tell someone, create a task, end).

When to use

- After an HTTP call, a tool call or a send that may fail, when the run should carry on rather than stop.

How it works

1. It does nothing itself: the routing is the failing step's On error port; this step passes straight through with `{{nodes.<failed step>.error}}` available to what follows. **What this path handles** is a label for the canvas.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
What this path handles `note`	Text	No	—	e.g. `e.g. the CRM write failed — tell the owner`

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **What this path handles** `note`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).

- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Pass-through. A failed step whose On error port is wired anywhere (not necessarily here) is caught; this step only makes the intent visible.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

Cannot fail.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.

VARIABLE	WHAT IT IS	AVAILABLE
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Writes nothing to records; repeating it only repeats the control effect (a second delay, another loop pass).

Transaction / rollback

Writes nothing.

Examples

Tell the owner the CRM write failed

Wire Update Field → On error → Error Handler → Send Notification with message `{{nodes.update_field.error}}`.

```
{
  "note": "the CRM write failed — tell the owner"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "note": "e.g. the CRM write failed — tell the owner"
}
```

Best practices

- ✓ There is no global handler: each step that may fail needs its own On error wire.
- ✓ Retries are the workflow setting Max retries, not something this step controls.

Limitations

- Catches nothing by itself.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

End

TERMINAL

Terminate the workflow

NODE TYPE	NODE KEY
Flow control	`flow.end`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Marks the run as completed on this path.

When to use

- To make an early finish explicit on the canvas.

How it works

1. Sets the run status to Completed and follows nothing after it.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Finish as `status`	Choice	No	`success`	Options: `success` = Done `error` = Failed `cancelled` = Cancelled

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Finish as** `status` — default `success`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Runs once when reached (once per item inside a loop). Its settings are resolved just before it runs, so it sees everything earlier steps saved.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.

VARIABLE	WHAT IT IS	AVAILABLE
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Nothing can connect out of it.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Writes nothing to records; repeating it only repeats the control effect (a second delay, another loop pass).

Transaction / rollback

Writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "status": "success"
}
```

Best practices

Limitations

- **Finish as** (success / error / cancelled) is not read by the engine — the run is marked completed regardless.
- Branches already queued (after a Fork, say) still run, and a later failure can still turn the run Failed.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Not again so soon

Stop here if this step already ran for this record recently — the guard against the same person being emailed five times in an hour

NODE TYPE	NODE KEY
Flow control	`flow.once_per_record`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Stops this path if the same step already ran for this record recently — the guard against emailing the same person five times in an hour.

When to use

- Before any send or create step whose trigger can fire repeatedly for the same record (updated triggers, score changes, scheduled scans).

How it works

1. Counts successful passes of **this step** (by step id) for **this record** within **Only once every (hours)** (default 24), across all of the workspace's runs. Already ran → `passed: false` and the branch stops; otherwise `passed: true` and the run continues.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Only once every (hours) `within_hours`	Number	Yes	`24`	Counts how often THIS step passed for THIS record

Required fields

- **Only once every (hours)** `within_hours`

Optional fields

None.

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`passed`	True when the run carried on	boolean
`already_ran`	How many times it passed recently	number

Execution behaviour

A blocked pass is logged as **skipped** (`passed: false` , reason "already ran Nx for this record in the last H h") so it never counts as an error — nor as a pass, so the window runs from the last real pass, not from the last attempt.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Only once every (hours)** must be filled in.
- Only once every (hours) is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

This is the idempotency guard — it makes the steps after it once-per-window.

Transaction / rollback

Writes nothing.

Examples

At most one nudge a week

```
{
  "within_hours": 168
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "within_hours": 24
}
```

Best practices

Limitations

- Counts this workflow's own real passes for the record (a copied workflow has its own window; Test runs never count).
- Runs with no record are never blocked.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Go To

Jump to a specific node

NODE TYPE	NODE KEY
Flow control	`flow.goto`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Jumps to another step on the canvas.

When to use

- To rejoin a shared tail from several branches without duplicate wires.

How it works

1. Queues the target step; the Go To step's own outgoing wires are ignored — only the target runs. A step that already ran in this scope is not run again, so Go To **cannot** make a loop — use Loop / For Each for that.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Jump to `target_node_id`	Step picker	Yes	—	Each step runs once per run, so a step already done is not run again

Required fields

- **Jump to** `target_node_id`

Optional fields

None.

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).

- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Runs once when reached (once per item inside a loop). Its settings are resolved just before it runs, so it sees everything earlier steps saved.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Jump to** must be filled in.
- Jump to is required; blank ends the branch.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Writes nothing to records; repeating it only repeats the control effect (a second delay, another loop pass).

Transaction / rollback

Writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "target_node_id": "<Jump to>"
}
```

Best practices

Limitations

- Cannot loop back to an already-run step.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Run Sub-Workflow

Execute another workflow as a sub-flow

NODE TYPE	NODE KEY
Flow control	`flow.sub_workflow`
MODULE	**OUTPUT PORTS**
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Runs another workflow as a child, on the same record, optionally waiting for it and reading its variables back.

When to use

- Reuse a shared "notify the account team" flow from many workflows.
- Split a long process into stages.

How it works

1. **Inputs** rows pass values into the child as `{{trigger.<name>}}`.
2. With **Wait for it to finish** on, the child runs right away (as a test run if the parent is one); its variables come back as `{{variables.<output variable>}}` (default `sub_result`). If the child pauses, the parent waits until it finishes.
3. With it off, the child is queued and the parent continues immediately with only the child's execution id — except in a Test run, where the child runs at once as a test run and its variables come back as with Wait on.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Workflow `workflow_id`	Record picker (workflow)	Yes	—	
Inputs (value → child trigger field) `input_mapping`	Field → value rows	No	—	
Wait for it to finish `wait_for_completion`	Yes / no	No	`true`	
Save its variables as `output_variable`	Text	No	`sub_result`	Shown when wait_for_completion is (blank) / true / true
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Workflow** `workflow_id`

Optional fields

- **Inputs (value → child trigger field)** `input_mapping`
- **Wait for it to finish** `wait_for_completion` — default `true`
- **Save its variables as** `output_variable` — default `sub_result`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.sub_result}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

Execution behaviour

Runs once when reached (once per item inside a loop). Its settings are resolved just before it runs, so it sees everything earlier steps saved.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A failed child fails this step; On error applies — except when the parent was woken after the child had paused, where only Stop on error decides.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user

for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Workflow** must be filled in.
- Workflow is required; a workflow cannot call itself directly.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "Run Sub-Workflow has no workflow selected" / "A workflow can't run itself as a sub-workflow" / "The selected sub-workflow no longer exists" / "Sub-workflows nested more than 5 deep — stopped" → failed.
- Child failed → the step fails with "Sub-workflow 'X' failed: …".

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

The parent's Time limit keeps counting while the child runs.

Idempotency

Re-running runs the child again (creating whatever it creates).

Transaction / rollback

Writes nothing.

Examples

Shared notification flow

```
{
  "workflow_id": "<workflow id>",
  "input_mapping": [
```

```
    {
      "key": "{{trigger.ticket_number}}",
      "value": "ticket_number"
    }
  ],
  "wait_for_completion": true,
  "output_variable": "notify"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "workflow_id": "<workflow id>",
  "input_mapping": [],
  "wait_for_completion": true,
  "output_variable": "sub_result",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

Limitations

- Nesting stops at depth 5.
- The child runs on the same record with the same triggering user.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Wait Until

Pause until a condition on the record is true (or give up)

NODE TYPE	NODE KEY
Flow control	`flow.wait_until`

MODULE	OUTPUT PORTS
core (always available)	met, timeout

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Parks the run until a condition on a record becomes true — or gives up after a limit — re-reading the record on each check.

When to use

- "Wait until the invoice is paid, then..."
- "If the ticket is still open after 3 days, escalate" (the timeout port).

How it works

1. Evaluates the rows against the watched record (the triggering record, or a found one). Met → `met` at once. No rows → met at once.
2. Otherwise the run waits **Check every (minutes)** (default 60 when blank or 0; a value under 1 is treated as 1) and re-reads the record; each check is a resume of the run.
3. When **Give up after (hours)** (default 72) passes without a match → `timeout`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Watch `target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `found_record` = A record found earlier
Record type `target_entity_type`	Choice	No	—	Options: `contact` `account` `opportunity` `campaign` `task` `ticket` Shown when target is found_record
Record id `target_record_id`	Value or variable	No	—	Shown when target is found_record

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				e.g. `{{variables.found.id}}`
Wait until `conditions`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions
Check every (minutes) `check_every_minutes`	Number	No	`60`	
Give up after (hours) `give_up_after_hours`	Number	No	`72`	

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Watch** `target` — default `trigger_record`
- **Record type** `target_entity_type`
- **Record id** `target_record_id`
- **Wait until** `conditions`
- **Match** `logic` — default `and`
- **Check every (minutes)** `check_every_minutes` — default `60`
- **Give up after (hours)** `give_up_after_hours` — default `72`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Only the named port is followed: an unwired met or timeout ends the branch.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`met`	The condition became true on a check.
`timeout`	The give-up limit passed first.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A bad found-record target fails the step.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

Its own **Give up after** limit. Time spent waiting is not charged to the run's Time limit.

Idempotency

Repeating waits again.

Transaction / rollback

Writes nothing.

Examples

Escalate if still open after 3 days

Wire `timeout` to Escalate Ticket.

```
{
  "target": "trigger_record",
  "conditions": [
    {
```

```
      "field": "status",
      "operator": "in",
      "value": "resolved,closed"
    }
  ],
  "check_every_minutes": 120,
  "give_up_after_hours": 72
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "target": "trigger_record",
  "target_entity_type": "contact",
  "target_record_id": "{{variables.found.id}}",
  "conditions": [],
  "logic": "and",
  "check_every_minutes": 60,
  "give_up_after_hours": 72
}
```

Best practices

✓ In a Test run the condition is not checked: the step takes its met path and says so.

Limitations

- Polls; the reaction is up to Check every late.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Wait for Event

Pause until something happens to the record (or anywhere), then continue with that event

NODE TYPE	NODE KEY
Flow control	`flow.wait_for_event`

MODULE	OUTPUT PORTS
core (always available)	default, timeout

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Pauses the run until a chosen event happens — to this record or to any — then continues with that event in hand; optionally gives up after a while.

When to use

- "Send the quote, then wait until the deal moves stage before chasing."
- "Create the ticket and wait for the customer to reply (a new message) — or escalate after 2 days."

How it works

1. Registers an event wait for the chosen trigger (**Wait for**) and, with **Which record** = this record, for the record the run is about; the run is parked as **Waiting** (other branches keep running).
2. When the platform dispatches a matching event, the run continues down the normal wires with the event saved under **Save the event as**: `{{variables.<name>.event_type}}`, `.entity_id`, `.data.<field>` (the event payload).
3. With **Give up after (hours)** set, a timeout takes the `timeout` port instead; the step is then marked skipped and nothing is saved.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Wait for `event_key`	Choice	Yes	—	Options: `trigger.calendar.booking_cancelled` = Booking Cancelled `trigger.calendar.booking_created` = Booking Created `trigger.calendar.booking_no_show` = Booking No-Show `trigger.calendar.booking_rescheduled` = Booking Rescheduled `trigger.campaigns.campaign_cancelled` = Campaign Cancelled

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				`trigger.campaigns.campaign_completed` = Campaign Completed `trigger.campaigns.campaign_scheduled` = Campaign Scheduled `trigger.campaigns.campaign_started` = Campaign Started `trigger.campaigns.campaign_updated` = Campaign Updated `trigger.campaigns.goal_achieved` = Campaign Goal Achieved `trigger.campaigns.goal_at_risk` = Campaign Goal At Risk `trigger.campaigns.goal_missed` = Campaign Goal Missed `trigger.chat.csat_low` = Low Chat CSAT `trigger.chat.handoff` = Chat Handed to Human `trigger.core.manual` = Manual Trigger `trigger.core.record_created` = Record Created (Any) `trigger.core.record_updated` = Record Updated (Any) `trigger.core.scheduled` = Scheduled `trigger.core.threshold` = When a number crosses a line `trigger.core.webhook` = Webhook Received `trigger.crm.account_created` = Account Created `trigger.crm.account_updated` = Account Updated `trigger.crm.contact_created` = Contact Created `trigger.crm.contact_updated` = Contact Updated `trigger.crm.lead_band_changed` = Lead Band Changed `trigger.crm.lead_disqualified` = Lead Disqualified `trigger.crm.lead_qualified` = Lead Qualified `trigger.crm.lead_score_changed` = Lead Score Changed `trigger.crm.opportunity_assigned` = Opportunity Assigned `trigger.crm.opportunity_created` = Opportunity Created `trigger.crm.opportunity_deleted` = Opportunity Deleted `trigger.crm.opportunity_lost` = Opportunity Lost `trigger.crm.opportunity_stage_changed` = Opportunity Stage Changed `trigger.crm.opportunity_updated` = Opportunity Updated `trigger.crm.opportunity_won` = Opportunity Won `trigger.custom.record_created` = Custom Record Created `trigger.custom.record_deleted` = Custom Record Deleted `trigger.custom.record_updated` = Custom Record Updated `trigger.events.person_captured` = Person Met at an Event `trigger.events.person_checked_in` = Checked In to an Event

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				`trigger.goals.at_risk` = Goal At Risk `trigger.goals.completed` = Goal Completed `trigger.goals.goal_progress_updated` = Goal Progress Updated `trigger.goals.goal_target_reached` = Goal Target Reached `trigger.goals.kpi_threshold_breached` = KPI Threshold Breached `trigger.goals.kr_target_reached` = Key Result Target Reached `trigger.goals.kr_value_updated` = Key Result Updated `trigger.leads.visitor_identified` = New Website / Chat Lead `trigger.links.link_clicked` = Link Clicked `trigger.messaging.message_delivered` = Message Delivered `trigger.messaging.message_failed` = Message Failed `trigger.omnichannel.contact_global_unsubscribed` = Contact Globally Unsubscribed `trigger.omnichannel.unsubscription_submitted` = Unsubscription Submitted `trigger.projects.sprint_completed` = Sprint Completed `trigger.projects.work_item_completed` = Work Item Completed `trigger.projects.work_item_created` = Work Item Created `trigger.projects.work_item_status_changed` = Work Item Status Changed `trigger.service_desk.sla_breached` = SLA Breached `trigger.service_desk.ticket_created` = Ticket Created `trigger.service_desk.ticket_status_changed` = Ticket Status Changed `trigger.surveys.nps_detractor` = NPS Detractor Detected `trigger.surveys.response_received` = Survey Response Received `trigger.surveys.response_started` = Survey Response Started `trigger.surveys.survey_archived` = Survey Archived `trigger.surveys.survey_published` = Survey Published `trigger.tasks.task_assigned` = Task Assigned `trigger.tasks.task_completed` = Task Completed `trigger.tasks.task_created` = Task Created `trigger.tasks.task_overdue` = Task Overdue `trigger.tasks.task_reopened` = Task Reopened `trigger.tasks.task_status_changed` = Task Status Changed `trigger.tasks.task_updated` = Task Updated `trigger.web_analytics.goal_conversion` = Web Goal Conversion `trigger.web_analytics.high_intent_visitor` = High Intent Visitor

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Which record `scope`	Choice	No	`same_record`	Options: `same_record` = The record this run is about `any_record` = Any record
Give up after (hours) `timeout_hours`	Number	No	—	e.g. `blank = wait as long as it takes`
Save the event as `output_variable`	Text	No	`event`	

Required fields

- **Wait for** `event_key`

Optional fields

- **Which record** `scope` — default `same_record`
- **Give up after (hours)** `timeout_hours`
- **Save the event as** `output_variable` — default `event`

Inputs

- The trigger key, the record scope, the optional timeout.

Outputs

The result is saved as `{{variables.event}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`event_type`	Event	text
`entity_type`	Record type	text
`entity_id`	Record id	id
`data`	Event data	text

The event (type, record, payload) under the output name.

Execution behaviour

Any record = the first matching event in the workspace wakes it. Only triggers something dispatches can be waited on (the same list the palette offers).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The event arrived.
`timeout`	Nobody did it in time (only with Give up after set).

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Wait for** must be filled in.
- Wait for is required ("wait_for_event: event is required").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A missing event key fails the step at once; nothing else can fail while waiting.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

Only your own **Give up after**. Time spent waiting is not charged to the run's Time limit.

Idempotency

One wait per run and step; a second visit (in a loop) registers its own.

Transaction / rollback

Cancelling the run leaves the wait row behind, inert — a later event finds the run no longer waiting and does nothing.

Examples

Chase a quote once the deal moves

```
{
  "event_key": "trigger.crm.opportunity_stage_changed",
  "scope": "same_record",
```

```
  "timeout_hours": 72,
  "output_variable": "moved"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "event_key": "trigger.calendar.booking_cancelled",
  "scope": "same_record",
  "timeout_hours": 0,
  "output_variable": "event"
}
```

Best practices

✓ Wire the timeout port whenever you set Give up after — an unwired timeout simply ends that branch.

✓ In a Test run the step never waits: it takes the normal wires with an empty event.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Sticky Note TERMINAL

A note on the canvas — never runs

NODE TYPE	NODE KEY
Flow control	`flow.note`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

A sticky note on the canvas to explain the workflow to the next person. Never runs.

When to use

- Whenever a future reader would ask "why?"

How it works

1. Nothing connects to it, so the engine never reaches it.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Note `text`	Multi-line text	No	—	

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Note** `text`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Not executed.

Success path

Not applicable.

Failure path

Not applicable.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Nothing can connect out of it.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not applicable.

Timeout

Not applicable.

Idempotency

Not applicable.

Transaction / rollback

Not applicable.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "text": "<Note>"
}
```

Best practices

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

Not logged.

Security considerations

Text stored with the workflow definition.

Related nodes

FLOW CONTROL · core

Run branches at once

Every step wired to this one runs; put a 'Wait for branches' step downstream to carry on once they have all finished.

NODE TYPE	NODE KEY
Flow control	`flow.fork`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Follows every wire out of it, so several branches run from one point; put a **Wait for branches** step downstream to continue once they are all done.

When to use

- Notify, create a task and update a sheet from one event without chaining them.

How it works

1. Queues every outgoing branch. Inline steps run **one after another** in the queue but independently: with Stop on error off a failing branch does not stop the others; with it on, the run fails once that branch fails — branches queued before it have already run.
2. With job workers on (see *Where steps run*), a branch whose step goes to a worker does not hold the others: their jobs are queued too, so sends, tool calls and AI steps on sibling branches run at the same time and each result resumes only its branch.

Configuration

This node has no settings.

Required fields

None — every setting is optional.

Optional fields

None.

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Runs once when reached (once per item inside a loop). Its settings are resolved just before it runs, so it sees everything earlier steps saved.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	Every outgoing wire is followed.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Writes nothing to records; repeating it only repeats the control effect (a second delay, another loop pass).

Transaction / rollback

Writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

Best practices

Limitations

- Inline steps are sequential under the hood; only worker-run steps overlap.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

FLOW CONTROL · core

Wait for branches

Carries on once the branches wired into it have finished. Each branch's output is kept under this step, keyed by the step it came from.

NODE TYPE	NODE KEY
Flow control	`flow.join`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Continues once the branches wired into it have finished, keeping each branch's output under the step keyed by where it came from.

When to use

- After a Fork, to do something once with the results of all branches.

How it works

1. **Every branch has finished** holds the join while any incoming branch can still arrive — including a branch whose step is out on a job worker — and the run parks as Waiting on those jobs; branches that can no longer arrive (their path ended) are skipped, so it never deadlocks. **The first branch finishes** continues on the first arrival; branches that arrive later still run but are listed under `skipped_branches`, and the join does not run again.
2. Outputs: `{{nodes.<join id>.branches.<source step id>}}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Continue when `merge_strategy`	Choice	No	`wait_all`	Options: `wait_all` = Every branch has finished `wait_any` = The first branch finishes

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Continue when** `merge_strategy` — default `wait_all`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Runs once per scope.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.

VARIABLE	WHAT IT IS	AVAILABLE
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be

retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Writes nothing to records; repeating it only repeats the control effect (a second delay, another loop pass).

Transaction / rollback

Writes nothing.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "merge_strategy": "wait_all"
}
```

Best practices

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

No data leaves the engine.

Related nodes

DATA & UTILITY · core

Set Variable

Set a workflow variable for use in downstream nodes

NODE TYPE	NODE KEY
Data & utility	`data.set_variable`
MODULE	**OUTPUT PORTS**
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Stores a value under a name for later steps.

When to use

- Compose a greeting once and reuse it; pin a cutoff date; build a JSON payload.

How it works

1. Resolves **Value**, converts it per **value_type** (string, number, boolean — true / yes / 1 / on —, json; a value that will not convert stays text) and saves it as `{{variables.<name>}}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Name `variable_name`	Text	Yes	—	Read it later as {{variables.greeting}} e.g. `greeting`
value `value`	Value or variable	No	—	
value type `value_type`	Choice	No	`string`	Options: `string` `number` `boolean` `json` `expression`

Required fields

- **Name** `variable_name`

Optional fields

- **value** `value`

- **value type** `value_type` — default `string`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Name** must be filled in.
- Name is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- Never fails; an unconvertible value is kept as text.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be

retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only or pure computation — safe to repeat. The exceptions (HTTP requests with a body, Find or create) say so on their page.

Transaction / rollback

Writes nothing to records.

Examples

A JSON payload

```
{
  "variable_name": "payload",
  "value": "{\"email\": \"{{record.email}}\", \"plan\": \"{{record.plan}}\"}",
  "value_type": "json"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "variable_name": "greeting",
  "value": "<value>",
  "value_type": "string"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

Data Transform

Change a value: trim, upper/lowercase, split, join, replace, to number…

NODE TYPE	NODE KEY
Data & utility	`data.transform`
MODULE	**OUTPUT PORTS**
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Reshapes a value with a chain of small operations: trim, upper / lower / title case, to number / boolean, JSON parse / stringify, split, join, replace, prefix, suffix, first, last, count.

When to use

- Clean a phone number, split a full name, count a list, build a display string.

How it works

1. Takes **Value to transform**, applies each **Transformations** row in order (operation → optional argument; `split` defaults to ",", `join` to ", ", `replace` takes "old|new"), and saves the result.
2. When **Value to transform** is a single `{{…}}` placeholder, the value itself (a list, an object, a number) is passed to the chain, so `count` / `first` / `json_stringify` work on a Find Records list. Text mixed with other characters is rendered to text first.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Value to transform `input`	Variable reference	No	—	
Transformations `transformations`	Rows of key / value	No	—	uppercase, lowercase, trim, title_case, number, boolean, json_parse, json_stringify, split, join, replace, prefix, suffix, first, last, count
Save the result as `output_variable`	Text	Yes	—	

Required fields

- **Save the result as** `output_variable`

Optional fields

- **Value to transform** `input`
- **Transformations** `transformations`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.transformed}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Save the result as** must be filled in.
- Save the result as is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "'x' is not a transformation (...)" → failed. Every operation tolerates bad arguments, so "<op> failed: ..." is rare in practice.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only or pure computation — safe to repeat. The exceptions (HTTP requests with a body, Find or create) say so on their page.

Transaction / rollback

Writes nothing to records.

Examples

First name from a full name

```
{
  "input": "{{trigger.full_name}}",
  "transformations": [
    {
      "key": "trim"
    },
    {
      "key": "split",
      "value": " "
    },
    {
      "key": "first"
    },
    {
      "key": "title_case"
    }
  ],
  "output_variable": "first_name"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "input": "{{variables.something}}",
```

```
  "transformations": [],
  "output_variable": "<Save the result as>"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

HTTP Request

Make an external API call

NODE TYPE	NODE KEY
Data & utility	`data.http_request`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Calls any HTTPS endpoint and keeps the response — status, body, chosen values, a filtered list — as a variable.

When to use

- Fetch a currency rate, push a record to another system, look something up in a partner API.

How it works

1. Resolves URL, headers and body (JSON text is parsed and sent as JSON; other text is sent as-is; nothing is sent on GET).
2. Streams the response up to **Max response size** (then `truncated`), within **Timeout** (1–60 s). Redirects are not followed.
3. Decides success from **Counts as success** (default 2xx). A non-success status fails the step with "HTTP <code> from <url>" and the first 500 characters of the body — the response is still saved as the variable, so an On error path can read `{{variables.<var>.status}}` / `.body`.
4. Shapes the result: **Keep these values** extracts named paths (with transforms), **List in the response** + **Only list items matching** produce `items` / `item_count`; **Response fields** (learned from a Test call) makes the fields pickable downstream and warns on drift.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
method `method`	Choice	No	`GET`	Options: `GET` `POST` `PUT` `PATCH` `DELETE`
URL `url`	Text	Yes	—	
Headers `headers`	Rows of key / value	No	—	
Body (JSON) `body`	JSON	No	—	Shown when method is POST / PUT / PATCH / DELETE
Save response as variable `output_variable`	Text	No	`http_response`	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Response fields `response_schema`	Response shape	No	—	Test call (or paste a sample) — its fields become pickable in later steps
Counts as success `success_statuses`	Text	No	—	e.g. `2xx – or e.g. 2xx, 404`
Keep these values `extract`	Extract rows	No	—	Name a value from the response; optional transforms clean it up
List in the response `items_path`	Path in response	No	—	e.g. `e.g. $.data.items`
Only list items matching `item_filters`	Filter rows	No	—	Shown when items_path is filled in
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions Shown when items_path is filled in
Timeout (seconds, max 60) `timeout_seconds`	Number	No	`30`	
Max response size (KB, up to 10240) `max_response_kb`	Number	No	`1024`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **URL** `url`

Optional fields

- **method** `method` — default `GET`
- **Headers** `headers`
- **Body (JSON)** `body`

- **Save response as variable** `output_variable` — default `http_response`
- **Response fields** `response_schema`
- **Counts as success** `success_statuses`
- **Keep these values** `extract`
- **List in the response** `items_path`
- **Only list items matching** `item_filters`
- **Match** `logic` — default `and`
- **Timeout (seconds, max 60)** `timeout_seconds` — default `30`
- **Max response size (KB, up to 10240)** `max_response_kb` — default `1024`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The resolved URL, headers and body.

Outputs

The result is saved as `{{variables.http_response}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run. Plus the fields of the Response shape, the "Keep these values" names, and the filtered list.

KEY	MEANING	TYPE
`status`	HTTP status code	number
`ok`	Succeeded (2xx)	boolean
`body`	Response body (parsed JSON when it is JSON)	object
`content_type`	Content type	text
`size_bytes`	Size (bytes)	number
`truncated`	Body was cut off	boolean
`headers`	Response headers (lower-case names)	object

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **URL** must be filled in.
- URL is required ("HTTP Request has no URL").
- An Extract name that is not a valid identifier, or is reserved (status, ok, body, headers, items…), is skipped with a warning in the step log; the step still succeeds.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "URL not allowed: …" — private, loopback or cloud-metadata address → failed.
- "Request failed: …" — connection, DNS, TLS, timeout → failed.
- "HTTP <code> from <url>" → failed.

Retry behaviour

No automatic retries for this step (a failed request is reported, not raised). For retries, paging and rate limits use Call API with a saved endpoint.

Timeout

Per request: **Timeout (seconds)**, default 30, max 60.

Idempotency

GET is safe to repeat; POST / PUT / PATCH / DELETE are sent again on every run or retry.

Transaction / rollback

Writes nothing to records.

Examples

Exchange rate

Later: `{{variables.fx.usd_rate}}`.

```
{
  "method": "GET",
  "url": "https://api.example.com/rates?base={{record.currency}}",
  "timeout_seconds": 15,
  "output_variable": "fx",
  "extract": [
```

```
    {
      "name": "usd_rate",
      "path": "$.rates.USD"
    }
  ]
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "method": "GET",
  "url": "<URL>",
  "headers": [],
  "body": {},
  "output_variable": "http_response",
  "response_schema": {},
  "success_statuses": "2xx – or e.g. 2xx, 404",
  "extract": [],
  "items_path": "e.g. $.data.items",
  "item_filters": [],
  "logic": "and",
  "timeout_seconds": 30,
  "max_response_kb": 1024,
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- HTTPS only; no redirects; body capped (default 1 MB, max 10 MB).
- No retries.
- **Timeout (seconds, max 60)** caps at 60.
- **Max response size (KB, up to 10240)** caps at 10240.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Only HTTPS to public addresses (SSRF guard). Header values you type (API keys) are stored in the workflow and appear resolved in the step log; response bodies are saved into variables — do not fetch secrets you do not need. In a Test run only GET is really sent.

Related nodes

DATA & UTILITY · core

Call API

Call a saved third-party API endpoint (auth, retries and paging handled)

NODE TYPE	NODE KEY
Data & utility	`data.call_api`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Calls a saved third-party endpoint (Automation → API connections, or an imported OpenAPI spec) with auth, retries, rate limiting and paging handled by the connection.

When to use

- Any recurring integration call — prefer this over HTTP Request once an endpoint is set up.

How it works

1. Loads the endpoint (where, how, auth) and sends **Values to send** as its parameters.
2. With **Fetch every page**, follows the endpoint's paging up to **At most this many pages** (default 10) and concatenates the items.
3. Applies the connection's retry policy (default 3 attempts, 1 s exponential backoff, at most 6; honours `Retry-After`) and its per-connection rate limit.
4. Shapes the response exactly as HTTP Request does (success statuses, extract, list, filters).

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Endpoint `endpoint_id`	Record picker (api_endpoint)	Yes	—	Set endpoints up under Automation → API connections (or import an OpenAPI spec)
Values to send `params`	Endpoint parameters	No	—	One box per parameter the endpoint takes — { } Insert works
Fetch every page `all_pages`	Yes / no	No	`false`	For endpoints that page their results — the list holds every page's items

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
At most this many pages `max_pages`	Number	No	`10`	Shown when all_pages is true / true
Save the response as `output_variable`	Text	No	`api_result`	
Response fields `response_schema`	Response shape	No	—	Taken from the endpoint — Test call to refresh
Counts as success `success_statuses`	Text	No	—	e.g. `2xx – or e.g. 2xx, 404`
Keep these values `extract`	Extract rows	No	—	
List in the response `items_path`	Path in response	No	—	e.g. `the endpoint's list, e.g. $.data`
Only list items matching `item_filters`	Filter rows	No	—	Shown when items_path is filled in
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions Shown when items_path is filled in
Timeout (seconds, max 60) `timeout_seconds`	Number	No	`30`	
Max response size per page (KB, up to 10240) `max_response_kb`	Number	No	`1024`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Endpoint** `endpoint_id`

Optional fields

- **Values to send** `params`
- **Fetch every page** `all_pages` — default `false`
- **At most this many pages** `max_pages` — default `10`
- **Save the response as** `output_variable` — default `api_result`
- **Response fields** `response_schema`
- **Counts as success** `success_statuses`
- **Keep these values** `extract`
- **List in the response** `items_path`
- **Only list items matching** `item_filters`
- **Match** `logic` — default `and`
- **Timeout (seconds, max 60)** `timeout_seconds` — default `30`
- **Max response size per page (KB, up to 10240)** `max_response_kb` — default `1024`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.api_result}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run. Plus the fields of the Response shape, the "Keep these values" names, and the filtered list.

KEY	MEANING	TYPE
`status`	HTTP status code	number
`ok`	Succeeded	boolean
`body`	Response body	object
`headers`	Response headers (lower-case names)	object
`pages`	Pages fetched	number
`attempts`	Attempts (retries included)	number

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Endpoint** must be filled in.
- Endpoint is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "the API endpoint is missing (deleted?) — pick it again" → failed.
- "<connection>/<endpoint>: <message>" for auth, transport or status failures → failed.

Retry behaviour

The endpoint's own retry policy (default 3 attempts, exponential backoff from 1 s, capped at 6 attempts; `Retry-After` honoured).

Timeout

Per request: **Timeout (seconds)**, default 30, max 60; paging multiplies it.

Idempotency

Depends on the endpoint's method: reads are safe, writes repeat.

Transaction / rollback

Writes nothing to records.

Examples

All invoices for the account

```
{
  "endpoint_id": "<endpoint id>",
  "params": {
    "customer_id": "{{record.external_id}}"
```

```
  },
  "all_pages": true,
  "max_pages": 5,
  "output_variable": "invoices"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "endpoint_id": "<api_endpoint id>",
  "params": "<Values to send>",
  "all_pages": false,
  "max_pages": 10,
  "output_variable": "api_result",
  "response_schema": {},
  "success_statuses": "2xx – or e.g. 2xx, 404",
  "extract": [],
  "items_path": "the endpoint's list, e.g. $.data",
  "item_filters": [],
  "logic": "and",
  "timeout_seconds": 30,
  "max_response_kb": 1024,
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- **Timeout (seconds, max 60)** caps at 60.
- **Max response size per page (KB, up to 10240)** caps at 10240.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

Credentials live on the connection, not in the step. Responses are stored in variables and the step log. Test runs only send non-mutating (GET) calls.

Related nodes

DATA & UTILITY · core

Find Records

Search records with a query; branch on found / not found

NODE TYPE	NODE KEY
Data & utility	`data.find_records`
MODULE	**OUTPUT PORTS**
core (always available)	found, not_found, done

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Searches records of any type (including your own entities) with filters, and branches on found / not found — the way a run reaches records other than the one it started on.

When to use

- Find the account's primary contact; list open tickets for a customer; pick the records a scheduled run should act on.

How it works

1. Searches **Record type** with **Match records where** rows (ALL / ANY), sorted by **Sort by / direction**, limited to what the Run as user can see. Date operators like "anniversary is today" use the company's time zone.
2. **If more than one record matches**: use the first (default) or last; **fail the run**; **loop** — run the Found path once per record with `{{variables.<name>}}` = that record and `<name>_number`; or **list** — keep all as a list (up to **Max records**, cap 500).
3. Saves `<name>` (record, list or null), `<name>_count` and `<name>_found`; takes `found` or `not_found`, then `done` once afterwards.
4. The step output also records `total`, `returned`, `truncated` (list / loop mode hit Max records) and the first 20 ids.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Picker (searchable_entity)	Yes	—	Your own entities are in this list too
Match records where `conditions`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions
Sort by `sort_by`	Record field	No	—	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Sort direction `sort_dir`	Choice	No	`asc`	Options: `asc` = Ascending `desc` = Descending
If more than one record matches `multiple`	Choice	No	`first`	Options: `first` = Use the first record `last` = Use the last record `error` = Fail the run `loop` = Run the Found path for each record `list` = Keep all records as a list
Max records `max_records`	Number	No	`50`	Shown when multiple is loop / list
Save result as variable `output_variable`	Text	No	`found`	

Required fields

- **Record type** `entity_type`

Optional fields

- **Match records where** `conditions`
- **Match** `logic` — default `and`
- **Sort by** `sort_by`
- **Sort direction** `sort_dir` — default `asc`
- **If more than one record matches** `multiple` — default `first`
- **Max records** `max_records` — default `50`
- **Save result as variable** `output_variable` — default `found`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.found}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. The fields are those of the record type picked in the step.

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`found`	At least one record matched (per record in loop mode).
`not_found`	Nothing matched.
`done`	Once, after the found / not-found path has finished (after every record in loop mode).

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Record type** must be filled in.
- Record type is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A search error (bad field, bad operator) → failed with the message.
- "Find Records matched N <type> records; expected at most one" in **error** mode → failed.
- A Run as user who cannot search → failed ("this workflow has no run-as user — …").

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only — safe to repeat. Runs for real in a Test run.

Transaction / rollback

Writes nothing to records.

Examples

The account's primary contact

Then: Send Email with Send to contact = `{{variables.primary.id}}`.

```
{
  "entity_type": "contact",
  "conditions": [
    {
      "field": "account_id",
      "operator": "eq",
      "value": "{{record.account_id}}"
    },
    {
      "field": "is_primary",
      "operator": "eq",
      "value": true
    }
  ],
  "multiple": "first",
  "output_variable": "primary"
}
```

Every overdue task, one at a time

```
{
  "entity_type": "task",
  "conditions": [
    {
      "field": "due_date",
      "operator": "lt",
      "value": "{{trigger.scheduled_at}}"
    },
    {
      "field": "status",
      "operator": "neq",
      "value": "done"
    }
  ],
  "multiple": "loop",
  "max_records": 200,
  "output_variable": "task"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "<Record type>",
  "conditions": [],
  "logic": "and",
  "sort_by": "<Sort by>",
  "sort_dir": "asc",
  "multiple": "first",
  "max_records": 50,
```

```
  "output_variable": "found"
}
```

Best practices

- ✓ Set Run as on the workflow — the search only sees what that user sees.
- ✓ In loop mode, steps that target "a record found earlier" use `{{variables.<name>.id}}`.
- ✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- At most 500 records per search.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

Aggregate values

Total, average, smallest or largest — over records that match, or over a list the run already has

NODE TYPE	NODE KEY
Data & utility	`data.aggregate`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Counts, sums or averages — over records matching a filter, or over a list the run already holds.

When to use

- Open tickets per customer; total pipeline for an account; the sum of line items from an API response.

How it works

1. **Records**: searches the record type with the filters (Run as scoped) inside a savepoint and applies count / sum / avg / min / max of **Field**.
2. **A list the run already has**: reads **The list** (`{{variables.x}}` or just `x` ; rows may be numbers or records — then **Field** names the value); non-numbers are skipped, count counts every row, a missing or empty list gives 0.
3. Saves the number as the output variable.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Work over `source`	Choice	No	`records`	Options: `records` = Records that match a filter `values` = A list the run already has
The list `values`	Variable reference	No	—	A Find Records result, line items, any list. Rows may be numbers or records — then name the field below. Shown when source is values e.g. `{{variables.found}}`

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Picker (searchable_entity)	Yes	—	Your own entities are in this list too Shown when source is (blank) / records
operation `operation`	Choice	Yes	`count`	Options: `count` = Count records `sum` = Sum of a field `avg` = Average of a field `min` = Lowest value `max` = Highest value
Field `field`	Record field	No	—	On a list of records, the field to read on each row Shown when operation is sum / avg / min / max
Only records where `filters`	Filter rows	No	—	Shown when source is (blank) / records
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions
Save result as variable `output_variable`	Text	No	`aggregate_result`	

Required fields

- **Record type** `entity_type` — only when source is (blank) / records
- **operation** `operation`

Optional fields

- **Work over** `source` — default `records`
- **The list** `values`
- **Field** `field`
- **Only records where** `filters`
- **Match** `logic` — default `and`
- **Save result as variable** `output_variable` — default `aggregate_result`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.aggregate_result}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level	After the step that sets it has run.

VARIABLE	WHAT IT IS	AVAILABLE
	default.	
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Record type** must be filled in when source is (blank) / records.
- **operation** must be filled in.
- Record type is required in records mode; operation defaults to count; Field is needed for sum / avg / min / max.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "Couldn't <op> '<field>' — some values aren't numbers" / a search error → failed.
- "'x' is not one of sum, avg, min, max, count" → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a

wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only.

Transaction / rollback

Writes nothing to records.

Examples

Open tickets for this account

```
{
  "source": "records",
  "entity_type": "ticket",
  "filters": [
    {
      "field": "account_id",
      "operator": "eq",
      "value": "{{record.account_id}}"
    },
    {
      "field": "status",
      "operator": "in",
      "value": "new,open,pending"
    }
  ],
  "operation": "count",
  "output_variable": "open_tickets"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "source": "records",
  "values": "{{variables.something}}",
  "entity_type": "<Record type>",
  "operation": "count",
  "field": "<Field>",
  "filters": [],
  "logic": "and",
  "output_variable": "aggregate_result"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

Adjust time

Take a date/time and shift it — 3 days after the deal closed, 2 hours before the meeting

NODE TYPE	NODE KEY
Data & utility	`data.adjust_time`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Shifts a date/time forward or back — 3 days after the close date, 2 working hours before the meeting.

When to use

- Compute a follow-up date, an SLA deadline, a reminder time for a Delay or a booking.

How it works

1. Reads **Starting from** (blank = now; a date without a time is read as midnight UTC), moves it by **amount** × **unit** in the chosen direction. Months / years move the calendar (31 Jan + 1 month = 28 Feb, 29 Feb + 1 year = 28 Feb).
2. **Working days / hours** skip weekends, holidays and out-of-hours time when the workspace has an active business calendar; with none set up they behave as plain days / hours rather than failing.
3. Saves an ISO string; with **Read it in** set, it carries that zone's offset.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Starting from `base_time`	Value or variable	No	—	e.g. `{{record.close_date}}` — blank means now
Move it `direction`	Choice	No	`add`	Options: `add` = Forward `subtract` = Back
By how much `amount`	Value or variable	Yes	—	e.g. `3`
Unit `unit`	Choice	No	`days`	Months and years move the calendar: 31 Jan + 1 month = 28 Feb. Working days / hours skip weekends, holidays and out-of-hours time on the workspace's business calendar — which is what an SLA or a follow-up date usually means

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `minutes` = Minutes `hours` = Hours `days` = Days `weeks` = Weeks `months` = Months `years` = Years `business_days` = Working days `business_hours` = Working hours
Read it in `timezone`	Time zone	No	—	e.g. `– UTC –`
Save the new time as `output_variable`	Text	No	`adjusted_time`	

Required fields

- **By how much** `amount`

Optional fields

- **Starting from** `base_time`
- **Move it** `direction` — default `add`
- **Unit** `unit` — default `days`
- **Read it in** `timezone`
- **Save the new time as** `output_variable` — default `adjusted_time`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.adjusted_time}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`adjusted_time`	The new date/time	text

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **By how much** must be filled in.
- By how much is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "'x' is not a date/time" / "'x' is not a number of business hours" (or the unit chosen) / "'x' is not one of minutes, ..." → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only or pure computation — safe to repeat. The exceptions (HTTP requests with a body, Find or create) say so on their page.

Transaction / rollback

Writes nothing to records.

Examples

Three working days after close

```
{
  "base_time": "{{record.close_date}}",
  "direction": "add",
  "amount": 3,
  "unit": "business_days",
  "timezone": "Asia/Kolkata",
  "output_variable": "followup_at"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "base_time": "{{record.close_date}} — blank means now",
  "direction": "add",
  "amount": "3",
  "unit": "days",
  "timezone": "Asia/Kolkata",
  "output_variable": "adjusted_time"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

Random number

Pick a number between two values — sampling, A/B splits, spreading work over a window

NODE TYPE	NODE KEY
Data & utility	`data.random_number`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Picks a number between two values — sampling, A/B splits, spreading work over a window.

When to use

- Send 10 % of contacts a survey; pick a random delay to smooth load.

How it works

1. Draws between **Smallest** and **Largest** (swapped if reversed) with **Decimal places** (0–6).

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Smallest `minimum`	Number	Yes	`1`	
Largest `maximum`	Number	Yes	`100`	
Decimal places `decimals`	Number	No	`0`	0 gives a whole number
Save the number as `output_variable`	Text	No	`random_number`	

Required fields

- **Smallest** `minimum`
- **Largest** `maximum`

Optional fields

- **Decimal places** `decimals` — default `0`

- **Save the number as** `output_variable` — default `random_number`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.random_number}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`random_number`	The number	number

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's	Always.

VARIABLE	WHAT IT IS	AVAILABLE
	Outputs).	
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Smallest** must be filled in.
- **Largest** must be filled in.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "the smallest and largest must be numbers" → failed.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts

back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Different every run, by design.

Transaction / rollback

Writes nothing to records.

Examples

10 % sample

Then a condition: `dice` ≤ 10.

```
{
  "minimum": 1,
  "maximum": 100,
  "decimals": 0,
  "output_variable": "dice"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "minimum": 1,
  "maximum": 100,
  "decimals": 0,
  "output_variable": "random_number"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- Smallest / Largest / Decimal places accept `{{…}}` placeholders; 0 decimals gives a whole number, more than 6 is capped at 6.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

Find or create

Look for a record, and make one if it isn't there — the usual way to stop a workflow creating duplicates

NODE TYPE	NODE KEY
Data & utility	`data.find_or_create`
MODULE	**OUTPUT PORTS**
core (always available)	found, created

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Looks for a record and makes one if it is not there — the usual way to stop a workflow creating duplicates.

When to use

- A webhook that may bring a known or a new contact; a company that may or may not exist yet.

How it works

1. Searches **Record type** with **Look for a record where** (Run as scoped). Found → the `found` port with the record as the variable.
2. Not found → creates it (contact / company / lead / deal through Create Record, a custom entity through Create Custom Record) with **If none exists, create it with** — the equals-values you searched on are filled in for you. Then the `created` port with `{id, …values}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Picker (searchable_entity)	Yes	—	Your own entities are in this list too
Look for a record where `conditions`	Filter rows	No	—	
Match `logic`	Choice	No	`and`	Options: `and` = ALL of these conditions `or` = ANY of these conditions
If none exists, create it with `field_mappings`	Rows of key / value	No	—	The matched-on values are filled in for you when you leave them out

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Save the record as `output_variable`	Text	No	`record`	

Required fields

- **Record type** `entity_type`

Optional fields

- **Look for a record where** `conditions`
- **Match** `logic` — default `and`
- **If none exists, create it with** `field_mappings`
- **Save the record as** `output_variable` — default `record`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.record}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

KEY	MEANING	TYPE
`id`	Id of what was created	id
`record_id`	Record id	id
`created`	True when it had to be made	boolean

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`found`	A record matched.
`created`	None matched, so one was made.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Record type** must be filled in.
- Record type and at least one match row with a field are required ("pick a record type and at least one thing to match on").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A create that is skipped (unsupported type) or fails passes that status and error to this step and nothing follows.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Converges — but not atomic: two runs at the same instant can both create the record.

Transaction / rollback

Writes nothing to records.

Examples

Contact by email

```
{
  "entity_type": "contact",
  "conditions": [
    {
      "field": "email",
      "operator": "eq",
```

```
      "value": "{{trigger.data.email}}"
    }
  ],
  "field_mappings": [
    {
      "key": "first_name",
      "value": "{{trigger.data.first_name}}"
    },
    {
      "key": "source",
      "value": "webhook"
    }
  ],
  "output_variable": "contact"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "<Record type>",
  "conditions": [],
  "logic": "and",
  "field_mappings": [],
  "output_variable": "record"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · integration_hub

Enrich record

Fill a contact or company from your connected data provider (Apollo, Clearbit) — tries each in turn, fills only empty fields, turns differences into suggestions and respects the monthly credit cap

NODE TYPE	NODE KEY
Data & utility	`data.enrich_record`

MODULE	OUTPUT PORTS
integration_hub (must be enabled)	enriched, not_found, cap_reached

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Fill a contact or company from your connected data provider (Apollo, Clearbit) — tries each in turn, fills only empty fields, turns differences into suggestions and respects the monthly credit cap

When to use

- To fetch, compute or reshape a value that a later step needs.

How it works

1. The engine reaches the step, resolves every `{{…}}` placeholder and `=formula` in its settings, runs it, records the result in the step log, then follows the outgoing wires.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Which record `target`	Choice	No	`trigger_record`	Options: `trigger_record` = The record that started this run `found_record` = A record found earlier
Record type `target_entity_type`	Choice	No	—	Options: `contact` = Contact `account` = Company Shown when target is found_record
Record id `target_record_id`	Text	No	—	Shown when target is found_record e.g. `{{variables.found.id}}`
Save the result as `output_variable`	Text	No	`enrichment`	

Required fields

None — every setting is optional, though the node does nothing useful until the main ones are filled in.

Optional fields

- **Which record** `target` — default `trigger_record`
- **Record type** `target_entity_type`
- **Record id** `target_record_id`
- **Save the result as** `output_variable` — default `enrichment`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.enrichment}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`found`	True when a provider knew the record	boolean
`provider`	Provider that answered	text
`applied_fields`	Fields filled in	list
`suggested_fields`	Fields left as suggestions	list
`profile`	Everything the provider returned	object

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`enriched`	—
`not_found`	Nothing matched.
`cap_reached`	—

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **integration_hub** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only or pure computation — safe to repeat. The exceptions (HTTP requests with a body, Find or create) say so on their page.

Transaction / rollback

Writes nothing to records.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "target": "trigger_record",
  "target_entity_type": "contact",
  "target_record_id": "{{variables.found.id}}",
  "output_variable": "enrichment"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}` .

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read` .

Related nodes

—

DATA & UTILITY · core

Log Message

Write a message to the execution log

NODE TYPE	NODE KEY
Data & utility	`data.log`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Writes a message to the server log for debugging a run.

When to use

- While building: print a resolved value to see what the run actually has.

How it works

1. Resolves **Message** and writes it at the chosen level to the application log ("[WF LOG] ..."). The step log records `logged: <the resolved message>`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Message `message`	Multi-line text	Yes	—	Shows on the run's step log — { } Insert works here
Level `level`	Choice	No	`info`	Options: `info` `warning` `error`

Required fields

- **Message** `message`

Optional fields

- **Level** `level` — default `info`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).

- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

This step saves nothing to variables; its raw output is at `{{nodes.<step id>.output}}`.

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Message** must be filled in.
- Message is required.
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only or pure computation — safe to repeat. The exceptions (HTTP requests with a body, Find or create) say so on their page.

Transaction / rollback

Writes nothing to records.

Examples

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "message": "<Message>",
  "level": "info"
}
```

Best practices

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}` .

Limitations

- Goes to the server log **and** the step's output shows the resolved message under `logged` , so a Log step is a cheap way to see a value in the Steps panel.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not

re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

DATA & UTILITY · core

Calculate

Work out a value with a formula — a date, an amount, a yes/no, some text — and keep it for later steps

NODE TYPE	NODE KEY
Data & utility	`data.calculate`

MODULE	OUTPUT PORTS
core (always available)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Works out a value with the same formula language as calculated fields — a date, an amount, a yes/no, a piece of text — and keeps it for later steps.

When to use

- A discounted price, a due date with WORKDAY(), a label built with IF(), a value from a linked record (`{account_id.industry}`).

How it works

1. Evaluates **Formula** against the record, its links, `{$trigger.x}` and `{$var.x}` values by the formulas engine, typed per **result type**, and saves the answer. An `#ERR` value fails the step — never a silent 0.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Formula `formula`	Formula	Yes	—	Same formulas as calculated fields: {amount} * 0.9, WORKDAY(TODAY(), 3), IF({discount} > 15, "Review", "OK"). Use {$var.name} for an earlier step's answer.
Save the answer as `output_variable`	Text	No	`result`	

Required fields

- **Formula** `formula`

Optional fields

- **Save the answer as** `output_variable` — default `result`

Inputs

- Its own settings, with placeholders resolved from the run (trigger payload, record, variables, earlier steps).
- The run context: the triggering record (if any), the Run as user, and whether this is a test run.

Outputs

The result is saved as `{{variables.result}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

Execution behaviour

Resolves its settings, computes or fetches the value, and saves it under the variable name you give ("Save the result as"). Nothing on a record changes (Find or create is the exception: it creates a record when none matches).

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

If this step's **On error** port is wired, the run carries on down that wire with the error text in `{{nodes.<step id>.error}}` (first 1000 characters). Otherwise, when the workflow's **Stop on error** is on (the default) the run is marked **Failed** with "Failed at node <id>: <error>" and nothing else runs. With Stop on error off, only this branch ends and the rest of the run continues.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.

VARIABLE	WHAT IT IS	AVAILABLE
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- Nothing beyond the automation engine itself.

Validation rules

- **Formula** must be filled in.
- Formula is required ("The Calculate step has no formula").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "<#ERR>: <message>" → failed (takes On error).

Retry behaviour

Every step that reaches outside the engine (actions, sends, AI, tools, HTTP Request, Call API, sub-workflows) has its own **Retry when it fails** setting: *only a temporary problem* (the default — timeouts, connection errors, HTTP 5xx, rate limits, whether raised or reported), *any failure*, or *never*, and **Retry at most** (blank = the workflow's Max retries, default 3). Attempts back off 0.5 s, 1 s, 2 s, 4 s, then 8 s; each retry writes an audit row and the step log keeps the retry count. A **Failed** run can be

retried by hand from its failed step (needs `workflows.manage`); after three such retries it is quarantined, listed under **Needs attention** on Execution Logs, and replayed from there.

Timeout

No limit on this step itself. The whole run must finish inside the workflow's **Time limit** (default 60 minutes), which is checked between steps — never in the middle of one — and counts **active** time only: time spent paused on a delay, an approval or a wait is not charged, so a run resumed days later keeps the budget it had. A run also fails after 2000 steps.

Idempotency

Read-only or pure computation — safe to repeat. The exceptions (HTTP requests with a body, Find or create) say so on their page.

Transaction / rollback

Writes nothing to records.

Examples

Due date, three working days out

```
{
  "formula": "WORKDAY(TODAY(), 3)",
  "output_variable": "due"
}
```

Review flag

```
{
  "formula": "IF({discount} > 15, \"Review\", \"OK\")",
  "output_variable": "review"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "formula": "<Formula>",
  "output_variable": "result"
}
```

Best practices

✓ Reference an earlier step's answer as `{$var.name}`.

✓ Name the output variable something a colleague will understand — `{{variables.open_ticket_count}}` beats `{{variables.result}}`.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on.

Security considerations

The step runs with the workflow's **Run as** identity (default: the workflow owner; "system" needs `workflows.run_as_system`). Record reads and writes to records the run *found* are limited to what that user may see; the triggering record itself is not re-checked. Anything you write into settings — including secrets in headers or messages — is stored in the workflow definition and, once resolved, in the step log, which is visible to anyone with `workflows.read`.

Related nodes

AI · ai_orchestration

AI Generate Text

Generate text using AI (via orchestration layer)

NODE TYPE	NODE KEY
AI	`ai.generate_text`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Writes text from a prompt — a draft reply, a summary line, a personalised paragraph.

When to use

- Draft the first reply to a ticket for an agent to review; write a personalised opener for an email.

How it works

1. Resolves the prompt (placeholders inject run data), sends it as a drafting task with **Length limit** tokens, saves the text.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Prompt `prompt`	Multi-line text	Yes	—	
Length limit `max_tokens`	Number	No	`500`	Roughly 4 characters per token — 500 is about 350 words
Save answer as variable `output_variable`	Text	No	`ai_text`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	`Number`	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Prompt** `prompt`

Optional fields

- **Length limit** `max_tokens` — default `500`
- **Save answer as variable** `output_variable` — default `ai_text`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.ai_text}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Prompt** must be filled in.
- Prompt is required ("AI Generate Text has no prompt").

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Draft a reply

```
{
  "prompt": "Write a short, friendly first reply to this support request. Do not promise a fix date.\n\nSubject
  "max_tokens": 300,
  "output_variable": "draft"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "prompt": "<Prompt>",
  "max_tokens": 500,
  "output_variable": "ai_text",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Fill fields

Read some text and propose values for a record's fields — the entity's OWN fields, so nothing invented can be written

NODE TYPE	NODE KEY
AI	`ai.fill_fields`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Reads some text and proposes values for a record's **own** fields — nothing invented can be written, and nothing is written by this step itself.

When to use

- Turn an inbound email or a form answer into contact / deal fields; then Create Record or Update with `{{variables.filled.<field>}}`.

How it works

1. Reads the entity's field list, asks the model for a JSON object limited to those fields, drops unknown keys and empty values, saves the object.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Record type `entity_type`	Picker (searchable_entity)	Yes	—	Which record's fields to fill — your own entities too
Read this `input`	Variable reference	Yes	—	e.g. `{{trigger.body}} — an email, a note, a form answer`
Anything else it should know `instruction`	Multi-line text	No	—	e.g. `e.g. the company name is in the signature`
Save the values as `output_variable`	Text	No	`filled`	Hand them to Create Record as {{variables.filled.<field>}}
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Record type** `entity_type`
- **Read this** `input`

Optional fields

- **Anything else it should know** `instruction`
- **Save the values as** `output_variable` — default `filled`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.filled}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Record type** must be filled in.
- **Read this** must be filled in.
- Record type and Read this are required ("AI Fill has no text to read").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "AI Fill: that record type has no fields I can read" → failed.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Contact from an email

Then Create Record with `{{variables.filled.first_name}}` etc.

```
{
  "entity_type": "contact",
  "input": "{{trigger.body}}",
  "instruction": "The company name is in the signature.",
  "output_variable": "filled"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "entity_type": "<Record type>",
  "input": "{{variables.something}}",
  "instruction": "e.g. the company name is in the signature",
  "output_variable": "filled",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

- ✓ Review before writing — pair with Ask a person for anything customer-facing.
- ✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.
- ✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Router

Let AI choose which path the run takes — describe the paths in plain words and it picks one, with its reason

NODE TYPE	NODE KEY
AI	`ai.route`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	<one per choice>, default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Lets the model choose which path the run takes — describe the paths in plain words and it picks one, with its reason and confidence.

When to use

- Route tickets to Sales / Support / Billing by their content; triage leads by fit.

How it works

1. Sends the question, the input (blank = the whole record, first 4000 characters) and the **Paths**; the model answers `{choice, why, confidence}`. The matching path's port (by slug, case-insensitive) is taken; no recognisable match → the first path (logged); an unwired path → `default`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
What to decide `question`	Multi-line text	No	—	e.g. `e.g. Which team should pick this up?`
Decide about `input`	Variable reference	No	—	Leave blank to use the record this run is about
Paths `choices`	List of names	Yes	—	Each one gets its own path on the canvas. The choice and the model's reason land in {{variables.<name>.choice}} / .why e.g. `Sales, Support, Billing`
Save the decision as `output_variable`	Text	No	`route`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
				doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Paths** `choices`

Optional fields

- **What to decide** `question`
- **Decide about** `input`
- **Save the decision as** `output_variable` — default `route`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.route}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`choice`	The path it picked	text
`why`	Why it picked that	text
`confidence`	How sure it was (0-1)	number

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With

`automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`<one per choice>`	The path the model picked.
`default`	The picked path has nothing wired.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.

VARIABLE	WHAT IT IS	AVAILABLE
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Paths** must be filled in.
- At least two paths ("AI Router needs at least two paths to choose between").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Triage tickets

`{{variables.route.why}}` explains the pick.

```
{
  "question": "Which team should handle this?",
  "input": "{{record.description}}",
  "choices": [
    "Sales",
    "Support",
    "Billing"
  ],
  "output_variable": "route"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "question": "e.g. Which team should pick this up?",
  "input": "{{variables.something}}",
  "choices": [],
  "output_variable": "route",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

- ✓ Wire every path and Other; branch on `confidence` with a condition when a wrong pick is costly.
- ✓ Leave **Save the decision as** set (default `route`) — the pick is read back from that variable.
- ✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.
- ✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current

step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Classify

Classify text into categories using AI

NODE TYPE	NODE KEY
AI	`ai.classify`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Puts text into exactly one of your categories, with a confidence.

When to use

- Label tickets, notes or survey comments; then branch with a Switch on `.category`.

How it works

1. Sends the text and the labels; saves `{category, confidence}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Text to classify `input`	Variable reference	Yes	—	
Categories `categories`	List of names	Yes	—	Separate the labels with commas — the AI picks exactly one. e.g. `urgent, billing, other`
Save answer as variable `output_variable`	Text	No	`ai_class`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Text to classify** `input`
- **Categories** `categories`

Optional fields

- **Save answer as variable** `output_variable` — default `ai_class`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.ai_class}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`category`	Category picked	text
`confidence`	Confidence (0-1)	number

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Text to classify** must be filled in.
- **Categories** must be filled in.
- Text is required ("Classify has no text to work on") and at least two categories ("AI Classify needs at least two category labels").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Ticket category

Then Switch on `{{variables.kind.category}}`.

```
{
  "input": "{{record.description}}",
  "categories": [
    "billing",
    "technical",
    "sales",
    "other"
```

```
  ],
  "output_variable": "kind"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "input": "{{variables.something}}",
  "categories": [],
  "output_variable": "ai_class",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Sentiment Analysis

Analyze sentiment of text

NODE TYPE	NODE KEY
AI	`ai.sentiment`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Rates text as positive, neutral or negative with a score from −1 to 1.

When to use

- Flag angry tickets or comments for a human.

How it works

1. Sends the text; saves `{sentiment, score}`.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Text to analyse `input`	Variable reference	Yes	—	
Save answer as variable `output_variable`	Text	No	`ai_sentiment`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Text to analyse** `input`

Optional fields

- **Save answer as variable** `output_variable` — default `ai_sentiment`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.ai_sentiment}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

KEY	MEANING	TYPE
`sentiment`	positive / neutral / negative	text
`score`	Score (-1 to 1)	number

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Text to analyse** must be filled in.
- Text is required ("Sentiment has no text to work on").

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Escalate negative tickets

Then a condition: `mood.score` < −0.5.

```
{
  "input": "{{record.description}}",
  "output_variable": "mood"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "input": "{{variables.something}}",
  "output_variable": "ai_sentiment",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Extract Data

Extract structured data from text using AI

NODE TYPE	NODE KEY
AI	`ai.extract`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Pulls structured fields out of free text into JSON.

When to use

- Order numbers, dates, amounts, company names from an email.

How it works

1. **Fields to extract** is a JSON object, list or comma list; the model answers JSON in that shape with null for absent fields; one variable key per field.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Text to read `input`	Variable reference	Yes	—	
Fields to extract `extraction_schema`	JSON	Yes	—	
Save answer as variable `output_variable`	Text	No	`ai_fields`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Text to read** `input`
- **Fields to extract** `extraction_schema`

Optional fields

- **Save answer as variable** `output_variable` — default `ai_fields`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.ai_fields}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`. One key per entry in the step's **Fields to extract** setting.

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Text to read** must be filled in.
- **Fields to extract** must be filled in.

- Text is required ("Extract has no text to work on") and Fields to extract ("AI Extract Data has no fields to extract").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Order details

```
{
  "input": "{{trigger.body}}",
  "extraction_schema": {
    "order_number": "",
    "amount": "",
    "delivery_date": ""
  },
  "output_variable": "order"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "input": "{{variables.something}}",
  "extraction_schema": {},
  "output_variable": "ai_fields",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Summarize

Summarize text using AI

NODE TYPE	NODE KEY
AI	`ai.summarize`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Condenses text to at most N words.

When to use

- A ticket thread summary for the assignee; a call transcript into a note.

How it works

1. Sends the text with **Max words** (10–2000, default 200); saves the summary text.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Text to summarize `input`	Variable reference	Yes	—	
Max words `max_length`	Number	No	`200`	
Save summary as variable `output_variable`	Text	No	`ai_summary`	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Text to summarize** `input`

Optional fields

- **Max words** `max_length` — default `200`
- **Save summary as variable** `output_variable` — default `ai_summary`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.ai_summary}}` (change the name in "Save the result as"), and the raw step output is always at `{{nodes.<step id>.output}}`.

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}} / loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}} / workflow_version / dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Text to summarize** must be filled in.
- Text is required ("AI Summarize has no text to summarize").

- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Note on the record

Then Create Activity with the summary.

```
{
  "input": "{{record.description}}",
  "max_length": 80,
  "output_variable": "summary"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "input": "{{variables.something}}",
  "max_length": 200,
  "output_variable": "ai_summary",
  "retry_on": "transient",
```

```
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_orchestration

AI Prompt

Run a prompt template through the tenant's AI route (in-process orchestration or external AI Platform). Output is stored under the node's output and, if output_variable is set, promoted to workflow variables so later nodes can reference it.

NODE TYPE	NODE KEY
AI	`ai.execute_prompt`

MODULE	OUTPUT PORTS
ai_orchestration (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Runs a stored prompt template through the workspace's AI route with your variables, in text, JSON or Markdown.

When to use

- A prompt maintained by an admin under AI → Prompt templates, reused by many workflows.

How it works

1. Loads the template by code, adds **Prompt variables** (the trigger payload `{{trigger.*}}` and the entity reference `{{entity.type}}` / `{{entity.id}}` are added automatically; pass record fields as variables yourself), runs it as the chosen **task type** with the given format, token limit and temperature, and saves the parsed result (JSON is parsed).

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Prompt template `prompt_code`	Record picker (ai_prompt_template)	Yes	—	
task type `task_type`	Choice	No	`drafting`	Options: `summarization` `drafting` `classification` `recommendation` `extraction` `enrichment` `search`
Prompt variables `variables`	Rows of key / value	No	—	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
output format `output_format`	Choice	No	`text`	Options: `text` `json` `markdown`
max tokens `max_tokens`	Number	No	`1000`	
temperature `temperature`	Number	No	`0.3`	
Store output in variable `output_variable`	Text	No	—	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Prompt template** `prompt_code`

Optional fields

- **task type** `task_type` — default `drafting`
- **Prompt variables** `variables`
- **output format** `output_format` — default `text`
- **max tokens** `max_tokens` — default `1000`
- **temperature** `temperature` — default `0.3`
- **Store output in variable** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The text or record you point it at, resolved from the run.
- The workspace's AI routing mode and rate limit.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_orchestration** module must be enabled for the workspace; otherwise the node is not offered in the palette.

Validation rules

- **Prompt template** must be filled in.
- Prompt template is required in the designer (the engine itself does not check it).
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- A **skipped** result (a setting missing, a record not found, nothing to do) keeps its reason in the step log and the run still continues down the normal wires.
- A **failed** result or an exception takes the On error wire if there is one; otherwise Stop on error decides whether the run fails.
- Errors from a formula in a setting (`=…` / `{= … }`) fail the step with the formula's message before the step runs.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

The provider's own request timeout applies; the run's Time limit is checked between steps.

Idempotency

Every run is a fresh model call and may return a different answer. Nothing is written to records — pair it with an action step for that.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Renewal summary as JSON

```
{
  "prompt_code": "renewal_summary",
  "task_type": "summarization",
  "variables": {
    "account": "{{record.name}}"
  },
  "output_format": "json",
  "max_tokens": 800,
  "temperature": 0.2,
  "output_variable": "renewal"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "prompt_code": "<ai_prompt_template id>",
  "task_type": "drafting",
  "variables": [],
  "output_format": "text",
  "max_tokens": 1000,
  "temperature": 0.3,
  "output_variable": "<Store output in variable>",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The text you send leaves the platform for the configured provider. Do not put credentials or unrelated personal data into prompts. The answer is stored in the step log and variables. AI steps **run for real in a Test run** and count against the AI rate limit.

Related nodes

AI · ai_agents

AI Agent (Tool-using)

Hand the problem to an autonomous agent with a scoped MCP session. The agent plans, calls tools via the Integration Hub, observes results, and returns a structured output. Each tool call is logged as a sub-step under this node.

NODE TYPE	NODE KEY
AI	`ai.agent_reasoning`

MODULE	OUTPUT PORTS
ai_agents (must be enabled)	default

Purpose · When to use · How it works · Configuration · Required · Optional · Inputs · Outputs · Execution · Success · Failure · Branches · Data mapping · Permissions · Dependencies · Validation · Errors · Retry · Timeout · Idempotency · Rollback · Examples · Best practices · Limitations · Audit · Security · Related

Purpose

Hands a problem to an autonomous agent that plans, calls tools through the Integration Hub, observes and returns a structured answer — each tool call logged as a sub-step.

When to use

- Multi-step research or reconciliation a fixed sequence cannot express ("find the invoice, compare with the CRM amount, draft the discrepancy note").

How it works

1. Starts the agent with the instruction, the run's variables, record and trigger as context, limited to **Allowed tools**, **Max agent planning steps** (default 8) and **timeout_seconds** (default 120). Every tool call goes through the execution gateway and is audited. A tool that needs approval parks the run as Waiting.

Configuration

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
Agent `agent_id`	Record picker (ai_agent_definition)	Yes	—	
Instruction to agent `input_message`	Multi-line text	No	—	
Allowed tools (subset of tenant's registered tools) `allowed_tools`	Record picker, many (integration_tool)	No	—	
Max agent planning steps `max_steps`	Number	No	`8`	
timeout seconds `timeout_seconds`	Number	No	`120`	

SETTING	TYPE	REQUIRED	DEFAULT	DETAILS
output variable `output_variable`	Text	No	—	
Retry when it fails `retry_on`	Choice	No	`transient`	How many times is the workflow's Max retries unless set below; the wait doubles from 0.5 s up to 8 s between attempts Options: `transient` = Only a temporary problem (timeout, connection, HTTP 5xx, rate limit) `any_failure` = Any failure `never` = Never
Retry at most (times) `retry_attempts`	Number	No	—	e.g. `blank = the workflow's Max retries`

Required fields

- **Agent** `agent_id`

Optional fields

- **Instruction to agent** `input_message`
- **Allowed tools (subset of tenant's registered tools)** `allowed_tools`
- **Max agent planning steps** `max_steps` — default `8`
- **timeout seconds** `timeout_seconds` — default `120`
- **output variable** `output_variable`
- **Retry when it fails** `retry_on` — default `transient`
- **Retry at most (times)** `retry_attempts`

Inputs

- The instruction and the run context.
- The agent definition and the allowed tool subset.

Outputs

The result is saved as `{{variables.<the name you give>}}`, and the raw step output is always at `{{nodes.<step id>.output}}`. Its shape depends on the far end — the designer adds keys it saw in the last test run.

Execution behaviour

Builds a prompt from the settings, sends it through the workspace's AI route (in-process orchestration by default, or the external AI Platform when configured — with automatic fallback to in-process), parses the answer and saves it. The model is chosen by the workspace's AI feature routes, not per step. Emits AI-action started / completed / failed events. With `automation/external_jobs` on, the model call runs on the AI job worker (see the Overview) and the run waits for it; AI Router and AI Agent always run inline.

Success path

Every outgoing wire except **On error** is followed. A step with nothing wired after it simply ends its branch; the run completes once no queued step is left.

Failure path

A provider error or a rate-limit hit raises "AI action failed: …" — the step fails and On error / Stop on error apply.

Branches

PORT	TAKEN WHEN
`default`	The normal continuation after the step succeeds.

Data mapping

Any setting can contain variables in the `{{namespace.path}}` form. A missing value resolves to an empty string; a list or object comes out as text. There are no filters or conditionals — use Data Transform, Calculate or Set Variable to shape a value first. Action settings also accept formulas: `=` for a whole value, `{= … }` inside text.

VARIABLE	WHAT IT IS	AVAILABLE
`{{trigger.<key>}}`	A key from the event payload that started the run (see each trigger's **Outputs**).	Always.
`{{record.<field>}}`	Any column of the triggering record, with the trigger keys laid over it.	When the canvas has a condition node, or any setting mentions `record.`.
`{{entity_fields.<column>}}`	The same record, the older name.	Same rule as `record`.
`{{entity.type}} / {{entity.id}}`	The record type and id the run is about.	Always (empty for runs with no record).
`{{variables.<name>}}`	A value an earlier step saved ("Save the result as"), or a workflow-level default.	After the step that sets it has run.
`{{nodes.<step id>.output}}`	The raw result of a specific earlier step; `.error` after an On error wire.	After that step has run.

VARIABLE	WHAT IT IS	AVAILABLE
`{{variables.loop_item}}` / `loop_index`	The current item and its position inside a Loop body.	Inside a loop scope only.
`{{env.workflow_name}}` / `workflow_version` / `dry_run`	Workflow metadata.	Always.

Permissions

Browsing this reference and the node catalog needs `workflows.read`. Editing the workflow needs ownership (or `workflows.manage_all`); publishing also needs `workflows.publish`. At run time the step acts as the workflow's **Run as** user for record lookups and writes to found records; it does not check that user's role permissions for the action itself.

Dependencies

- The **ai_agents** module must be enabled for the workspace; otherwise the node is not offered in the palette.
- AI Agents enabled; an agent definition; Integration Hub tools registered for the workspace.

Validation rules

- **Agent** must be filled in.
- Agent is required ("agent_reasoning: agent_id is required" / "invalid agent_id").
- Validate (in the designer) reports these before the workflow can be published.

Error handling

- "Agent reasoning failed: <error>" → failed.

Retry behaviour

Not retried automatically: the AI bridge turns every provider error (connection, timeout, 5xx, rate limit) into a plain failure, which the workflow's Max retries does not cover. A rate-limit error names the retry-after seconds in its message; re-run the step by hand (Retry) when the cause was temporary.

Timeout

timeout_seconds (default 120) for the whole agent run.

Idempotency

Not idempotent — tools may write again on a re-run.

Transaction / rollback

Writes nothing to records. Tokens used are not refunded.

Examples

Reconcile an invoice

```
{
  "agent_id": "<agent id>",
  "input_message": "Find invoice {{trigger.invoice_number}} in the accounting system and compare its total with
  "allowed_tools": [
    "accounting.invoices.get"
  ],
  "max_steps": 6,
  "timeout_seconds": 90,
  "output_variable": "reconciliation"
}
```

Every setting, with placeholder values

Generated from the node's schema — a starting point, not a recommendation.

```
{
  "agent_id": "<ai_agent_definition id>",
  "input_message": "<Instruction to agent>",
  "allowed_tools": [],
  "max_steps": 8,
  "timeout_seconds": 120,
  "output_variable": "<output_variable>",
  "retry_on": "transient",
  "retry_attempts": 0
}
```

Best practices

✓ Save the answer as a variable and branch on it with a condition; do not let free text drive a write without a check.

✓ AI steps run for real in a Test run — expect token usage.

Limitations

- No known restrictions beyond the engine-wide ones in the Overview.

Audit / logging

The step log (Run → Steps) records the step id, type and label, status (running / success / failed / skipped / waiting), start and end time, duration, the step's output and any error message. The run itself keeps the trigger payload, who started it, current step, saved variables and idempotency key. Run-level audit rows are always written (started, resumed, retried, completed / failed / cancelled); per-step audit rows only when the workspace setting `audit/workflow_step_level_enabled` is on. The output also records route, provider, model, generation id, trace id, token counts and latency.

Security considerations

The agent can call every allowed tool with the run's data; keep **Allowed tools** minimal and prefer read-only scopes. Tool inputs and outputs are audited by the gateway. Runs for real in a Test run.

Related nodes